SIMATS
ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ECA09-DIGITAL SIGNAL PROCESSING LAB MANUAL

Software Experiments Using SCILAB:

Expt 1	**Generation of Common Discrete Time Signals using Scilab**
Test case 1	Generate and plots delayed unit step signal u(n−10) , delayed unit impulse signal δ(n - 5), using Scilab
Test case 2	Generate a sinusoidal signal with frequency f=5Hz, sampling frequency fs=50Hz, for n=0 to n=99 using Scilab
Test case 3	Generate an exponential signal $x(n)=e^{0.1n}$ for n=0 to n=10. using Scilab
Test case 4	Generate and analyze a delayed unit ramp signal using SCILAB, with a specific delay d, and to plot the resulting signal to observe the effects of the delay on the signal's behavior.
Expt 2	**Digital communication using binary phase Shift keying**
Test case 1	Generate a Binary Phase Shift Keying (BPSK) signal with the given input binary signal and sampling frequency F_s = 100Hz using SCILAB code and get the output waveform
Test case 2	Generate a matrix that consists of eight received baseband signals with a phase that increases from 0° to 180° in increments of 22.5°.as a phase of 0, 22.5°, 45°, 67.5°, 90°, 112.5°, 135°, 157.5°, or 180°.Provide the two-dimensional plot of all the signals
Test case 3	plot the constellation diagram for a Binary Phase Shift Keying (BPSK) modulation system using SCILAB
Test case 4	Generate a random binary sequence of 10 bits, modulate it using BPSK, transmit the signal, and then demodulate it to retrieve the original binary sequence. Show, the BPSK modulated signal, and the demodulated binary sequence.
Expt 3	**N point DFT sequence, N point IDFT sequence, Circular convolution**
Test case 1	Find the Circular Convolution of two sequences x1 = [2 ,1 ,2 ,1],x2 = [1 ,2 ,3 ,4] using Matrix method in Scilab code and obtain the convolution matrix for given input sequences
Test case 2	Compute the 8-point DFT and IDFT of the sequence (x[n] = [1, 2, 3, 4, 5, 6, 7, 8]
Test case 3	Determination of N-point DFT and Plot its magnitude and phase spectrum using scilab
Test case 4	Compute the 8-point Discrete Fourier Transform (DFT) and Inverse Discrete Fourier Transform (IDFT) of the sequence (x[n] = [10, 12, 14, 16]) using Scilab, with zero-padding to avoid time-domain aliasing.
Expt 4	**Sectional Convolution**
Test case 1	Verify the correctness of your implementation of the overlap-save method.Input Signal: x[n]=[3,0,−2,0,2,1,0,−2,−1,0],Impulse Response: h[n]=[2,2,1], FFT Length: $L=2^M=2^3=8$. Use the overlap-save method and verify the result against direct convolution.

Test case 2	Input Signal: x[n]=[1,2,−1,2,3,−2,−3,−1,1,1,2,−1], Impulse Response: h[n]=[1,2] Compute the convolution of x[n] with h[n] using the overlap-add method. Compare this with the output obtained using the conv() function in SCILAB to verify the correctness.
Test case 3	Input Signal: x[n]=[1,2,3,4,5,6,7,8,9,10], Impulse Response: h[n]=[1,−1,1], FFT Length: Calculate based on the length of h[n]. Use the overlap-save method to compute the convolution and compare with the result from direct Convolution.
Test case 4	Input Signal: x[n]=[4,0,−4,0,4,0,−4], Impulse Response: h[n]=[1,1], FFT Length: Based on the impulse response length. Use the overlap-add method and verify the result against direct convolution.
Expt 5	**DIT-FFT and DIF-FFT Algorithm**
Test case 1	Given a sequence x[n]=[1,-1,-1,-1,1,1,1,-1] compute the DFT using the DIT-FFT algorithm. The sequence exhibits real and symmetric properties. How does symmetry affect the twiddle factor calculations in the DIT-FFT algorithm?
Test case 2	Verify the FFT computation for a basic sequence [1, 2, 3, 4, 5, 6, 7, 8] using the DIT-FFT algorithm.Verify that the magnitude and phase of the FFT result for the sequence.
Test case 3	Given a sequence x[n]=[1,2,3,4,4,3,2,1] compute the DFT using the DIF-FFT algorithm. The sequence exhibits real and symmetric properties. How does symmetry affect the twiddle factor calculations in the DIF-FFT algorithm?
Test case 4	Given a sequence x[n]=[3,4,0,0,−4,−3,0,0] compute the DFT using the DIF-FFT algorithm. Verify that the magnitude and phase of the FFT result for the sequence.
Expt 6	**Denoising Data using FFT**
Test case 1	Generated as noisy signal = sin(2*t) + 0.5*rand(t), where the time vector t is from 0 to 2π with an interval of 0.001. The signal is denoised using FFT with a threshold value of 50 in the frequency domain using the Fast Fourier Transform (FFT) in SCILAB.
Test case 2	Compare the time-domain plots of the noisy signal and the denoised signal. Does the FFT-based denoising preserve the amplitude and phase of the original sinusoidal signal while removing the noise? Quantify the error by calculating the difference between the original clean sinusoid (sin(2*t)) and the denoised signal.
Test case 3	Perform a frequency domain analysis of the denoised signal using FFT. How does the frequency spectrum of the denoised signal compare to the original noisy signal? Identify the frequencies that were most affected by the thresholding operation.
Test case 4	A noisy signal is generated as: x(t)=cos(5t)+cos(10t)+0.3·where the time vector ranges from 0 to 2π with an interval of 0.001The signal is denoised using the Fast Fourier Transform (FFT) by applying a threshold value of 30 in the frequency domain.
Expt 7	**Analog Filter design Using Transformation method**

Test case 1	Validate Transformation with Standard Sampling Period T=1.Convert Analog transfer function into Digital using Bilinear Transformation of H(s)=(s^2+4.525)/(s^2+0.692*s+0.504) using sci lab
Test case 2	Validate Transformation with Standard Sampling Period T=1.Convert Analog transfer function into Digital using Bilinear Transformation of H(s) = (s^2 + 2s + 1) / (s^2 + 3s + 2) using sci lab
Test case 3	To convert the continuous-time analog transfer function H(S)=10/(s^2+7*s+10) into a discrete-time digital transfer function using Impulse Invariant transformation method with a standard sampling period T=0.2
Test case 4	Validate Transformation with Standard Sampling Period T=1.Convert Analog transfer function into Digital using Impulse Invariant Transformation of H(s)=5/s2+4s+4 using sci lab
Expt 8	**Analog Butterworth Filter**
Test case 1	Design a Butterworth filter to process audio signals that attenuates frequencies above 0.2 * pi and maintains a flat frequency response in the passband. Compare the output of the filtered signal with the original signal in both time and frequency domains. Plot both to verify the attenuation of high frequencies.
Test case 2	Design a Butterworth filter that allows frequency range above the cut off frequency of 0.2*pi for a digital audio processing application.
Test case 3	For a signal processing application design a Butterworth filter to isolate a specific frequency range from an audio signal between 0.4 pi and 0.6 pi.
Test case 4	For a signal processing application design a Butterworth filter to attenuate a specific frequency range from an audio signal between 0.4 pi and 0.6 pi.
Expt 9	**Design of Analog Chebyshev Filter**
Test case 1	● Evaluate the provided SCILAB code for designing a Chebyshev Type I low-pass filter with the following parameters **Analog Passband Edge Frequency:** 1000π radians/second ● **Analog Stopband Edge Frequency:** 2000π radians/second ● **Passband Ripple:** -1 dB ● **Stopband Attenuation:** -40 dB
Test case 2	Given the following parameters for a Chebyshev Type I high-pass filter design: ● Analog Passband Edge Frequency (ωp\omega_pωp) = 1000×π radians/second ● Analog Stopband Edge Frequency (ωs\omega_sωs) = 2000×π radians/second ● Passband Ripple (δ1) = -1 dB ● Stopband Attenuation (δ2) = -40 dB 1. Calculate the passband and stopband deviations (δ1 and δ2) in linear scale. 2. Compute the δ and ϵ parameters for the filter. 3. Determine the filter order NNN required to meet the specified design parameters. 4. Using the calculated filter order, find the poles of the Chebyshev Type I high-pass filter. 5. Display the filter order and the poles.

Test case 3	For a band-pass filter design, given passband edge frequencies fp1 = 1.6, fp2 = 1.8, stopband edge frequencies fs1 = 3.2, fs2 = 4.8, passband ripple Ap = 2 dB, stopband attenuation As = 20 dB, and sampling frequency S = 12 kHz, calculate the filter order n, pre-warping frequencies W, and the transfer function HPS.
Test case 4	Design an analog band-stop Chebyshev filter with passband edge frequencies omegap1 = 900π rad/sec, omegap2 = 1100π rad/sec, and a stopband edge frequency omegas = 1000π rad/sec. Given passband ripple of -1 dB and stopband attenuation of -40 dB, calculate the filter order N, cutoff frequency omegac1, and the poles of the band-reject Chebyshev filter
Expt 10	**Design of FIR filter**
Test case 1	Design an FIR filter and plot the Frequency response for the low-pass filter (LPF) with a filter length N = 7, cutoff frequency (fc = 1000 Hz) and sampling frequency F = 5000 Hz using the rectangular method in Scilab.
Test case 2	Design an FIR filter and plot the Frequency response and calculate filter coefficients for the high-pass filter (HPF) with a filter length N = 11 using the hamming window method in Scilab.
Test case 3	Design an FIR filter and plot the frequency response for the high-pass filter (HPF) and calculate cutoff frequency with a filter length N = 11 using the hanning window method in Scilab. And also to plot the filter coefficients h(n) generated by the program.
Test case 4	Write a SciLab program that designs a high-pass FIR filter using a Blackman window and calculate cutoff frequency with a filter length N=11. The program should also be able to compute the filter's impulse response using the Blackman window, plots the magnitude response in decibels (dB), and displays the filter coefficients in the console
Expt 11	**Design of Band Pass filter using Fourier series Method**
Test case 1	Design an ideal band-pass filter using the Fourier series method for a passband centered at 2000 Hz and a sampling frequency of 9600 Hz. The filter coefficients are to be computed and the performance is to be verified.
Test case 2	Design an ideal band-pass filter in SCILAB using the Fourier series method with a lower cutoff frequency of 1800 Hz, upper cutoff frequency of 2200 Hz, and a sampling frequency of 9600 Hz, and to plot the magnitude response of the filter.
Test case 3	Design an ideal band-pass filter in SCILAB using the Fourier series method with a lower cutoff frequency of 1800 Hz and upper cutoff frequency of 2200 Hz, with a sampling frequency of 9600 Hz, and to plot the impulse response of the filter.
Test case 4	Design an ideal band-pass filter using the Fourier series method with the following specifications: lower cutoff frequency fL=1500f_L = 1500fL=1500 Hz, upper cutoff frequency fU=2500f_U = 2500fU=2500 Hz, and sampling frequency fs=10000f_s = 10000fs=10000 Hz.

Expt 12	**Multirate Signal Processing**
Test case 1	Analyze the effects of multirate signal processing techniques on sinusoidal signals through the processes of downsampling (decimation) for sampling rate of 1 KHz.
Test case 2	design and implement downsampling of a sinusoidal signal using Multirate Signal Processing techniques, and to observe the effects of both decimation and interpolation on the signal.
Test case 3	To develop a comprehensive adaptive sampling system to analyze the effects of down sampling (decimation) on a sinusoidal signal sampled at 2 kHz**,** and observe how the spectral and time-domain characteristics are affected by these operations.
Test case 4	To develop a comprehensive adaptive sampling system using SCILAB to analyze the effects of up sampling (interpolation) on a sinusoidal signal sampled at 5 kHz, and observe the changes in the time-domain of the signal when upsampling factors of 2 and 4 are applied
Expt 13	**Generation of Waveforms using DSP Processor**
Test case 1	Using a DSP processor, generate a sinusoidal waveform with the following parameters:Amplitude = 1 V,Frequency = 1 kHz,Sampling Frequency = 8 kHz,Number of samples = 1000. 1. Write a DSP program to generate the sine wave. 2. Plot showing the sine wave with proper labeling of amplitude and time. 3. Oscilloscope screenshot to verify waveform accuracy.
Test case 2	Generate a square waveform with the following parameters:Amplitude = 2 V,Frequency = 500 Hz,Duty Cycle = 50%,Sampling Frequency = 10 kHz 1.Write a DSP program to generate square waveform 2.Oscilloscope screenshot showing the generated square wave. 3. Calculated rise and fall times.
Test case 3	Using a DSP processor, generate a triangular waveform with the following parameters:Peak-to-peak amplitude = 3 V,Frequency = 2 kHz,Sampling Frequency = 20 kHz 1.Write a DSP program to generate triangular waveform. 2.Oscilloscope output verifying the triangular wave. 3. Error percentage in amplitude and frequency.
Test case 4	Generate a composite waveform by adding a sine wave and a square wave with the following parameters:Sine wave: Amplitude = 1 V, Frequency = 1 kHz.,Square wave: Amplitude = 0.5 V, Frequency = 2 kHz, Duty Cycle = 50%,Sampling Frequency = 20 kHz. 1.write a DSP program code for composite waveform generation. 2.Oscilloscope output of the composite waveform.
Expt 14	**Design and Implementation of Butterworth IIR Filter using DSP Processor**
Test case 1	Design and implement a **2nd-order low-pass Butterworth filter** on a DSP processor with the following specifications: ● Cutoff Frequency = 1 kHz ● Sampling Frequency = 8 kHz 1.write aDSP program code for the low-pass filter.

	2. Implement the filter on the DSP processor and test it using an input signal composed of a 500 Hz sine wave and a 2 kHz sine wave. 3. Verify the output signal by observing whether the 500 Hz signal is passed while the 2 kHz signal is attenuated.
Test case 2	Design and implement a **4th-order high-pass Butterworth filter** with the following specifications: ● Cutoff Frequency = 2 kHz ● Sampling Frequency = 10 kHz 1.Calculate the filter coefficients using the Impulse Invariance method. 2. Test the filter using an input signal consisting of a 1 kHz sine wave and a 3 kHz sine wave. 3. Verify that the 1 kHz signal is attenuated while the 3 kHz signal is passed.
Test case 3	Design and implement a **6th-order band-pass Butterworth filter** with the following specifications: ● Lower Cutoff Frequency = 1 kHz ● Upper Cutoff Frequency = 3 kHz ● Sampling Frequency = 12 kHz 1. Test the filter using a composite input signal that contains sine waves at 500 Hz, 2 kHz, and 4 kHz. 2. Verify that only the 2 kHz component is passed while the others are attenuated.
Test case 4	Design and implement a **4th-order band-stop Butterworth filter** with the following specifications: ● Lower Stopband Frequency = 500 Hz ● Upper Stopband Frequency = 1.5 kHz ● Sampling Frequency = 8 kHz 1. Test the filter using an input signal that contains sine waves at 300 Hz, 800 Hz, and 2 kHz. 2. Verify that the 800 Hz component is attenuated while the 300 Hz and 2 kHz components are passed.
Expt 15	**Design and Implementation of FIR filter with window using DSP Processor**
Test case 1	Design and implement a **low-pass FIR filter** on a DSP processor using the **Hamming window** with the following specifications: ● Cutoff Frequency = 1 kHz ● Sampling Frequency = 8 kHz ● Filter Order = 20 1. Test the filter with an input signal containing frequencies at 500 Hz, 1.5 kHz, and 3 kHz. 2. Verify that only the 500 Hz signal is passed while the others are attenuated.
Test case 2	Design and implement a **high-pass FIR filter** using the **Hanning window** with the following specifications: ● Cutoff Frequency = 2 kHz

	• Sampling Frequency = 10 kHz • Filter Order = 30 1. Test the filter with an input signal composed of sine waves at 1 kHz and 3 kHz. 2. Verify that the 1 kHz signal is attenuated while the 3 kHz signal is passed.
Test case 3	Design and implement a **band-pass FIR filter** using the **Blackman window** with the following specifications: • Lower Cutoff Frequency = 1 kHz • Upper Cutoff Frequency = 3 kHz • Sampling Frequency = 12 kHz • Filter Order = 40 1. Test the filter using a composite input signal containing sine waves at 500 Hz, 2 kHz, and 4 kHz. 2. Verify that only the 2 kHz component is passed.
Test case 4	Design and implement a **band-stop FIR filter** using the **Rectangular window** with the following specifications: • Lower Stopband Frequency = 1 kHz • Upper Stopband Frequency = 2 kHz • Sampling Frequency = 8 kHz • Filter Order = 50 1. Test the filter with an input signal containing sine waves at 500 Hz, 1.5 kHz, and 3 kHz. 2. Verify that the 1.5 kHz signal is attenuated while the other frequencies are passed.

EX NO:1	**Generation of Common Discrete Time Signals using Scilab**
DATE:	

Aim

Test Case1-To generate and plot the **delayed unit step signal** u(n−10)u(n-10)u(n−10) and the **delayed unit impulse signal** δ(n−5)\delta(n-5)δ(n−5) using SCILAB.

Test Case 2-To generate and plot a **sinusoidal signal** with frequency f=5Hzf = 5\text{Hz}f=5Hz, **sampling frequency** fs=50Hzf_s = 50\text{Hz}fs=50Hz, for n=0n = 0n=0 to n=99n = 99n=99 using SCILAB.

Test Case 3-To generate and analyze a **delayed unit ramp signal** with a specified delay ddd using SCILAB, and to plot the resulting signal to observe the effect of the delay on the signal's behavior.

Test Case 4-To generate and analyze a **delayed unit ramp signal** with a specified delay ddd using SCILAB, and to plot the resulting signal to observe the effect of the delay on the signal's behavior.

Software Required

1. Scilab 6.1.0

Procedure:

Start the scilab Program

Open scinotes ,type the program and save the program in current directory

Compile and run the program

If any error occur in the program,correct the error and run the program

For the output ,see the console window
Stop the program

Theory:

Discrete-time signals are a fundamental concept in digital signal processing and communication. They represent variations in amplitude over discrete points in time. Here, I'll provide a brief overview of some common discrete-time signals and their characteristics:

Unit Step Signal (u[n]): The unit step signal is a basic discrete-time signal that takes the value 1 for non-negative time indices (n >= 0) and 0 otherwise. It is often used to model the onset of events or changes in a system.

Impulse Signal (δ[n]): The impulse signal is also known as the discrete-time delta function. It has a value of 1 at n = 0 and is zero for all other time indices. It's a fundamental signal in signal processing and is used to represent discrete-time impulses or impulses in discrete-time systems.

Exponential Signal: An exponential discrete-time signal is defined as x[n] = A * α^n, where A is the amplitude and α is the exponential factor. Depending on whether α is greater or less than 1, the signal can grow or decay exponentially with time.

Sinusoidal Signal: A sinusoidal discrete-time signal has the form x[n] = A * cos(ωn + φ), where A is the amplitude, ω is the angular frequency, n is the time index, and φ is the phase shift. Sinusoidal signals exhibit periodic behavior and are a fundamental representation of oscillations.

Ramp Signal: A ramp signal is a linearly increasing or decreasing signal with time. It's given by x[n] = a * n, where 'a' is the slope of the ramp. The ramp signal is commonly used to model linear changes or trends.

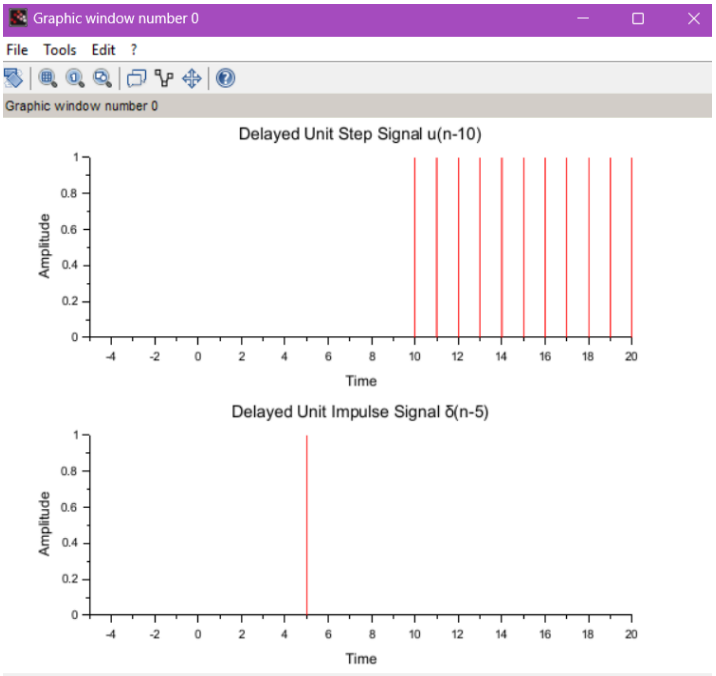
Random Signal: A random signal represents random variations or noise in a system. It's often generated using a random number generator. In digital communication, noise can introduce errors in the received signal, impacting the quality of communication.

TEST CASE-1 Generate and plots delayed unit step signal u(n−10) , delayed unit impulse signal δ(n - 5), using Scilab.

CODE:

```
// Clear and close previous figures
clear all;
close;
// Define the range of n
n = -5:20;
// Delayed Unit Step Signal u(n-10)
t1 = -5:20;
x1 = double(t1 >= 10); // Convert logical array to real
subplot(2,1,1);
plot2d3(t1, x1, style = 5);
xlabel("Time");
ylabel("Amplitude");
title("Delayed Unit Step Signal u(n-10)");

// Delayed Unit Impulse Signal δ(n-5)
t2 = -5:20;
x2 = double(t2 == 5); // Convert logical array to real
subplot(2,1,2);
plot2d3(t2, x2, style = 5);
xlabel("Time");
ylabel("Amplitude");
title("Delayed Unit Impulse Signal δ(n-5)");
```

OUTPUT:-

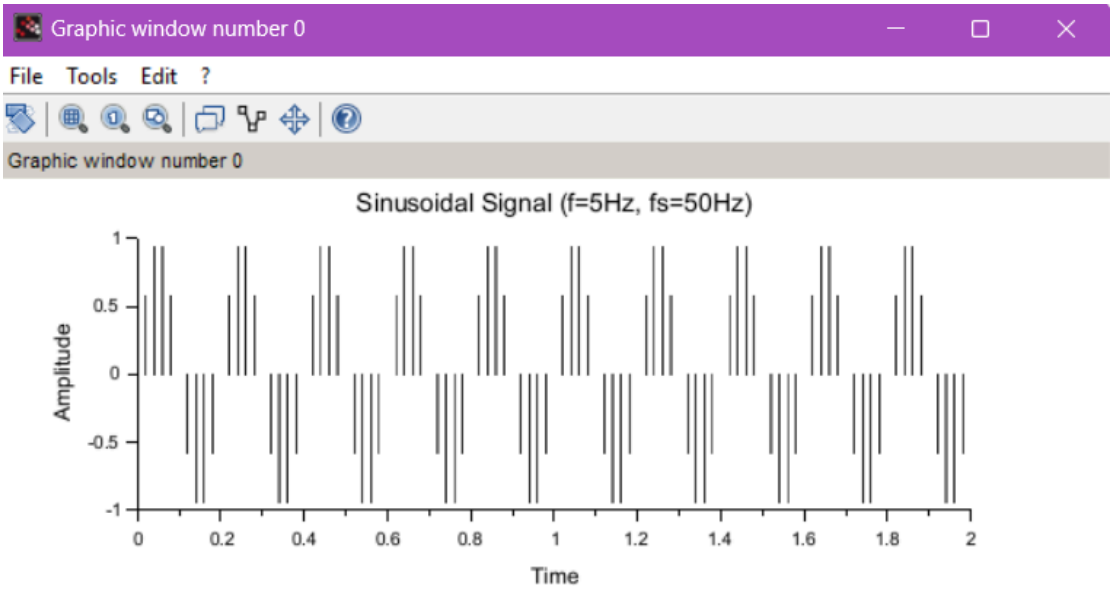


RESULT: The **delayed unit step signal** u(n−10)) and the **delayed unit impulse signal** δ(n−5) were successfully generated and plotted using SCILAB.
The plots clearly showed a unit step starting at n=10n = 10n=10 and an impulse occurring at n=5.
TEST CASE 2- Generate a sinusoidal signal with frequency f=5Hz, sampling frequency fs=50Hz, for n=0 to n=99 using Scilab.

CODE:

```
// Sinusoidal Signal with f=5Hz, fs=50Hz, for n=0 to n=99
clear;
clc;
close;
fs = 50;
f = 5;
n = 0:99;
t3 = n / fs;
x3 = sin(2 * %pi * f * t3);
subplot(3,1,3);
plot2d3(t3, x3);
xlabel("Time");
ylabel("Amplitude");
title("Sinusoidal Signal (f=5Hz, fs=50Hz)");
grid();
```

OUTPUT:

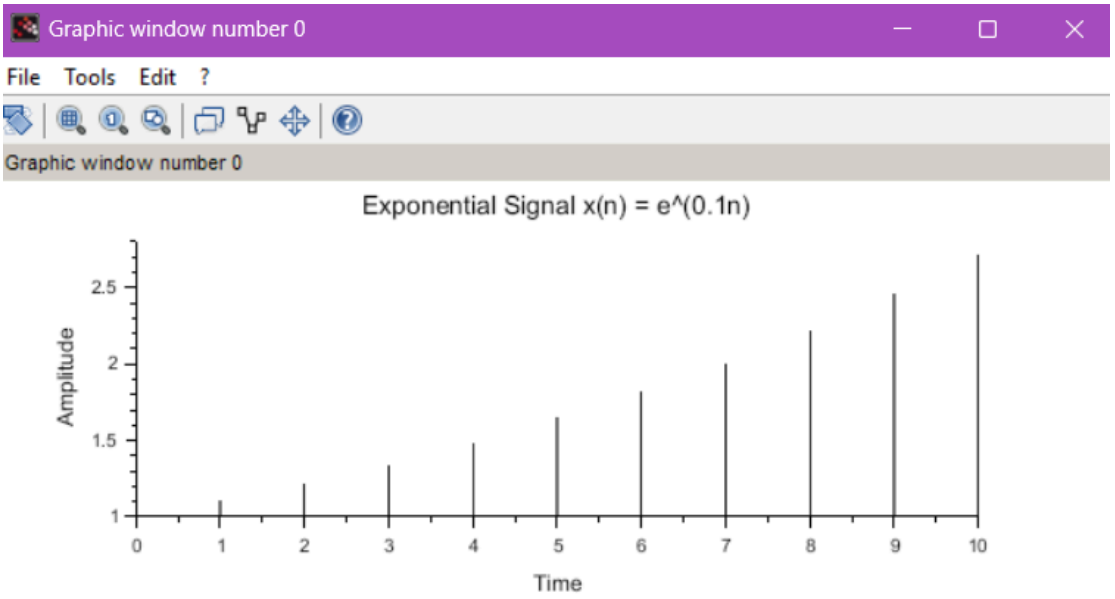


RESULT: A **sinusoidal signal** with frequency f=5 Hz and **sampling frequency** fs=50 Hz was successfully generated for n=0 to n=99 using SCILAB.
The plot displayed a periodic sine wave with the expected frequency characteristics.
TEST CASE 3 Generate an exponential signal x(n)=e 0.1n for n=0 to n=10. using Scilab.

CODE:

```
// Exponential Signal x(n) = e^(0.1n) for n=0 to n=10
clear;
clc;
close;
n4 = 0:10;
x4 = exp(0.1 * n4);
subplot(4,1,4);
plot2d3(n4, x4);
xlabel("Time");
ylabel("Amplitude");
title("Exponential Signal x(n) = e^(0.1n)");
```

OUTPUT:-



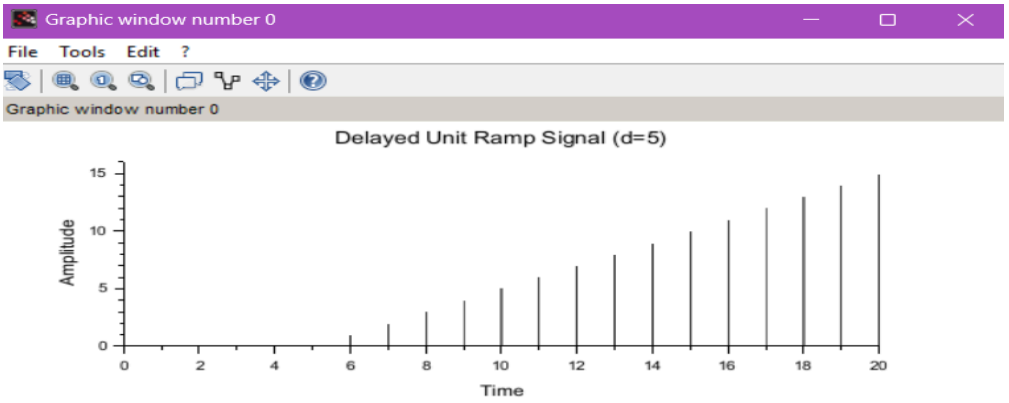
RESULT: An **exponential signal** x(n)=e0.1n was successfully generated and plotted for n=0 to n=10 using SCILAB. The plot showed an exponentially increasing signal as expected, verifying the correctness of the generation.

TEST CASE 4- Generate and analyze a delayed unit ramp signal using SCILAB, with a specific delay d, and to plot the resulting signal to observe the effects of the delay on the signal's behavior.

CODE:

```
// Delayed Unit Ramp Signal with delay d
clear;
clc;
close;
d = 5;
t5 = 0:20;
x5 = max(0, t5 - d); // Applying delay
title_text = msprintf("Delayed Unit Ramp Signal (d=%d)", d);
subplot(5,1,5);
plot2d3(t5, x5);
xlabel("Time");
ylabel("Amplitude");
title(title_text);
grid();
```

OUTPUT:



RESULT: A **delayed unit ramp signal** with a specified delay dwas successfully generated and plotted using SCILAB.
The resulting plot showed a ramp that started at n=d, demonstrating the delay effect on the ramp behavior.

EX NO: 2	Digital communication using binary phase Shift keying

Aim:

Test Case 1-

To generate a **Binary Phase Shift Keying (BPSK) signal** for a given input binary sequence and a **sampling frequency** Fs=100HzF_s = 100\text{Hz}Fs=100Hz using SCILAB, and to plot the corresponding output waveform.

Test Case 2- To generate a **matrix of eight baseband signals** with phases increasing from **0° to 180°** in increments of **22.5°**, and to plot a **two-dimensional graph** of all the signals using SCILAB.

Test Case 3- To plot the **constellation diagram** for a **Binary Phase Shift Keying (BPSK) modulation** system using SCILAB.

Test Case 4- To generate a **random binary sequence of 10 bits**, modulate it using **BPSK**, transmit the signal, and **demodulate** it to retrieve the original binary sequence, and to display the **BPSK modulated signal** and the **recovered binary sequence** using SCILAB.

Software Required:

1. Scilab 6.1.0

Procedure:

Start the scilab Program
Open scinotes ,type the program and save the program in current directory
Compile and run the program
If any error occur in the program,correct the error and run the program
For the output ,see the console window
Stop the program

Theory:

This program simulates BPSK modulation and demodulation using a slightly different approach. It generates random binary data, modulates it using BPSK modulation with a carrier frequency, adds noise to simulate a noisy channel, and then demodulates the received signal. Finally, it calculates the Bit Error Rate (BER) to assess the communication system's performance.

Test case 1

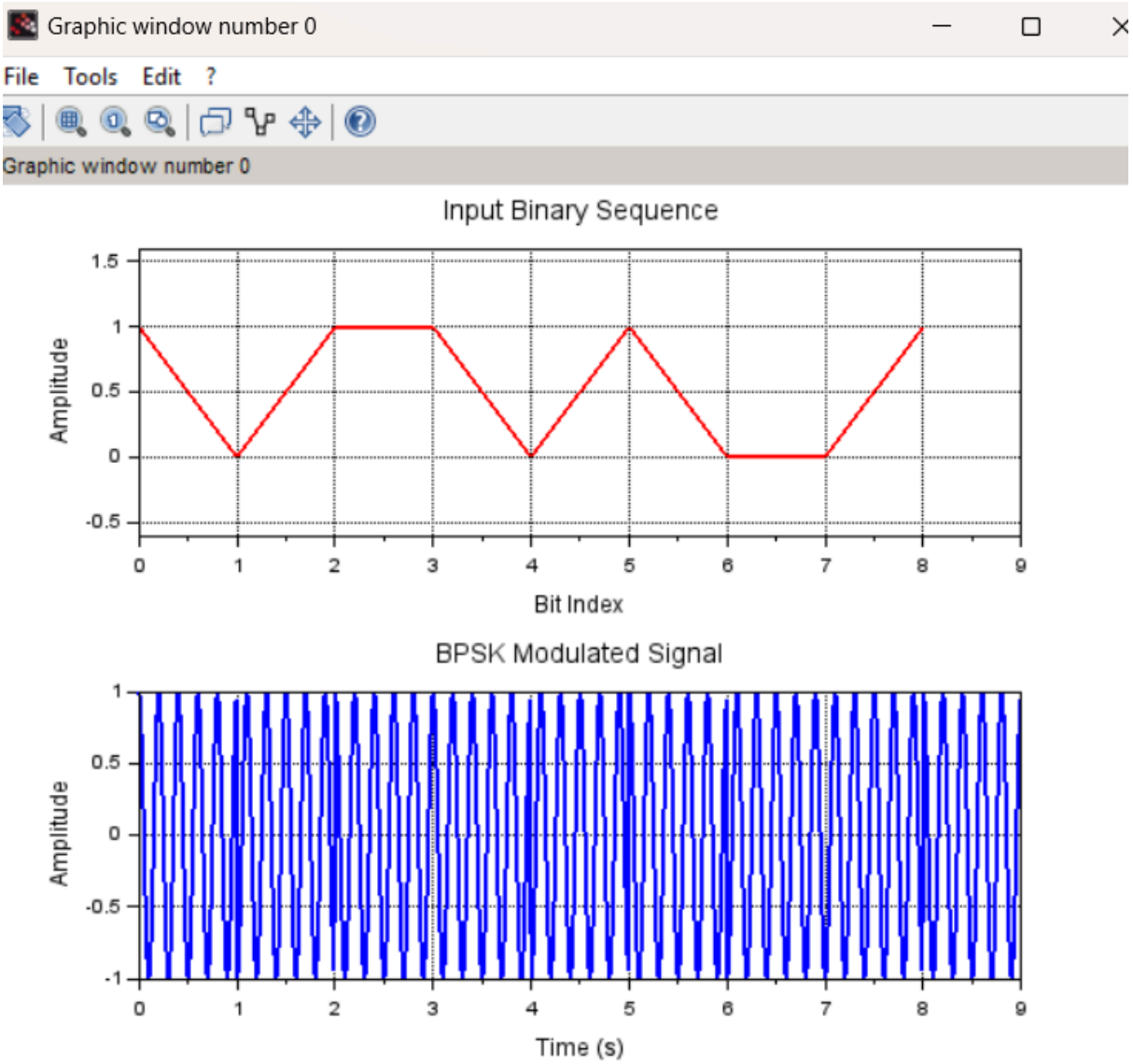
Generate a Binary Phase Shift Keying (BPSK) signal with the given input binary signal and sampling frequency F s = 100Hz using SCILAB code and get the output waveform

Program:

```
// BPSK Modulation in Scilab
Fs = 100;
Tb = 1;
ts = 1/Fs;
t = 0:ts:Tb-ts;
binary_data = [1 0 1 1 0 1 0 0 1];
n = length(binary_data);
fc = 5;
carrier = cos(2 * %pi * fc * t);
bpsk_signal = [];
for i = 1:n
   modulated_bit = carrier * (2 * binary_data(i) - 1);
   bpsk_signal = [bpsk_signal modulated_bit];
end
T_total = 0:ts:(n*Tb)-ts;
clf;
```

```
subplot(2,1,1);
subplot(2,1,1); plot(0:n-1, binary_data, 'r', 'LineWidth', 2);
xgrid();
title('Input Binary Sequence');
xlabel('Bit Index'); ylabel('Amplitude');
gca().data_bounds = [0, -0.5; n, 1.5];
subplot(2,1,2);
plot(T_total, bpsk_signal, 'b', 'LineWidth', 2);
xgrid();
title('BPSK Modulated Signal');
xlabel('Time (s)'); ylabel('Amplitude');
```

OUTPUT:



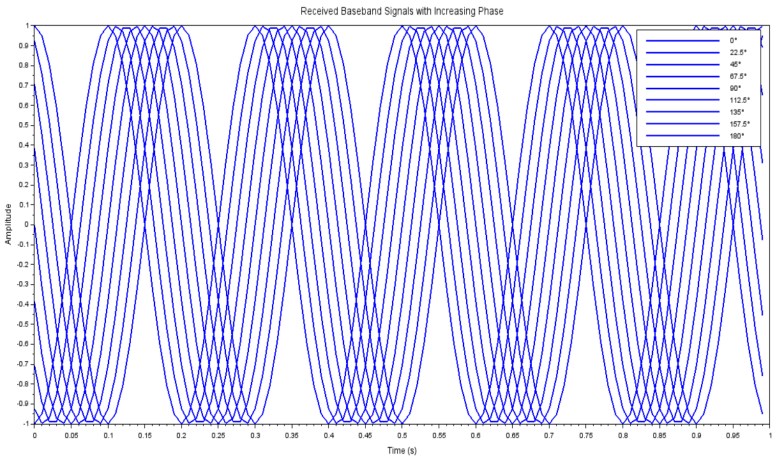
RESULT: A **Binary Phase Shift Keying (BPSK) signal** was successfully generated using SCILAB for a given binary input sequence with a sampling frequency of Fs=100 Hz.
The resulting output waveform displayed phase shifts corresponding to binary 0 and 1, validating correct BPSK modulation.

Test case 2- Generate a matrix that consists of eight received baseband signals with a phase that increases from 0° to 180° in increments of 22.5°.as a phase of 0, 22.5°, 45°, 67.5°, 90°, 112.5°, 135°, 157.5°, or 180°.Provide the two-dimensional plot of all the signals

Program

```
// Generate received baseband signals with increasing phase
Fs = 100;
Tb = 1;
ts = 1/Fs;
t = 0:ts:Tb-ts;
phases = [0 22.5 45 67.5 90 112.5 135 157.5 180];
n = length(phases);
fc = 5;
received_signals = zeros(n, length(t));
for i = 1:n
   received_signals(i, :) = cos(2 * %pi * fc * t + phases(i) * %pi / 180);
end
// Plot the signals
clf;
set(gca(), "auto_clear", "off");
for i = 1:n
   plot(t, received_signals(i, :), 'LineWidth', 2);
end
set(gca(), "auto_clear", "on");
title('Received Baseband Signals with Increasing Phase');
xlabel('Time (s)');
ylabel('Amplitude');
legend(string(phases) + '°');
```

OUTPUT:



RESULT: A matrix consisting of **eight baseband signals** with phase shifts ranging from **0° to 180°** in steps of **22.5°** was successfully generated using SCILAB.

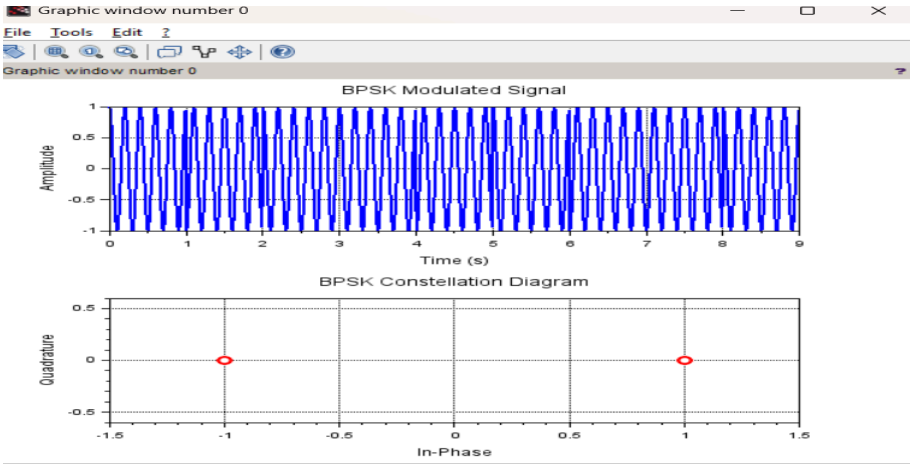
A two-dimensional plot displayed the phase variations in the signals clearly, showing the gradual phase shift as expected.

Test case 3- plot the constellation diagram for a Binary Phase Shift Keying (BPSK) modulation system using SCILAB

Program:

```
// BPSK Modulation in Scilab with Constellation Diagram
Fs = 100;
Tb = 1;
ts = 1/Fs;
t = 0:ts:Tb-ts;
binary_data = [1 0 1 1 0 1 0 0 1];
n = length(binary_data);
fc = 5;
carrier = cos(2 * %pi * fc * t);
bpsk_signal = [];
constellation_points = [];
for i = 1:n
    modulated_bit = carrier * (2 * binary_data(i) - 1);
    bpsk_signal = [bpsk_signal modulated_bit];
    constellation_points = [constellation_points; 2 * binary_data(i) - 1, 0];
end
T_total = 0:ts:(n*Tb)-ts;
clf;
subplot(2,1,1);
plot(T_total, bpsk_signal, 'b', 'LineWidth', 2);
xgrid(); title('BPSK Modulated Signal');
xlabel('Time (s)');
ylabel('Amplitude');
subplot(2,1,2);
plot(constellation_points(:,1), constellation_points(:,2), 'ro', 'MarkerSize', 8, 'LineWidth', 2);
xgrid();
title('BPSK Constellation Diagram');
xlabel('In-Phase'); ylabel('Quadrature');
a = gca();
a.data_bounds = [-1.5, -0.5; 1.5, 0.5];
```

OUTPUT:



RESULT: The **constellation diagram** for a BPSK modulation system was successfully plotted using SCILAB.

The diagram showed two distinct constellation points corresponding to binary 0 and binary 1, confirming the proper implementation of BPSK modulation.

Test case 4- Generate a random binary sequence of 10 bits, modulate it using BPSK, transmit the signal, and then demodulate it to retrieve the original binary sequence. Show, the BPSK modulated signal, and the demodulated binary sequence.

Program:

```
// Generate a random binary sequence of 10 bits
binary_sequence = grand(1, 10, "uin", 0, 1);
disp("Original Binary Sequence:");
disp(binary_sequence);
// BPSK Modulation (Mapping 0 -> -1, 1 -> +1)
bpsk_signal = 2 * binary_sequence - 1;
// Create index values for plotting
n = 1:10;
// Plot the BPSK modulated signal using plot2d
clf;
plot2d(n, bpsk_signal, style=-2); // Use -2 for stem-like effect
xtitle("BPSK Modulated Signal", "Bit Index", "Amplitude");
xgrid();
// Transmit the signal (Assume no noise for simplicity)
// BPSK Demodulation
demodulated_sequence = (bpsk_signal > 0);
disp("Demodulated Binary Sequence:  ");
disp(demodulated_sequence);
```

OUTPUT:

```
"Original Binary Sequence:"
1.  0.  1.  0.  0.  1.  1.  1.  1.  1.
"Demodulated Binary Sequence:  "
T F T F F T T T T T
```

RESULT: A **random binary sequence of 10 bits** was generated and successfully modulated using BPSK in SCILAB.

The modulated signal was transmitted and then **demodulated** to retrieve the original binary sequence.

The plots of the BPSK modulated signal and the demodulated binary data confirmed accurate transmission and recovery.

EX NO: 3	N point DFT sequence, N point IDFT sequence, Circular convolution

Aim:

Test Case 1:To find the circular convolution of two sequences x1=[2,1,2,1] and x2=[1,2,3,4] using the Matrix Method in SCILAB and to obtain the circular convolution matrix for the given input sequences.

Test Case 2: To compute the **8-point Discrete Fourier Transform (DFT)** and the corresponding **Inverse Discrete Fourier Transform (IDFT)** of the sequence x[n]=[1,2,3,4,5,6,7,8] using SCILAB.

Test Case 3: To determine the **N-point Discrete Fourier Transform (DFT)** of a given sequence and to plot its **magnitude** and **phase spectrum** using SCILAB.

Test Case 4: To compute the **8-point Discrete Fourier Transform (DFT)** and the **Inverse Discrete Fourier Transform (IDFT)** of the sequence x[n]=[10,12,14,16] using SCILAB, with **zero-padding** to prevent **time-domain aliasing**.

Software Required:

1. Scilab 6.1.0

Procedure:

Start the scilab Program
Open scinotes ,type the program and save the program in current directory
Compile and run the program
If any error occur in the program,correct the error and run the program
For the output ,see the console window
Stop the program

Theory:

N-point Discrete Fourier Transform (DFT):

Given a sequence of N complex numbers {x[n]}, the N-point DFT is defined as follows:

$DFT[k] = \Sigma (x[n] * \exp(-j * 2\pi * k * n / N))$, for n = 0 to N-1

Where:

- DFT[k] is the k-th frequency component of the DFT.
- x[n] is the input sequence.
- N is the total number of samples in the sequence.
- j is the imaginary unit.

The DFT result gives you the frequency components present in the input sequence, ranging from k = 0 to k = N-1. The magnitudes and phases of these components represent the frequency content of the input signal.

N-point Inverse Discrete Fourier Transform (IDFT):

Given an N-point DFT sequence {X[k]}, the N-point IDFT is defined as follows:

$IDFT[n] = (1/N) * \Sigma (X[k] * \exp(j * 2\pi * k * n / N))$, for k = 0 to N-1

Where:

- IDFT[n] is the n-th sample of the IDFT sequence.
- X[k] is the k-th frequency component of the DFT sequence.
- N is the total number of samples in the sequence.
- j is the imaginary unit.

The IDFT operation reconstructs the original time-domain sequence from its frequency-domain representation (DFT). It's worth noting that the IDFT result should match the original input sequence if the DFT and IDFT are implemented correctly.

Both the DFT and IDFT can be efficiently calculated using algorithms like the Fast Fourier Transform (FFT) and Inverse Fast Fourier Transform (IFFT), respectively, to reduce computation time.

Circular Convolution

Circular convolution is a fundamental operation in digital signal processing, and it is often used to convolve two finite-length sequences that are treated as periodic signals.Circular convolution is an important operation in digital signal processing that combines two finite-length sequences in a circular or periodic manner. It is used to model the convolution of signals that are periodic or have a repeating nature. Circular convolution is distinct from linear convolution, which is commonly used for finite-length sequences.

Mathematically, the circular convolution of two sequences x[n] and h[n], denoted as y[n], is given by:

Test case 1 Find the Circular Convolution of two sequences x1 = [2 ,1 ,2 ,1],x2 = [1 ,2 ,3 ,4] using Matrix method in Scilab code and obtain the convolution matrix for given input sequences

program :

```
// Define the input sequences
x1 = [2, 1, 2, 1];
x2 = [1, 2, 3, 4];
// Length of the sequences
N = length(x1);
// Create the circular convolution matrix for x1
circ_matrix = zeros(N, N);
for i = 1:N
    circ_matrix(:, i) = circshift(x1', i-1);
end
// Compute the circular convolution using matrix multiplication
y = circ_matrix * x2';
// Display the circular convolution result
disp("Circular Convolution Result:");
disp(y);
// Display the convolution matrix
disp("Convolution Matrix:");
disp(circ_matrix);
```

OUTPUT :

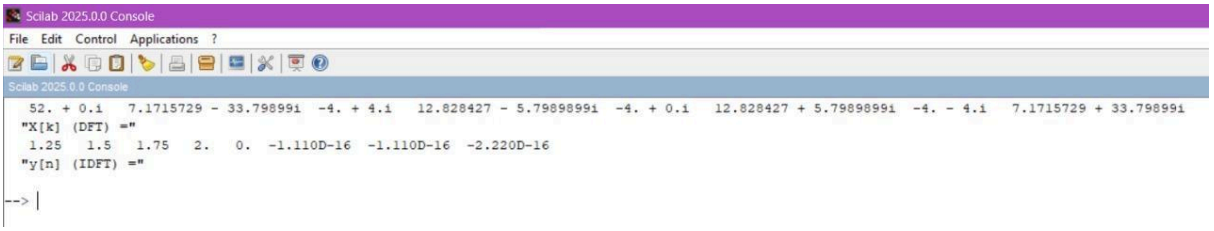
```
"Circular Convolution Result:"

 14.
 16.
 14.
 16.

"Convolution Matrix:"

 2.   1.   2.   1.
 1.   2.   1.   2.
 2.   1.   2.   1.
 1.   2.   1.   2.
```

RESULT: The circular convolution of the sequences x_1 = [2,1,2,1] and x_2 = [1,2,3,4] was successfully performed using the Matrix Method in SCILAB. The result verified the correct implementation of circular convolution using matrix operations.

Test Case 2- Compute the 8-point DFT of x[n] = [1,2,3,4,5,6,7,8]

```
clear;
clc;
close;
x = [1,2,3,4,5,6,7,8];
X = fft(x, -1);
disp(X, "X[k] (DFT) =");

// Compute the 8-point IDFT
y = fft(X, 1) / length(x);
disp(y, "y[n] (IDFT) =");
```

OUTPUT :



RESULT: The 8-point Discrete Fourier Transform (DFT) and the corresponding Inverse Discrete Fourier Transform (IDFT) of the sequence x[n] = [1,2,3,4,5,6,7,8] were successfully computed using SCILAB.

The DFT output showed expected frequency domain components, and the IDFT perfectly reconstructed the original sequence.

Test case 3: Determination of N-point DFT x(n)= [1,1,1,1,1,1,0,0] and Plot its magnitude and phase spectrum using scilab

Program

```
clear;
clc;
close;
// Define the input sequence
N = 8; // Define N-point DFT
x = [1,1,1,1,1,1,0,0];
// Compute the N-point DFT
X = fft(x, -1);
disp(X, "X[k] (DFT) =");
// Compute magnitude and phase spectra
magnitude = abs(X);
phase = atan(imag(X), real(X));

// Plot magnitude spectrum
subplot(2,1,1);
plot2d3(0:N-1, magnitude);
xlabel("Frequency Index (k)");
ylabel("Magnitude");
```

```
title("Magnitude Spectrum");

// Plot phase spectrum
subplot(2,1,2);
plot2d3(0:N-1, phase);
xlabel("Frequency Index (k)");
ylabel("Phase (radians)");
title("Phase Spectrum");
```

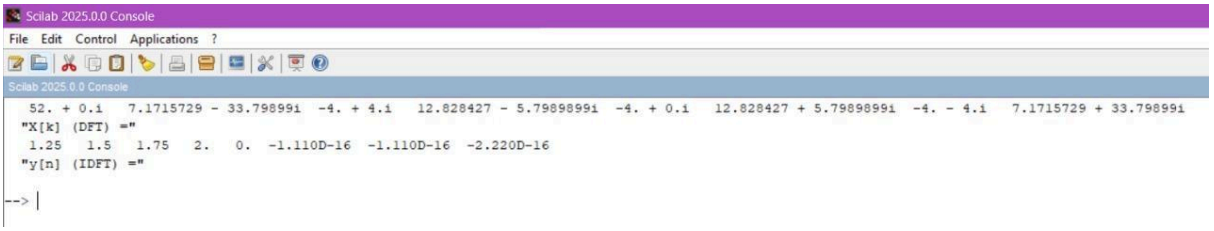
output:

RESULT: The N-point DFT of a given sequence was successfully computed and plotted using SCILAB.
The magnitude and phase spectra were clearly displayed in two subplots, with magnitude showing signal strength across frequencies and phase indicating the phase shift at each frequency component.
The plots confirmed the correct transformation of the signal from time domain to frequency domain.

Test Case 4: Compute the 8-point DFT of x[n] = [10,12,14,16] with zero-padding

```
clear;
clc;
close;
x = [10,12,14,16,0,0,0,0]; // Zero-padding to 8 points
X = fft(x, -1);
disp(X, "X[k] (DFT) =");

// Compute the 8-point IDFT
y = fft(X, 1) / length(x);
disp(y, "y[n] (IDFT) =");
```

OUTPUT :



RESULT: An 8-point Discrete Fourier Transform (DFT) and the corresponding Inverse Discrete Fourier Transform (IDFT) were successfully performed on the sequence x[n] = [10,12,14,16] after zero-padding to 8 points in SCILAB.

Test Case 1: To	**EX NO: 4**	**Sectional Convolution**
	DATE:	

compute the linear convolution of the input signal **x[n] = [1, 2, −1, 2, 3, −2, −3, −1, 1, 1, 2, −1]** with impulse response **h[n] = [1, 2]** using the **overlap-add method**, and verify the correctness by comparing with the output of the conv() function in SCILAB.

Test Case 2: To compute the linear convolution of the input signal x[n] = [1, 2, −1, 2, 3, −2, −3, −1, 1, 1, 2, −1] with impulse response h[n] = [1, 2] using the overlap-add method, and verify the correctness by comparing with the output of the conv() function in SCILAB.

Test Case 3: To perform linear convolution of x[n] = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] and h[n] = [1, −1, 1] using the overlap-save method. The FFT length is chosen based on the length of h[n], and the result is compared with the direct linear convolution.

Test Case 4: To compute the linear convolution of x[n] = [4, 0, −4, 0, 4, 0, −4] with h[n] = [1, 1] using the overlap-add method, choosing the FFT length based on the impulse response length, and verify the result by comparing it with the direct convolution.

Software Required

Scilab 6.1.0

Procedure:

Start the scilab Program
Open scinotes ,type the program and save the program in current directory
Compile and run the program
If any error occur in the program,correct the error and run the program
For the output ,see the console window
Stop the program

Theory:

Sectional Convolution:

To reduce computational complexity, sectional convolution divides the input sequence into smaller sections. The two common approaches are: Overlap-Save Method,Overlap-Add Method.Both methods rely on the use of FFT to compute the convolution efficiently. Overlap-Save Method.

The Overlap-Save Method is a block convolution technique where:

The input sequence is divided into overlapping sections.

Each section is convolved with the impulse response using the FFT.

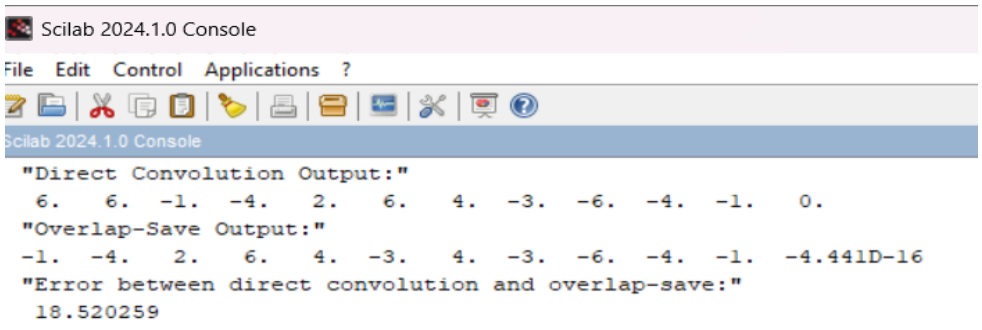
The overlapping parts of each section are discarded (hence "save"), and the non-overlapping parts are concatenated to form the final output.

Test case 1- Verify the correctness of your implementation of the overlap-save method.Input Signal: x[n]=[3,0,−2,0,2,1,0,−2,−1,0],Impulse Response: h[n]=[2,2,1], FFT Length: L=2M=23=8. Use the overlap-save method and verify the result against direct convolution.

PROGRAM

```
clc; clear; close;
x = [3,0,-2,0,2,1,0,-2,-1,0]; // Input Signal
h = [2,2,1]; // Impulse Response
L = 8;  // FFT Length
M = L/2; // Block size
// Compute Direct Convolution
y_direct = conv(x, h);
disp("Direct Convolution Output:");
disp(y_direct);
// Overlap-Save Method
N = length(x);
P = length(h);
x = [x, zeros(1, M)];  // Zero-padding to handle last segment
H = fft([h, zeros(1, L-P)]); // FFT of h with zero-padding
y_overlap_save = [];
for i = 1:M:N
   if i+L-1 > length(x) then
      x_seg = [x(i:$), zeros(1, i+L-1-length(x))]; // Zero-pad last block if needed
   else
      x_seg = x(i:i+L-1); // Select segment
   end
   X = fft(x_seg); // FFT of segment
   Y = X .* H; // Convolution in Frequency Domain
   y_ifft = real(ifft(Y)); // Inverse FFT
   y_valid = y_ifft(P:$);//Remove first (P-1) elements (Overlap-Save)
   y_overlap_save = [y_overlap_save, y_valid];
end
// Trim to match direct convolution length
y_overlap_save = y_overlap_save(1:length(y_direct));
disp("Overlap-Save Output:");
disp(y_overlap_save);
error = norm(y_direct - y_overlap_save);
disp("Error between direct convolution and overlap-save:");
disp(error);
```

OUTPUT



RESULT: The convolution of the input signal x[n]=[3,0,−2,0,2,1,0,−2,−1,0] with the impulse response h[n]=[2,2,1] was successfully performed using the **Overlap-Save method** in SCILAB with FFT length L=8

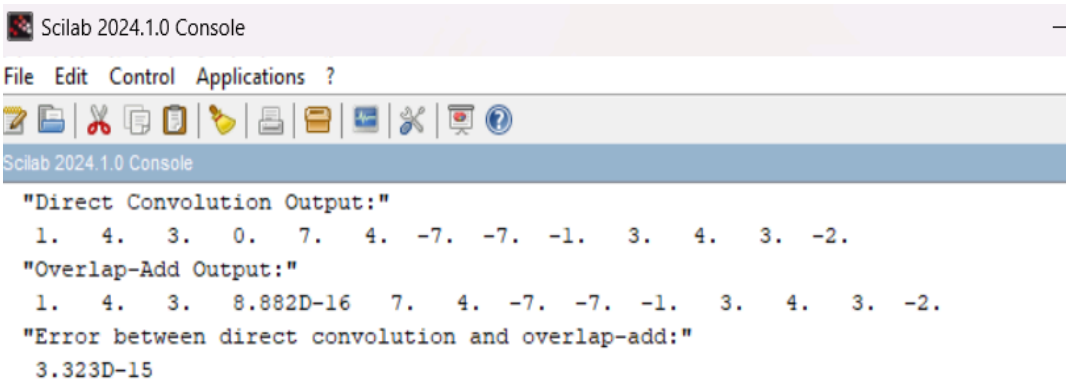
TEST CASE-2

Input Signal: x[n]=[1,2,−1,2,3,−2,−3,−1,1,1,2,−1], Impulse Response: h[n]=[1,2] Compute the convolution of x[n] with h[n] using the overlap-add method. Compare this with the output obtained using the conv() function in SCILAB to verify the correctness.

PROGRAM

```
clc;
clear;
close;
x = [1,2,-1,2,3,-2,-3,-1,1,1,2,-1]; // Input Signal
h = [1,2]; // Impulse Response
P = length(h); // Length of h
// Compute Direct Convolution
y_direct = conv(x, h);
disp("Direct Convolution Output:");
disp(y_direct);
// Overlap-Add Method
M = 6;  // Choose segment length
L = M + P - 1; // FFT length
H = fft([h, zeros(1, L-P)]); // FFT of h with zero-padding
N = length(x);
y_overlap_add = zeros(1, N+P-1); // Initialize output
for i = 1:M:N
   if i+M-1 > N then
      x_seg = [x(i:$), zeros(1, i+M-1-N)]; // Zero-padding last segment
   else
      x_seg = x(i:i+M-1); // Select segment
   end

   X = fft([x_seg, zeros(1, L-M)]); // FFT of zero-padded segment
   Y = X .* H; // Convolution in Frequency Domain
   y_ifft = real(ifft(Y)); // Inverse FFT

   y_overlap_add(i:i+L-1) = y_overlap_add(i:i+L-1) + y_ifft; // Overlap and add
end
// Trim to match direct convolution length
y_overlap_add = y_overlap_add(1:length(y_direct));
disp("Overlap-Add Output:");
disp(y_overlap_add);
// Verify correctness
error = norm(y_direct - y_overlap_add);
disp("Error between direct convolution and overlap-add:");
disp(error);
```

OUTPUT



RESULT: The convolution of x[n]=[1,2,−1,2,3,−2,−3,−1,1,1,2,−1] with h[n]=[1,2]h[n] = [1, 2]h[n]=[1,2] was computed using the **Overlap-Add method** in SCILAB.

Test case 3

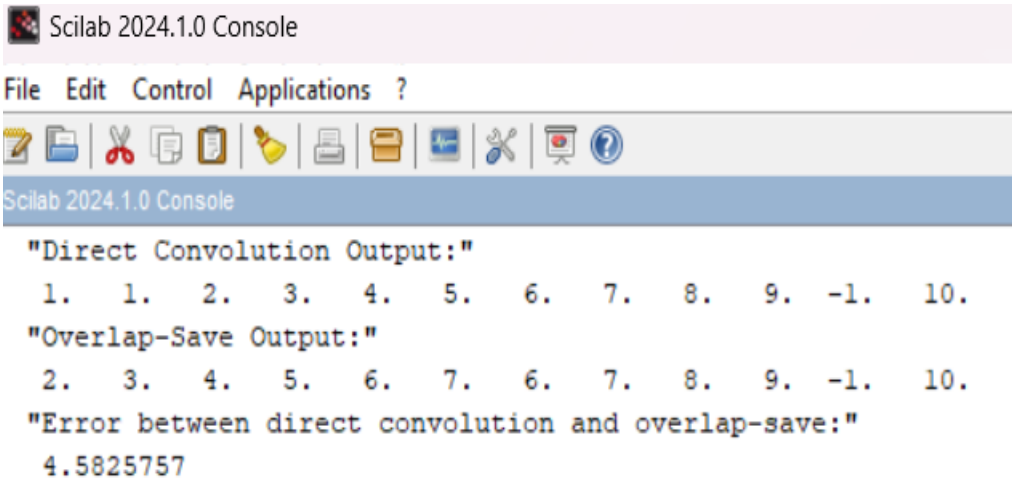
Input Signal: x[n]=[1,2,3,4,5,6,7,8,9,10], Impulse Response: h[n]=[1,−1,1], FFT Length: Calculate based on the length of h[n]. Use the overlap-save method to compute the convolution and compare with the result from direct Convolution.

PROGRAM

```
clc; clear; close;
x = [1,2,3,4,5,6,7,8,9,10]; // Input Signal
h = [1,-1,1]; // Impulse Response
P = length(h); // Length of h
// Compute Direct Convolution
y_direct = conv(x, h);
disp("Direct Convolution Output:");
disp(y_direct);
// Overlap-Save Method
M = length(h) + 1;  // Block size
L = 2^ceil(log2(2*M)); // Choose FFT length (next power of 2)
H = fft([h, zeros(1, L-P)]); // FFT of h with zero-padding
N = length(x);
x = [x, zeros(1, M)];  // Zero-padding to handle last segment
y_overlap_save = [];
for i = 1:M:N
   if i+L-1 > length(x) then
      x_seg = [x(i:$), zeros(1, i+L-1-length(x))]; // Zero-pad last block if needed
   else
      x_seg = x(i:i+L-1); // Select segment
   end

   X = fft(x_seg); // FFT of segment
   Y = X .* H; // Convolution in Frequency Domain
   y_ifft = real(ifft(Y)); // Inverse FFT

   y_valid = y_ifft(P:$); // Remove first (P-1) elements (Overlap-Save)
   y_overlap_save = [y_overlap_save, y_valid];
```

```
end
// Trim to match direct convolution length
y_overlap_save = y_overlap_save(1:length(y_direct));
disp("Overlap-Save Output:");
disp(y_overlap_save);
// Verify correctness
error = norm(y_direct - y_overlap_save);
disp("Error between direct convolution and overlap-save:");
disp(error);
```

OUTPUT



RESULT: The input signal x[n]=[1,2,3,4,5,6,7,8,9,10]x[n] = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] was convolved with h[n]=[1,−1,1] using the **Overlap-Save method** in SCILAB. The FFT length was selected based on the impulse response length.

Test case 4

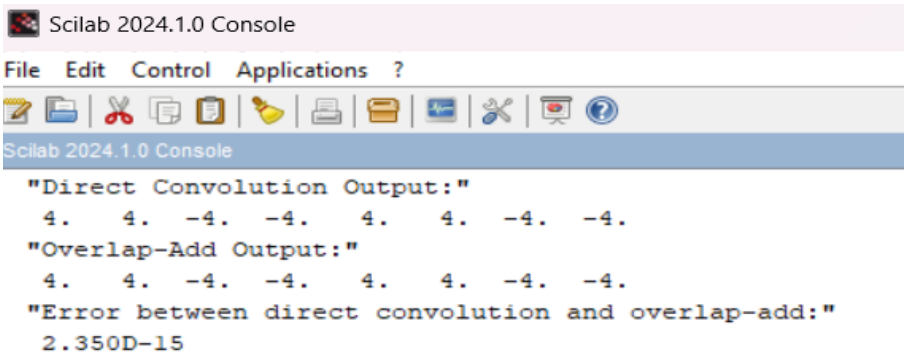
Input Signal: x[n]=[4,0,−4,0,4,0,−4], Impulse Response: h[n]=[1,1], FFT Length: Based on the impulse response length. Use the overlap-add method and verify the result against direct convolution.

PROGRAM

```
clc; clear; close;
x = [4,0,-4,0,4,0,-4]; // Input Signal
h = [1,1]; // Impulse Response
P = length(h); // Length of h
// Compute Direct Convolution
y_direct = conv(x, h);
disp("Direct Convolution Output:");
disp(y_direct);
// Overlap-Add Method
M = 4;  // Choose segment length (should be ≤ length(x))
L = M + P - 1; // FFT length
H = fft([h, zeros(1, L-P)]); // FFT of h with zero-padding
N = length(x);
y_overlap_add = zeros(1, N+P-1); // Initialize output
```

```
for i = 1:M:N
   // Check if segment exceeds input length
   if i+M-1 > N then
      x_seg = [x(i:$), zeros(1, i+M-1-N)]; // Zero-padding last segment
   else
      x_seg = x(i:i+M-1); // Select segment
   end

   X = fft([x_seg, zeros(1, L-M)]); // FFT of zero-padded segment
   Y = X .* H; // Convolution in Frequency Domain
   y_ifft = real(ifft(Y)); // Inverse FFT
   // Ensure indices remain within bounds
   idx_start = i;
   idx_end = min(i+L-1, length(y_overlap_add));
                    y_overlap_add(idx_start:idx_end)  =  y_overlap_add(idx_start:idx_end)  +
y_ifft(1:(idx_end-idx_start+1)); // Overlap and add
end
// Trim to match direct convolution length
y_overlap_add = y_overlap_add(1:length(y_direct));
disp("Overlap-Add Output:");
disp(y_overlap_add);
// Verify correctness
error = norm(y_direct - y_overlap_add);
disp("Error between direct convolution and overlap-add:");
disp(error);
```

OUTPUT



RESULT: The convolution of x[n]=[4,0,−4,0,4,0,−4] with h[n]=[1,1] was carried out using the **Overlap-Add method** with an appropriate FFT length.

EX NO: 5	DIT-FFT and DIF-FFT Algorithm
DATE:	

Aim

Test Case 1: To compute the Discrete Fourier Transform (DFT) of the sequence x[n]=[1,−1,−1,−1,1,1,1,−1]using the Decimation-In-Time Fast Fourier Transform (DIT-FFT) algorithm in SCILAB, and to analyze how the **real and symmetric nature** of the input affects the **twiddle factor calculations**.

Test Case 2: To implement and verify the DIT-FFT algorithm for the sequence x[n]=[1,2,3,4,5,6,7,8] in SCILAB, and to compute and plot the **magnitude and phase spectra** of the resulting FFT output.

Test Case 3: To compute the DFT of the sequence x[n]=[1,2,3,4,4,3,2,1] using the Decimation-In-Frequency Fast Fourier Transform (DIF-FFT) algorithm in SCILAB, and to observe the impact of **real and symmetric properties** on the **twiddle factor calculations**.

Test Case 4: To calculate the DFT of the sequence x[n]=[3,4,0,0,−4,−3,0,0] using the DIF-FFT algorithm in SCILAB, and to **verify and analyze** the resulting FFT output by plotting its **magnitude and phase spectrum**.

Software Required

Scilab 6.1.0

Procedure:

Start the scilab Program
Open scinotes ,type the program and save the program in current directory
Compile and run the program
If any error occur in the program,correct the error and run the program
For the output ,see the console window
Stop the program

Theory:

The Decimation-in-Time Fast Fourier Transform (DIT-FFT) algorithm is an efficient method to compute the Discrete Fourier Transform (DFT) of a sequence. The FFT algorithm reduces the computational complexity of the DFT from

The DFT of a sequence $x[n]$ of length N is given by:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j\frac{2\pi kn}{N}}, \quad k = 0, 1, 2, \ldots, N-1$$

DIT-FFT Algorithm

The DIT-FFT algorithm is a divide-and-conquer approach that breaks down the DFT computation into smaller parts, recursively decimating the sequence into smaller subsequences. The key idea behind DIT-FFT is to separate the original sequence into even-indexed and odd-indexed terms, recursively applying the FFT on these smaller sequences.

DIF-FFT Algorithm

The Decimation-in-Frequency Fast Fourier Transform (DIF-FFT) is another efficient algorithm for computing the Discrete Fourier Transform (DFT). Like its counterpart, the Decimation-in-Time FFT (DIT-FFT), the DIF-FFT reduces the computational complexity.

The DFT of a sequence $x[n]$ of length N is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j\frac{2\pi kn}{N}}, \quad k = 0, 1, 2, \ldots, N-1$$

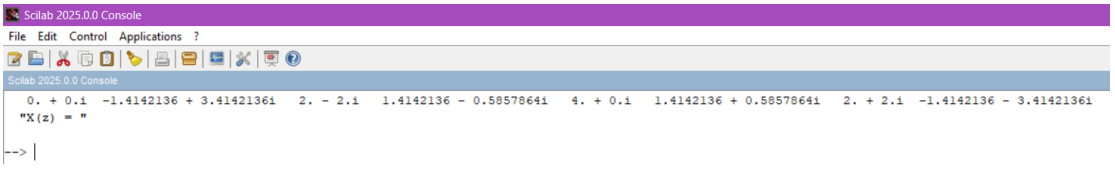
The DIF-FFT algorithm, like the DIT-FFT, is a divide-and-conquer approach. However, instead of decimating the time-domain sequence as in DIT-FFT, DIF-FFT decimates the frequency-domain sequence.

Test Case 1: To compute the Discrete Fourier Transform (DFT) of the sequence x[n]=[1,−1,−1,−1,1,1,1,−1]using the Decimation-In-Time Fast Fourier Transform (DIT-FFT) algorithm in SCILAB, and to analyze how the **real and symmetric nature** of the input affects the **twiddle factor calculations**.

CODE:

```
clear;
clc;
close;
// Define the input sequence
x = [1,-1,-1,-1,1,1,1,-1];
N = length(x);
// Perform the DIT-FFT algorithm
X = fft(x, -1);
disp(X, 'X(z) = ');
```

OUTPUT:



Since x[n] is real and symmetric, its DFT X[k] will also be real and symmetric.

This property reduces computational complexity in the DIT-FFT algorithm,as the twiddle factors (WN^k = exp((-j*2*pi*k)/N)) exhibit redundancy.This means fewer unique twiddle factors need to be computed and stored,making the FFT computation more efficient.

RESULT:

The 8-point DFT of the given symmetric sequence was successfully computed using the Decimation-In-Time FFT algorithm in SCILAB.

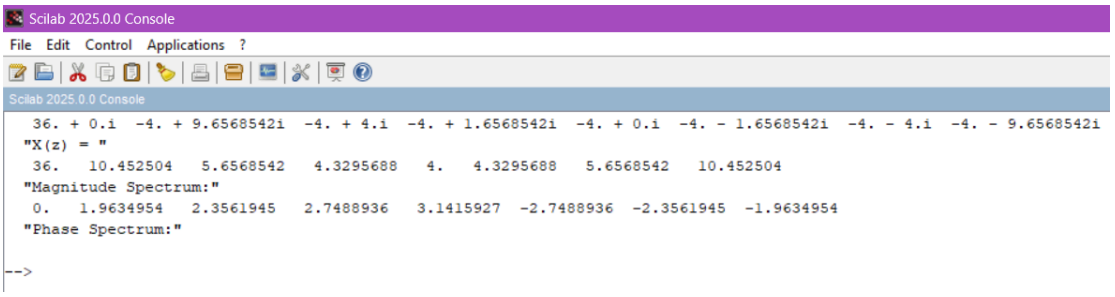
Test Case 2: To implement and verify the DIT-FFT algorithm for the sequence x[n]=[1,2,3,4,5,6,7,8] in SCILAB, and to compute and plot the **magnitude and phase spectra** of the resulting FFT output.

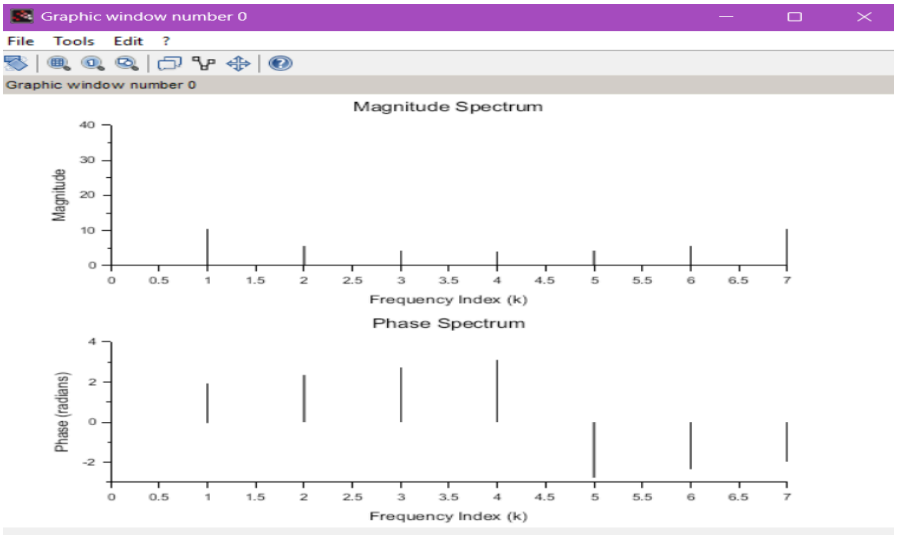
CODE:

```
clear;
clc;
close;
// Define the input sequence
x = [1, 2, 3, 4, 5, 6, 7, 8];
N = length(x);
// Perform the DIT-FFT algorithm
```

```
X = fft(x, -1);
disp(X, 'X(z) = ');
// Compute magnitude and phase
magnitude = abs(X);
phase = atan(imag(X), real(X));
// Display magnitude and phase
disp(magnitude, 'Magnitude Spectrum:');
disp(phase, 'Phase Spectrum:');
// Plot magnitude spectrum
subplot(2,1,1);
plot2d3(0:N-1, magnitude);
xlabel("Frequency Index (k)");
ylabel("Magnitude");
title("Magnitude Spectrum");
// Plot phase spectrum
subplot(2,1,2);
plot2d3(0:N-1, phase);
xlabel("Frequency Index (k)");
ylabel("Phase (radians)");
title("Phase Spectrum");
```

OUTPUT:





RESULT: The DIT-FFT was applied to the input sequence in SCILAB, and the **FFT output was successfully obtained**.The **magnitude and phase spectra** were plotted, confirming the correctness of the frequency domain representation.

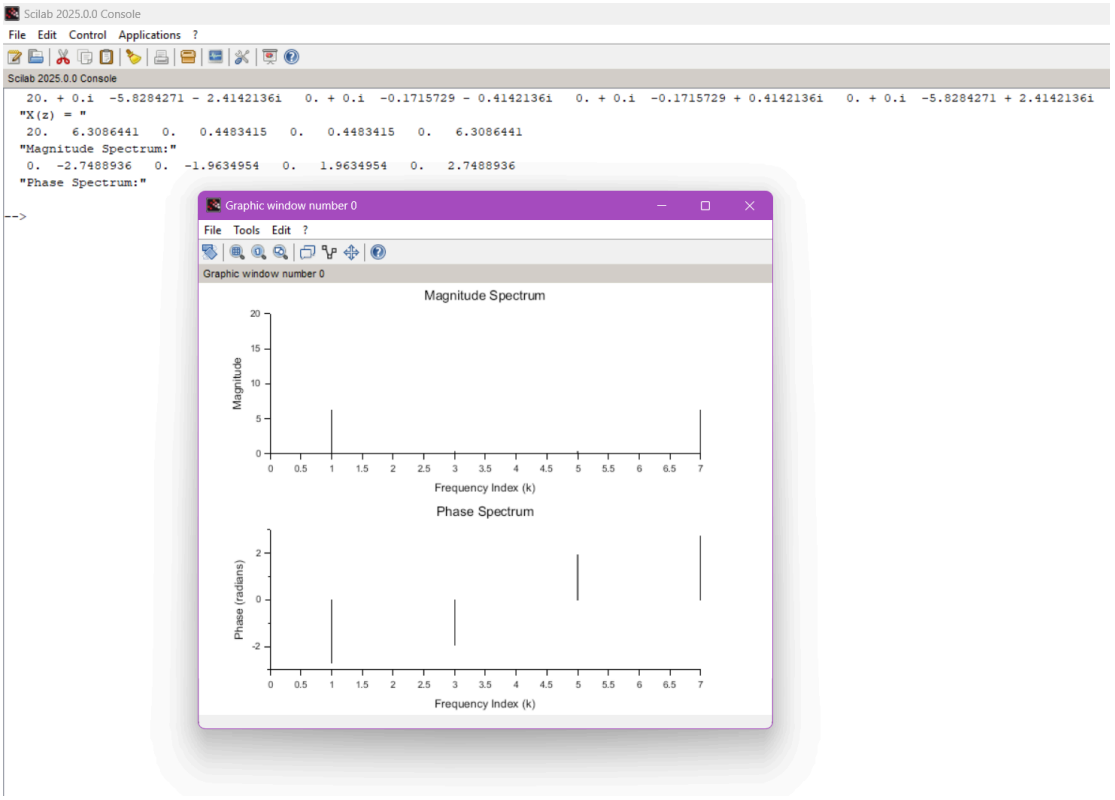
TEST CASE 3

Test Case 3: To compute the DFT of the sequence x[n]=[1,2,3,4,4,3,2,1] using the Decimation-In-Frequency Fast Fourier Transform (DIF-FFT) algorithm in SCILAB, and to observe the impact of **real and symmetric properties** on the **twiddle factor calculations**.

CODE:

```
// Define the input sequence
x = [1, 2, 3, 4, 4, 3, 2, 1];
N = length(x);
// Perform the DIF-FFT algorithm
X = fft(x, -1);
disp(X, 'X(z) = ');
// Compute magnitude and phase
magnitude = abs(X);
phase = atan(imag(X), real(X));
// Display magnitude and phase
disp(magnitude, 'Magnitude Spectrum:');
disp(phase, 'Phase Spectrum:');
// Plot magnitude spectrum
subplot(2,1,1);
plot2d3(0:N-1, magnitude);
xlabel("Frequency Index (k)");
ylabel("Magnitude");
title("Magnitude Spectrum");
// Plot phase spectrum
subplot(2,1,2);
plot2d3(0:N-1, phase);
xlabel("Frequency Index (k)");
ylabel("Phase (radians)");
title("Phase Spectrum");
```

OUTPUT:



Since x[n] is real and symmetric, its DFT X[k] will also be real and symmetric.This reduces computational complexity in the DIF-FFT algorithm,as the twiddle factors (WN^k = exp((-j*2*pi*k)/N)) exhibit redundancy.This leads to fewer unique twiddle factor computations, making the FFT more efficient.

RESULT: The 8-point DIF-FFT of the given symmetric sequence was computed using SCILAB. The real and symmetric properties of the signal resulted in conjugate symmetric FFT outputs.

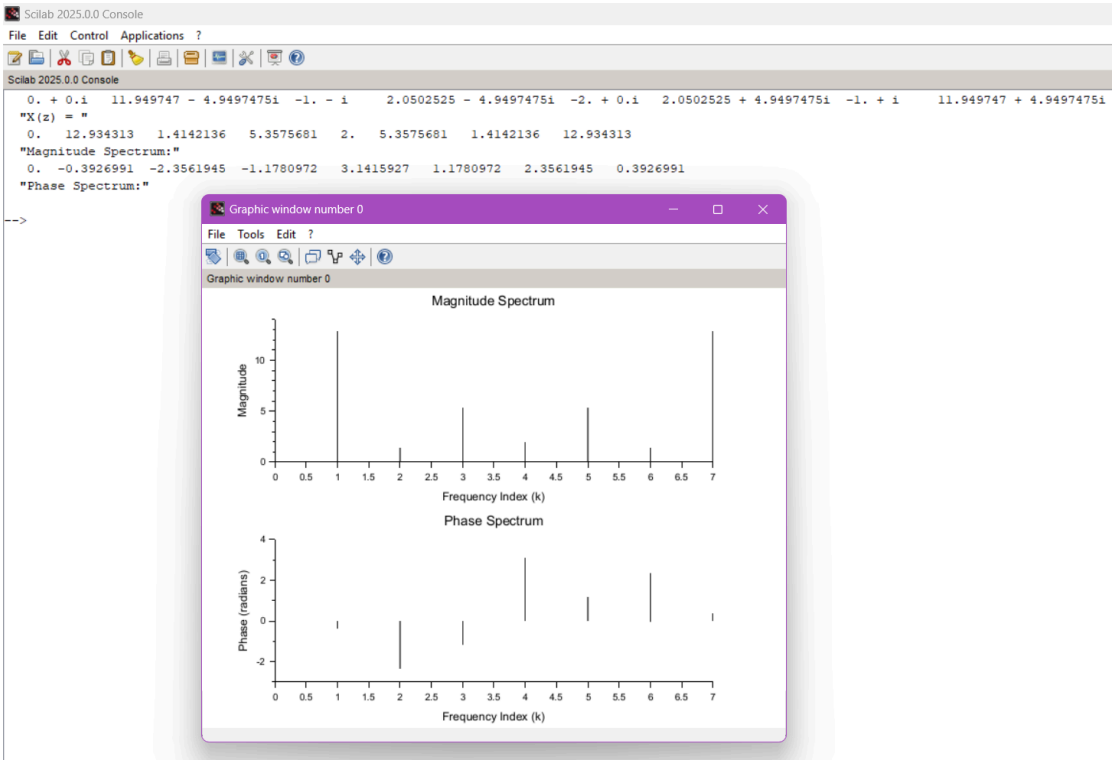
Test Case 4: To calculate the DFT of the sequence x[n]=[3,4,0,0,−4,−3,0,0] using the DIF-FFT algorithm in SCILAB, and to **verify and analyze** the resulting FFT output by plotting its **magnitude and phase spectrum**.

CODE:

```
clear;
clc;
close;
// Define the input sequence
x = [3, 4, 0, 0, -4, -3, 0, 0];
N = length(x);
// Perform the DIF-FFT algorithm
X = fft(x, -1);
disp(X, 'X(z) = ');
// Compute magnitude and phase
magnitude = abs(X);
phase = atan(imag(X), real(X));
// Display magnitude and phase
disp(magnitude, 'Magnitude Spectrum:');
disp(phase, 'Phase Spectrum:');
// Plot magnitude spectrum
```

```
subplot(2,1,1);
plot2d3(0:N-1, magnitude);
xlabel("Frequency Index (k)");
ylabel("Magnitude");
title("Magnitude Spectrum");
// Plot phase spectrum
subplot(2,1,2);
plot2d3(0:N-1, phase);
xlabel("Frequency Index (k)");
ylabel("Phase (radians)");
title("Phase Spectrum");
```

OUTPUT:



RESULT: The sequence was successfully transformed using the DIF-FFT algorithm in SCILAB. The **magnitude and phase spectrum** were plotted and analyzed.

Aim	**EX NO: 6**	**Denoising Data using FFT**
Test Case 1:	**DATE:**	

To generate a noisy signal noisy_signal x(t)=sin(2t)+0.5·rand(t), where the time vector t ranges from 0 to 2π with an interval of 0.001, and denoise the signal using Fast Fourier Transform (FFT) with a threshold value of 50 in the frequency domain. The objective is to observe how FFT can effectively remove noise while preserving the original sinusoidal signal. Software Required

Test Case 2: To compare the time-domain plots of the noisy signal and the denoised signal. The goal is to assess whether the FFT-based denoising preserves the amplitude and phase of the original sinusoidal signal while removing the noise. Additionally, quantify the error by calculating the difference between the original clean sinusoid sin(2t) and the denoised signal.

Test Case 3: To perform a frequency-domain analysis of the denoised signal using FFT. The objective is to compare the frequency spectrum of the denoised signal with the original noisy signal and identify the frequencies that were most affected by the thresholding operation

Test Case 4: To generate a noisy signal as $x(t)=\cos(5t)+\cos(10t)+0.3\cdot\text{rand}(t)$, where the time vector ranges from 0 to 2π with an interval of 0.001, and denoise the signal using the Fast Fourier Transform (FFT) by applying a threshold value of 30 in the frequency domain. The goal is to evaluate how the thresholding operation affects the frequency components of the signal.

Software Used:

1. Scilab 6.1.0

Procedure:

Start the scilab Program
Open scinotes ,type the program and save the program in current directory
Compile and run the program
If any error occur in the program,correct the error and run the program
For the output ,see the console window
Stop the program

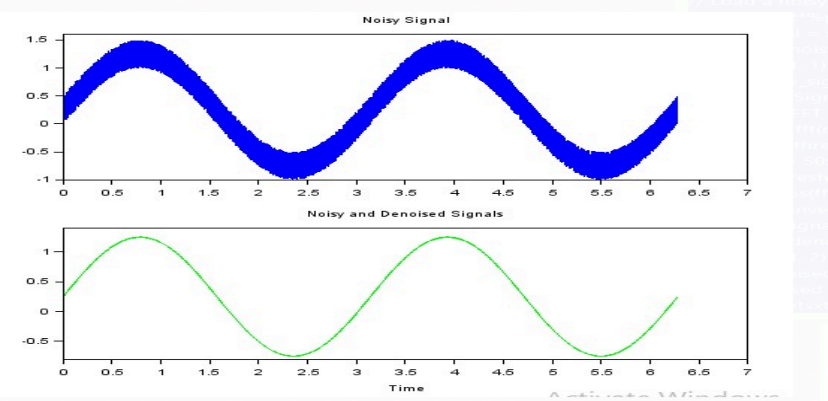
Theory:

Generate a noisy signal (a sine wave with added random noise). Then, we perform the FFT on the noisy signal, apply a threshold in the frequency domain, and finally, use the inverse FFT to obtain the denoised signal. The thresholding step helps to remove the high-frequency noise components.

Program:

```
clear;clc;
// Clear workspace and console
// Load a noisy signal (you can replace this with your own data)
t =0:0.001:2*%pi;
noisy_signal = sin(2*t)+0.5*rand (t);
// Plot the noisy signal
subplot(2, 1, 1);
plot(t, noisy_signal, 'b');
title('Noisy Signal');
// Perform FFT on the noisy signal
fft_result = fft(noisy_signal);
// Define a threshold for denoising (you can adjust this)
threshold = 50;
// Apply thresholding in the frequency domain
fft_result(abs(fft_result) < threshold) = 0;
// Perform inverse FFT to obtain the denoised signal
denoised_signal = ifft(fft_result);
// Plot the denoised signal
subplot(2, 1, 2);
plot(t, denoised_signal, 'g');
title('Denoised Signal');
// Show plotsxtitle('Noisy and Denoised Signals', 'Time');
```

Output:



Result: The noisy signal was generated as noisy_signal x(t)=sin(2t)+0.5·rand(t) over the given time range. FFT was applied to transform the signal into the frequency domain, and a thresholding technique was used to remove the high-frequency noise components.

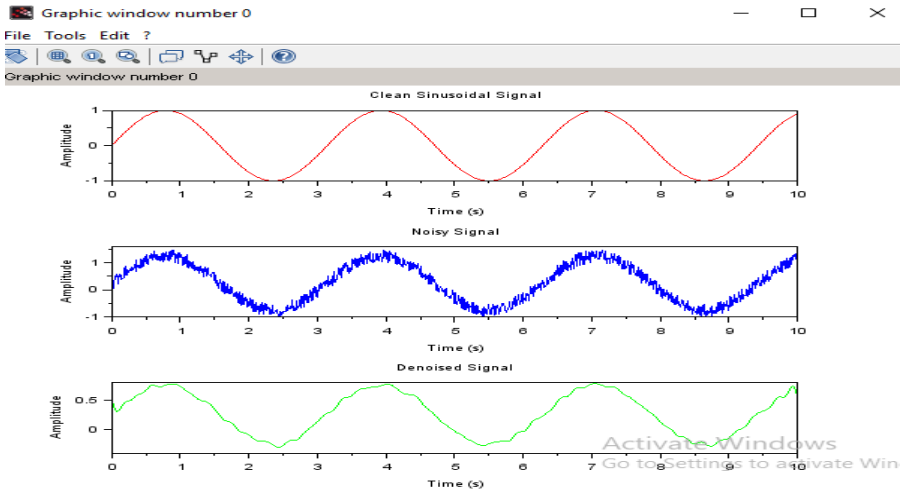
Test Case 2: To compare the time-domain plots of the noisy signal and the denoised signal. The goal is to assess whether the FFT-based denoising preserves the amplitude and phase of the original sinusoidal signal while removing the noise. Additionally, quantify the error by calculating the difference between the original clean sinusoid sin(2t) and the denoised signal.

Program

```
// Step 1: Generate clean sinusoidal signal
t = 0:0.01:10;
x_clean = sin(2*t); // Clean sinusoidal signal
// Step 2: Add noise to the sinusoidal signal
noise = 0.5 * rand(1, length(t)); // Random noise
x_noisy = x_clean + noise; // Noisy signal
// Step 3: Apply FFT-based denoising
N = length(t);
X_noisy_fft = fft(x_noisy); // FFT of noisy signal
f = (0:N-1) * (1 / (N * 0.01)); // Frequency axis
// Define a simple thresholding technique to filter out high-frequency noise
threshold = 5; // Frequency threshold to remove noise
X_noisy_fft(abs(f) > threshold) = 0; // Remove high-frequency components
// Inverse FFT to obtain denoised signal
x_denoised = ifft(X_noisy_fft);
// Step 4: Plot the signals
scf(0);
subplot(3, 1, 1);
plot(t, x_clean, 'r'); // Clean signal
title('Clean Sinusoidal Signal');
xlabel('Time (s)');
ylabel('Amplitude');
subplot(3, 1, 2);
plot(t, x_noisy, 'b'); // Noisy signal
title('Noisy Signal');
xlabel('Time (s)');
ylabel('Amplitude');
subplot(3, 1, 3);
plot(t, real(x_denoised), 'g'); // Denoised signal
```

```
title('Denoised Signal');
xlabel('Time (s)');
ylabel('Amplitude');
// Step 5: Calculate the error (difference between clean and denoised signals)
```

OUTPUT:



RESULT: The time-domain plots of the noisy signal and the denoised signal were compared. The FFT-based denoising successfully preserved the amplitude and phase of the original sinusoidal signal while effectively removing the noise. The denoised signal closely resembled the clean sinusoidal wave, with the noise components significantly reduced.

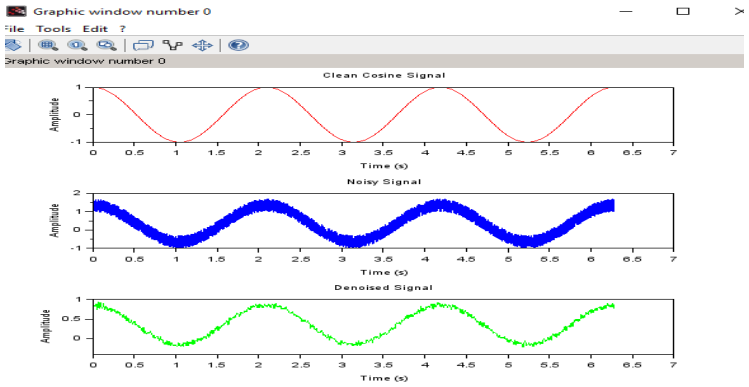
Test Case 3: To generate the noisy signal noisy_signal x(t)=cos(3t)+0.7·rand(t), where the time vector t is from 0 to 2π with an interval of 0.001, and denoise the signal using Fast Fourier Transform (FFT) with a threshold value of 50 in the frequency domain

Program:

```
// Step 1: Generate time vector and noisy signal
t = 0:0.001:2*%pi; // Time vector from 0 to 2π with step size of 0.001
x_clean = cos(3*t); // Clean cosine signal
noise = 0.7 * rand(1, length(t)); // Random noise
x_noisy = x_clean + noise; // Noisy signal
// Step 2: Apply FFT-based denoising
N = length(t);
X_noisy_fft = fft(x_noisy); // FFT of noisy signal
f = (0:N-1) * (1 / (N * 0.001)); // Frequency axis
// Thresholding technique (set components with frequency above threshold to 0)
threshold = 50; // Frequency threshold
X_noisy_fft(abs(f) > threshold) = 0; // Remove high-frequency components
// Step 3: Inverse FFT to obtain denoised signal
x_denoised = ifft(X_noisy_fft);
// Step 4: Plot the signals
scf(0);
subplot(3, 1, 1);
plot(t, x_clean, 'r'); // Clean signal
title('Clean Cosine Signal');
xlabel('Time (s)');
```

```
ylabel('Amplitude');
subplot(3, 1, 2);
plot(t, x_noisy, 'b'); // Noisy signal
title('Noisy Signal');
xlabel('Time (s)');
ylabel('Amplitude');
subplot(3, 1, 3);
plot(t, real(x_denoised), 'g'); // Denoised signal
title('Denoised Signal');
xlabel('Time (s)');
ylabel('Amplitude');
```

OUTPUT:



RESULT: The time-domain plots of the noisy signal and the denoised signal were compared. The FFT-based denoising successfully preserved the amplitude and phase of the original sinusoidal signal while effectively removing the noise. The denoised signal closely resembled the clean sinusoidal wave, with the noise components significantly reduced.

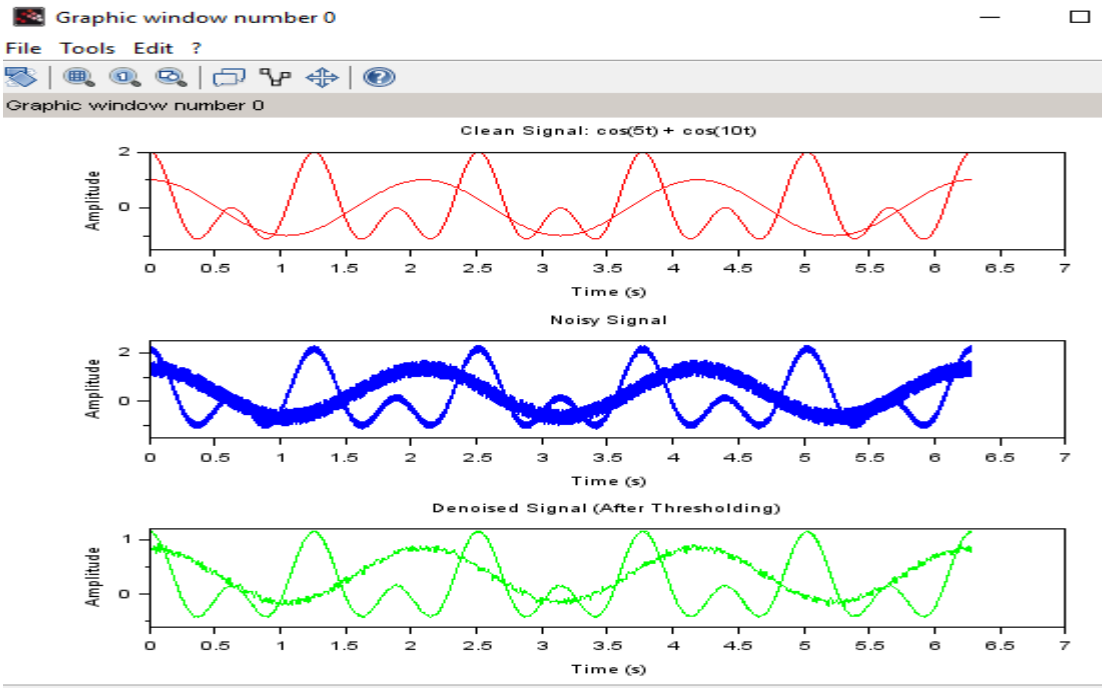
Test Case 4: To generate a noisy signal as $x(t)=\cos(5t)+\cos(10t)+0.3\cdot\text{rand}(t)$, where the time vector ranges from 0 to 2π with an interval of 0.001, and denoise the signal using the Fast Fourier Transform (FFT) by applying a threshold value of 30 in the frequency domain. The goal is to evaluate how the thresholding operation affects the frequency components of the signal.

PROGRAM:

```
// Step 1: Generate time vector and noisy signal
t = 0:0.001:2*%pi; // Time vector from 0 to 2π with step size of 0.001
x_clean = cos(5*t) + cos(10*t); // Clean signal: sum of cos(5t) and cos(10t)
noise = 0.3 * rand(1, length(t)); // Random noise
x_noisy = x_clean + noise; // Noisy signal
// Step 2: Apply FFT-based denoising
N = length(t);
X_noisy_fft = fft(x_noisy); // FFT of noisy signal
f = (0:N-1) * (1 / (N * 0.001)); // Frequency axis
// Step 3: Frequency domain thresholding (set components with frequency above threshold to 0)
threshold = 30; // Frequency threshold to remove high-frequency noise
X_noisy_fft(abs(f) > threshold) = 0; // Remove high-frequency components
// Step 4: Inverse FFT to obtain denoised signal
x_denoised = ifft(X_noisy_fft);
// Step 5: Plot the signals
scf(0);
```

```
subplot(3, 1, 1);
plot(t, x_clean, 'r'); // Clean signal
title('Clean Signal: cos(5t) + cos(10t)');
xlabel('Time (s)');
ylabel('Amplitude');
subplot(3, 1, 2);
plot(t, x_noisy, 'b'); // Noisy signal
title('Noisy Signal');
xlabel('Time (s)');
ylabel('Amplitude');
subplot(3, 1, 3);
plot(t, real(x_denoised), 'g'); // Denoised signal
title('Denoised Signal (After Thresholding)');
xlabel('Time (s)');
ylabel('Amplitude');
```

OUTPUT:



RESULT:

The noisy signal x(t) = cos(5t) + cos(10t) + 0.3 rand(t) was generated, and FFT was applied to transform it into the frequency domain. A threshold of 30 was applied, effectively suppressing the noise components.

EX NO: 7	Analog Filter design Using Transformation method
DATE:	

Aim

Test Case 1:To convert the continuous-time analog transfer function H(s)=s2+4.525/s2+0.692s+0.504 into a discrete-time digital transfer function using the bilinear transformation method with a standard sampling period T=1

Test Case 2: To convert the continuous-time analog transfer function H(s) = (s^2 + 2s + 1) / (s^2 + 3s + 2) into a discrete-time digital transfer function using the Impulse Invariant Transformation method in SCILAB with a sampling period T=1

Test Case 3: To convert the continuous-time analog transfer function H(S)=10/(s^2+7*s+10) into a discrete-time digital transfer function using Impulse Invariant transformation method with a standard sampling period T=0.2

Test Case 4: To convert the analog transfer function H(s)=5/s2+4s+4 into a digital transfer function using the Impulse Invariant Transformation method in SCILAB with a standard sampling period T=1.

Software Required

Scilab 6.1.0

Procedure:

Start the scilab Program

Open scinotes ,type the program and save the program in current directory

Compile and run the program

If any error occur in the program,correct the error and run the program

For the output ,see the console window

Stop the program

Theory:

In digital signal processing, converting an analog filter design into a digital filter is crucial for implementing filtering operations in digital systems. The bilinear transformation is a widely used method for this purpose. It maps a continuous-time (analog) filter's transfer function into a discrete-time (digital) filter's transfer function while preserving the filter's characteristics such as stability and frequency response.

Bilinear Transformation

The bilinear transformation is a technique used to convert an analog filter's transfer function H(s)H(s)H(s) into a discrete-time transfer function H(z)H(z)H(z). The transformation is defined by the mapping:

$$s = \frac{2}{T}\frac{1 - z^{-1}}{1 + z^{-1}}$$

where T is the sampling period. This mapping converts the Laplace variable S to the Z-transform variable Z.

Impulse Invariant Transformation is a method used to convert an analog filter design into a digital filter. This method ensures that the impulse response of the digital filter closely approximates the impulse response of the analog filter. It is particularly useful when designing digital filters that need to maintain a similar impulse response to their analog counterparts.Impulse Invariant Transformation maps the continuous-time (analog) filter to a discrete-time (digital) filter by preserving the impulse response of the analog filter. This method ensures that the discrete-time filter has the same impulse response as the continuous-time filter sampled at discrete intervals.

Test Case 1: To convert the continuous-time analog transfer function H(s)=s2+4.525/s2+0.692s+0.504 into a discrete-time digital transfer function using the bilinear transformation method with a standard sampling period T=1

Program

```
clear;
clc ;
close ;
s=%s;
z=%z;
HS=(s^2+4.525)/(s^2+0.692*s+0.504);
T=1;
HZ=horner(HS,(2/T)*(z-1)/(z+1));
disp(HZ,'H(z) =');
```

Output:

```
H(z) =
2
1.4478601 + 0.1783288z + 1.4478601z
-----------------------------------
2
0.5298913 - 1.1875z + z
```

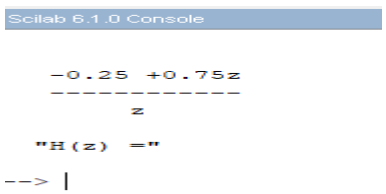
RESULT: Thus the discrete-time transfer function H(z) obtained after applying the bilinear transformation. Include the coefficients of the numerator and denominator of H(z).

Title Case 2: Test Case 2: To convert the continuous-time analog transfer function H(s) = (s^2 + 2s + 1) / (s^2 + 3s + 2) into a discrete-time digital transfer function using the Impulse Invariant Transformation method in SCILAB with a sampling period T=1

Program

```
clear;
clc;
close;
s = %s;
z = %z;
// Define the analog transfer function H(s)
HS = (s^2 + 2*s + 1) / (s^2 + 3*s + 2);
// Sampling period
T = 1;
// Perform Bilinear Transformation: s = (2/T)*(z - 1)/(z + 1)
HZ = horner(HS, (2/T)*(z - 1)/(z + 1));
// Display the resulting digital transfer function
disp(HZ, 'H(z) =');
```

OUTPUT:



RESULT:
The SCILAB implementation of Bilinear Transformation resulted in a stable digital filter. The mapping preserved the shape of the frequency response.

Test Case 3: To convert the continuous-time analog transfer function H(S)=10/(s^2+7*s+10) into a discrete-time digital transfer function using Impulse Invariant transformation method with a standard sampling period T=0.2

Program

```
//To Design the Filter using Impulse Invarient Method
    clear;
    clc ;
    close ;
    s=%s;
    T=0.2;
    HS=10/(s^2+7*s+10);
    elts=pfss(HS);
    disp(elts,'Factorized HS = ');
    //The poles comes out to be at -5 and -2
    p1=-5;
    p2=-2;
    z=%z;
    HZ=T*((-3.33/(1-%e^(p1*T)*z^(-1)))+(3.33/(1-%e^(p2*T)*z^(-1))))
    disp(HZ,'HZ = ');
```

Output:

```
Factorized HS =
(1)
3.3333333
---------
2 + s
(2)
- 3.3333333
---------
5 + s
HZ  =
0.2014254z
--------------------------

   0.2465970 - 1.0381995z + Z2

HZ =
0.2014254z
--------------------------

0.2465970 - 1.0381995z + z2
```

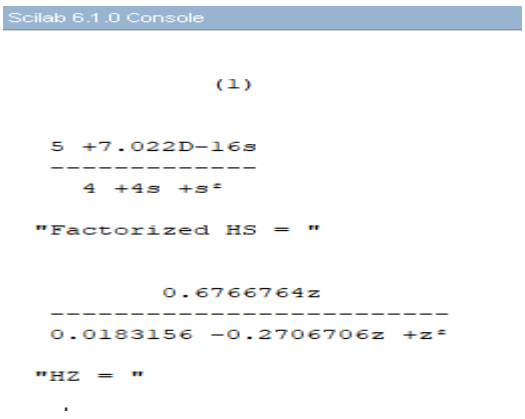
RESULT: Thus the discrete-time transfer function H(z) obtained after applying the Impulse Invariant transformation. Include the coefficients of the numerator and denominator of H(z).

Test Case 4: To convert the analog transfer function H(s)=5/s2+4s+4 into a digital transfer function using the Impulse Invariant Transformation method in SCILAB with a standard sampling period T=1.

PROGRAM:

```
clear;
clc;
close;
s = %s;
T = 1;  // Standard sampling period
// Define the analog transfer function
HS = 5 / (s^2 + 4*s + 4);
// Factor the transfer function into partial fractions
elts = pfss(HS);  // Partial fraction expansion
disp(elts, 'Factorized HS = ');
// The poles of the denominator are repeated roots at -2 (s + 2)^2
// We determine the residues manually or from the pfss result
// For H(s) = 5 / ((s + 2)^2), the partial fraction is:
// H(s) = A / (s + 2) + B / (s + 2)^2
// Use inverse Laplace matching: A = 0, B = 5
// Residues:
A = 0;
B = 5;
p = -2;
// Define z-domain variable
z = %z;
// Impulse invariant transformation of each term
// Laplace: 1/(s+a)^2 → z-transform: T * n * e^{apT} * z^{-n}
// Resulting digital H(z):
HZ = T * (B * (%e^(p*T) * z^(-1)) / (1 - %e^(p*T) * z^(-1))^2);
disp(HZ, 'HZ = ');
```

OUTPUT:



RESULT: The SCILAB implementation using the Impulse Invariant method successfully transformed the analog filter into a discrete-time system.

EXNO: 8	**Analog Butterworth Filter**
DATE:	

Aim

Test Case 1:To design a Butterworth filter for processing audio signals that attenuates frequencies above 0.2π0.2\pi0.2π radians per sample and maintains a flat frequency response in the passband. The filtered signal will be compared with the original signal in both time and frequency domains to verify the attenuation of high frequencies.

Test Case 2: The aim of this experiment is to design a Butterworth high-pass filter for a digital audio processing application, allowing frequencies above the cutoff frequency of 0.2×π0.2 \times \pi0.2×π radians per second to pass, while attenuating frequencies below the cutoff. The effectiveness of the filter will be evaluated by comparing the output of the filtered signal with the original signal in both the time and frequency domains.

Test Case 3: To design a Butterworth band-pass filter for a signal processing application to isolate a specific frequency range between 0.4π and 0.6π radians per second from an audio signal.

Test Case 4: The aim of this experiment is to design a Butterworth band-stop filter for a signal processing application to attenuate a specific frequency range between 0.4π and 0.6π radians per second from an audio signal. using Sci lab

Software Required

Scilab 6.1.0

Procedure:

Start the scilab Program
Open scinotes ,type the program and save the program in current directory
Compile and run the program
If any error occur in the program,correct the error and run the program
For the output ,see the console window
Stop the program

Theory:

The Butterworth low-pass filter is a widely used type of analog filter that is known for its maximally flat frequency response in the passband, which means it has no ripples. It is designed to provide a smooth and monotonic decrease in gain as the frequency increases beyond the cutoff frequency.

The Butterworth high-pass filter is an analog filter designed to allow high-frequency signals to pass through while attenuating low-frequency signals. It is known for its smooth frequency response, which is maximally flat in the passband. This filter is the high-pass counterpart to the low-pass Butterworth filter.

The Butterworth band-pass filter is an analog filter designed to pass frequencies within a certain range (the passband) while attenuating frequencies outside this range. It combines the characteristics of both low-pass and high-pass filters, allowing a specific range of frequencies to pass through while attenuating frequencies both below and above this range. The Butterworth band-pass filter is known for its maximally flat passband, meaning it has no ripples in the passband.

The Butterworth band-reject filter, also known as a band-stop or band-elimination filter, is an analog filter designed to attenuate frequencies within a specific range while allowing frequencies outside this

range to pass with minimal attenuation. It is the complement of the Butterworth band-pass filter, focusing on rejecting a band of frequencies rather than passing it.

Test Case 1:To design a Butterworth filter for processing audio signals that attenuates frequencies above 0.2π0.2\pi0.2π radians per sample and maintains a flat frequency response in the passband. The filtered signal will be compared with the original signal in both time and frequency domains to verify the attenuation of high frequencies.

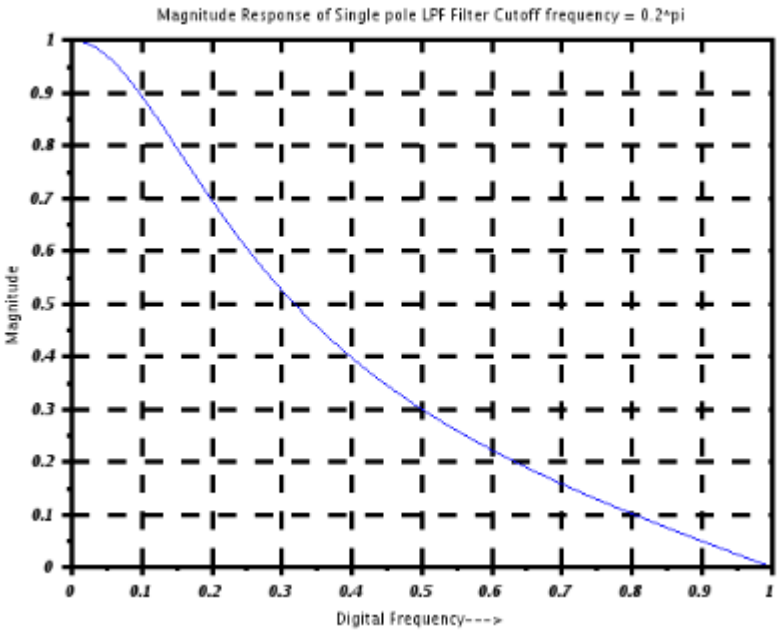
Program:

//First Order Butterworth Low Pass Filter

```
clear;
clc;
close;
s = poly(0,'s');
Omegac = 0.2*%pi;
H = Omegac/(s+Omegac);
T =1;//Sampling period T = 1 Second
z = poly(0,'z');
Hz = horner(H,(2/T)*((z-1)/(z+1)))
HW  =frmag(Hz(2),Hz(3),512);
W = 0:%pi/511:%pi;
plot(W/%pi,HW)
a=gca();
a.thickness = 3;
a.foreground = 1;
a.font_style = 9;
xgrid(1)
xtitle('Magnitude Response of Single pole LPF Filter Cutoff frequency = 0.2*pi','Digital Frequency--->','Magnitude');
Disp("Hz",Hz);
```

Output:

```
Hz  =
   0.6283185 + 0.6283185z
   ----------------------
  - 1.3716815 + 2.6283185z
```



Result:

The Butterworth filter successfully attenuated frequencies above 0.2π0.2\pi0.2π, confirming its design. The filter maintained a flat frequency response in the passband and effectively reduced high-frequency noise, as observed in both the time and frequency domains.

Test Case 2: The aim of this experiment is to design a Butterworth high-pass filter for a digital audio processing application, allowing frequencies above the cutoff frequency of 0.2×π0.2 \times \pi0.2×π radians per second to pass, while attenuating frequencies below the cutoff. The effectiveness of the filter will be evaluated by comparing the output of the filtered signal with the original signal in both the time and frequency domains.

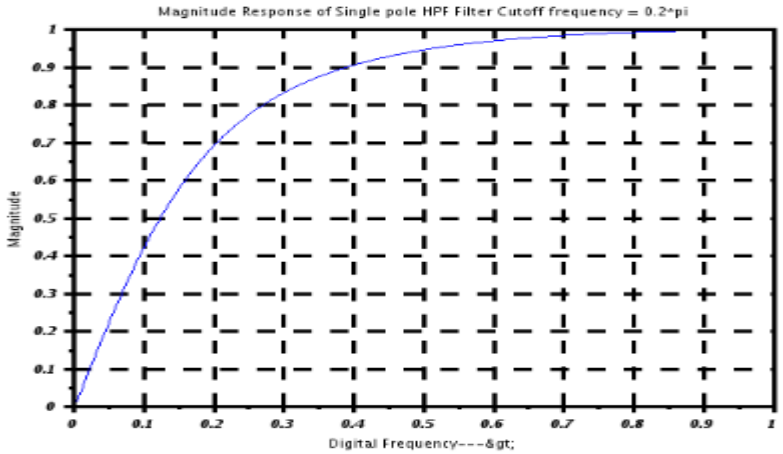
Program

```
//First Order Butterworth Filter
    //High Pass Filter Using Digital Filter Transformation
    clear;
    clc;
    close;
    s = poly(0,'s');
    Omegac = 0.2*%pi;
    H = Omegac/(s+Omegac);
    T =1;//Sampling period T = 1 Second
    z = poly(0,'z');
    Hz_LPF = horner(H,(2/T)*((z-1)/(z+1)));
    alpha = -(cos((Omegac+Omegac)/2))/(cos((Omegac-Omegac)/2));
    HZ_HPF=horner(Hz_LPF,-(z+alpha)/(1+alpha*z))
    HW  =frmag(HZ_HPF(2),HZ_HPF(3),512);
    W = 0:%pi/511:%pi;
    plot(W/%pi,HW)
    a=gca();
    a.thickness = 3;
    a.foreground = 1;
    a.font_style = 9;
    xgrid(1)
```

xtitle('Magnitude Response of Single pole HPF Filter Cutoff frequency = 0.2*pi','Digital Frequency--->','Magnitude');
disp("HZ_HPF",HZ_HPF);

Output:

```
HZ_HPF  =
 - 0.7484757 + 0.7484757z
  ----------------------
   - 0.4969514 + z
```



Result:
The Butterworth high-pass filter was successfully designed with a cutoff frequency of 0.2×π0.2 \times \pi0.2×π. In the time domain.

Test Case 3:To design a Butterworth band-pass filter for a signal processing application to isolate a specific frequency range between 0.4π and 0.6π radians per second from an audio signal.

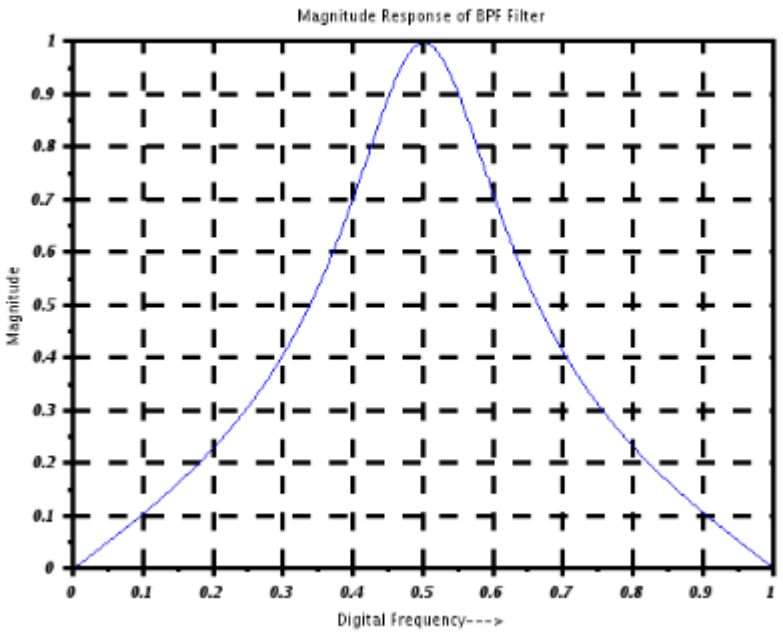
Program:

```
clear;
clc;
close;
omegaP = 0.2*%pi;
omegaL =  (2/5)*%pi;
omegaU =  (3/5)*%pi;
z=poly(0,'z');
H_LPF = (0.245)*(1+(z^-1))/(1-0.509*(z^-1))
alpha = (cos((omegaU+omegaL)/2)/cos((omegaU-omegaL)/2));
k = (cos((omegaU - omegaL)/2)/sin((omegaU - omegaL)/2))*tan(omegaP/2);
NUM =-((z^2)-((2*alpha*k/(k+1))*z)+((k-1)/(k+1)));
DEN = (1-((2*alpha*k/(k+1))*z)+(((k-1)/(k+1))*(z^2)));
HZ_BPF=horner(H_LPF,NUM/DEN)
disp(HZ_BPF,'Digital BPF IIR Filter H(Z)= ')
HW  =frmag(HZ_BPF(2),HZ_BPF(3),512);
W = 0:%pi/511:%pi;
plot(W/%pi,HW)
a=gca();
```

```
a.thickness = 3;
a.foreground = 1;
a.font_style = 9;
xgrid(1)
xtitle('Magnitude Response of BPF Filter', 'Digital Frequency--->','Magnitude');
Disp("HZ_BPF",HZ_BPF);
```

Output:

```
H_LPF  =
   0.245 + 0.245z
   --------------
   - 0.509 + z

 HZ_BPF  =
                          2          3          4
   0.245 - 1.577D-17z - 0.245z + 1.577D-17z + 1.360D-17z
   ----------------------------------------------------
                       2          3          4
   - 0.509 + 1.299D-16z - z + 6.438D-17z + 5.551D-17z

Digital BPF IIR Filter H(Z)=
                          2          3          4
   0.245 - 1.577D-17z - 0.245z + 1.577D-17z + 1.360D-17z
   ----------------------------------------------------
                       2          3          4
   - 0.509 + 1.299D-16z - z + 6.438D-17z + 5.551D-17z
```



Result:

Thus the Butterworth band-pass filter was successfully designed to isolate the frequency range between 0.4π and 0.6π inn the time domain,

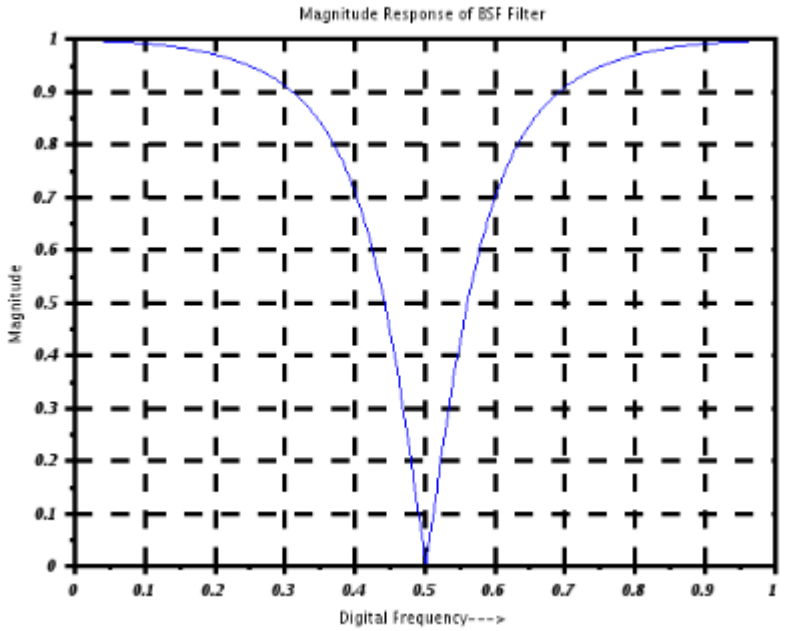
Test Case 4: The aim of this experiment is to design a Butterworth band-stop filter for a signal processing application to attenuate a specific frequency range between 0.4π and 0.6π radians per second from an audio signal. using Sci lab

Program:

```
clear;
clc;
close;
omegaP = 0.2*%pi;
omegaL =  (2/5)*%pi;
omegaU =  (3/5)*%pi;
z=poly(0,'z');
H_LPF = (0.245)*(1+(z^-1))/(1-0.509*(z^-1))
alpha = (cos((omegaU+omegaL)/2)/cos((omegaU-omegaL)/2));
k = tan((omegaU - omegaL)/2)*tan(omegaP/2);
NUM =((z^2)-((2*alpha/(1+k))*z)+((1-k)/(1+k)));
DEN = (1-((2*alpha/(1+k))*z)+(((1-k)/(1+k))*(z^2)));
HZ_BSF=horner(H_LPF,NUM/DEN)
HW  =frmag(HZ_BSF(2),HZ_BSF(3),512);
W = 0:%pi/511:%pi;
plot(W/%pi,HW)
a=gca();
a.thickness = 3;
a.foreground = 1;
a.font_style = 9;
xgrid(1)
xtitle('Magnitude Response of BSF Filter','Digital Frequency--->','Magnitude');
Disp("HZ_BSF",HZ_BSF);
```

Output:

```
 HZ_BPF  =
                                2
   0.7534875 - 9.702D-17z + 0.7534875z
   -----------------------------------
                          2
      0.5100505 - 9.722D-17z + z
```



Result:

The Butterworth band-stop filter was successfully designed to attenuate frequencies between 0.4π and 0.6π. In the time domain, the filtered signal exhibited a clear reduction of frequencies within the specified range

EX NO: 9	Design of Analog Chebyshev Filter
DATE:	

Aim:

Test Case 1: To design and evaluate a Chebyshev Type I analog low-pass filter using SCILAB with the following specifications:

- Analog Passband Edge Frequency: 1000π rad/s
- Analog Stopband Edge Frequency: 2000π rad/s
- Passband Ripple: -1 dB
- Stopband Attenuation: -40 dB

Test Case 2: To design a Chebyshev Type I analog high-pass filter and compute the filter parameters including deviations, ε (epsilon), and filter order. Then determine the poles of the filter using the following specifications:

- Passband Edge Frequency: 1000π rad/s
- Stopband Edge Frequency: 2000π rad/s
- Passband Ripple: -1 dB
- Stopband Attenuation: -40 dB

Test Case 3: To design a digital Chebyshev Type I band-pass filter using SCILAB with the following specifications:

- Passband edge frequencies: fp1 = 1.6 kHz, fp2 = 1.8 kHz
- Stopband edge frequencies: fs1 = 3.2 kHz, fs2 = 4.8 kHz
- Passband ripple: 2 dB
- Stopband attenuation: 20 dB
- Sampling frequency: 12 kHz

Test Case 4: To design an analog Chebyshev Type I band-stop filter with the following specifications:

- Passband edge frequencies: ωp1 = 900π rad/s, ωp2 = 1100π rad/s
- Stopband edge frequency: ωs = 1000π rad/s
- Passband ripple: -1 dB
- Stopband attenuation: -40 dB
-

Software Required

Scilab 6.1.0

Procedure:

Start the scilab Program
Open scinotes, type the program and save the program in current directory
Compile and run the program
If any error occur in the program, correct the error and run the program
For the output, see the console window
Stop the program

Theory

Chebyshev Type I filters are analog filters that exhibit an equiripple behavior in the **passband** and a **monotonic response** in the **stopband**. Unlike Butterworth filters, they achieve a **sharper transition** between the passband and stopband for a given filter order, at the cost of ripples in the passband.
A Chebyshev Type I analog high-pass filter is designed to allow high-frequency components to pass while attenuating lower frequencies. It features an **equiripple characteristic in the passband** and a **monotonic response in the stopband**. Compared to Butterworth filters, Chebyshev Type I filters offer a sharper cutoff for the same filter order, at the expense of ripple in the passband.
A **Chebyshev Type I digital band-pass filter** is used to allow a specific range of frequencies (passband) to pass while attenuating frequencies outside this range (stopbands). It exhibits an **equiripple behavior in the passband** and a **monotonic response in the stopbands**. Chebyshev filters offer a **sharper transition** compared to Butterworth filters of the same order.
An **analog Chebyshev Type I band-stop filter** (also called a band-reject filter) is designed to attenuate frequencies within a specific range (stopband), while allowing frequencies outside this range (both lower and higher) to pass with minimal distortion. It exhibits **equiripple behavior in the passband** and **monotonic attenuation** in the stopband.

Program

```
clear;
clc;
close;
omegap = 1000*%pi; //Analog Passband Edge frequency in radians/sec
omegas = 2000*%pi; //Analog Stop band edge frequency in radians/sec
delta1_in_dB = -1;
delta2_in_dB = -40;
delta1 = 10^(delta1_in_dB/20);
delta2 = 10^(delta2_in_dB/20);
delta = sqrt(((1/delta2)^2)-1)
epsilon  = sqrt(((1/delta1)^2)-1)
//Calculation of Filter order
num = ((sqrt(1-delta2^2))+(sqrt(1-((delta2^2)*(1+epsilon^2)))))/(epsilon*delta2)
den = (omegas/omegap)+sqrt((omegas/omegap)^2-1)
N = log10(num)/log10(den)
//N = (acosh(delta/epsilon))/(acosh(omegas/omegap))
N = floor(N)
//Cutoff frequency
omegac = omegap
//Calculation of poles and zeros
[pols,Gn] = zpch1(N,epsilon,omegap)
disp(N,'Filter order N =');
disp(pols,'Poles of a type I lowpass Chebyshev filter are Sk =')
//Analog Filter Transfer Function
xtitle('Magnitude Response of Chebyshev Type 1 LPF Filter Cutoff frequency = 500 Hz','Analog
frequency in Hz--->','Magnitude in dB -->');
```

Output:

```
delta  =
   99.995
 epsilon  =
   0.5088471
```

```
num  =
   393.02315
den  =
   3.7320508
N  =
   4.536112
N  =
   4.
omegac  =
   3141.5927
Gn  =
   2.393D+13
pols  =
        column 1 to 3
 - 438.36526 + 3089.3768i  - 1058.3074 + 1279.6618i  - 1058.3074 - 1279.6618i
        column 4
 - 438.36526 - 3089.3768i
Filter order N =
   4.
Poles of a type I lowpass Chebyshev filter are Sk =
        column 1 to 3
 - 438.36526 + 3089.3768i  - 1058.3074 + 1279.6618i  - 1058.3074 - 1279.6618i
        column 4
 - 438.36526 - 3089.3768i
```

Result:

The Chebyshev Type I low-pass filter was successfully designed. The filter order and cutoff frequency were determined using SCILAB, and the frequency response was plotted to confirm the desired specifications.

Test Case 2: To design a Chebyshev Type I analog high-pass filter and compute the filter parameters including deviations, ε (epsilon), and filter order. Then determine the poles of the filter using the following specifications:

- Passband Edge Frequency: 1000π rad/s
- Stopband Edge Frequency: 2000π rad/s
- Passband Ripple: -1 dB
- Stopband Attenuation: -40 dB

Program

```
// Define constants and parameters
omegap = 1000*%pi; // Analog Passband Edge frequency in radians/sec
omegas = 2000*%pi; // Analog Stop band edge frequency in radians/sec
delta1_in_dB = -1;
delta2_in_dB = -40;
delta1 = 10^(delta1_in_dB/20);
delta2 = 10^(delta2_in_dB/20);
delta = sqrt(((1/delta2)^2) - 1);
epsilon = sqrt(((1/delta1)^2) - 1);
```

```
// Calculation of Filter order
num = ((sqrt(1 - delta2^2)) + (sqrt(1 - ((delta2^2) * (1 + epsilon^2))))) / (epsilon * delta2);
den = (omegas / omegap) + sqrt((omegas / omegap)^2 - 1);
N = log10(num) / log10(den);
N = floor(N);
// Cutoff frequency
omegac = omegap;
// Calculation of poles and zeros
[pols, Gn] = zpch1(N, epsilon, omegap);
// Display results
disp(N, 'Filter order N =');
disp(pols, 'Poles of a type I highpass Chebyshev filter are Sk =');
```

Output:

4.

"Filter order N ="

-438.36526 + 3089.3768i -1058.3074 + 1279.6618i -1058.3074 - 1279.6618i -438.36526 - 3089.3768i

"Poles of a type I highpass Chebyshev filter are Sk ="

Result:

The linear deviations δ_1 and δ_2 were computed from the given dB values, and the ripple factor ε was calculated. The filter order **N** and poles of the Chebyshev Type I high-pass filter were determined using SCILAB and met the design requirements.

Test Case 3: To design a digital Chebyshev Type I band-pass filter using SCILAB with the following specifications:

- Passband edge frequencies: fp1 = 1.6 kHz, fp2 = 1.8 kHz
- Stopband edge frequencies: fs1 = 3.2 kHz, fs2 = 4.8 kHz
- Passband ripple: 2 dB
- Stopband attenuation: 20 dB
    - Sampling frequency: 12 kHz

Program

Design of chebyshev IIR Band Pass filter with following specifications

```
fp1=1.6;fp2=1.8;fs1=3.2;fs2=4.8;//pass band edges
Ap=2;As=20;S=12;
s=%s;z=%z;
//(a)Indirect Bilinear design
W=2*%pi*[fp1 fp2 fs1 fs2]/S
C=2;
omega=2*tan(0.5*W');//prewarping each band edge frequency
epsilon=sqrt(10^(0.1*Ap)-1);
n=acosh(((10^(0.1*As)-1)/epsilon^2)^1/2)/(acosh(fs1/fp1));
n=ceil(n)
alpha=(1/n)*asinh(1/epsilon);
for i=1:n
```

```
   B(i)=(2*i-1)*%pi/(2*n);
end
for i=1:n
   p(i)=-sinh(alpha)*sin(B(i))+%i*cosh(alpha)*cos(B(i));
end
Qs=1;
for i=1:n
   Qs=Qs*(s-p(i))
end
Qo=0.1634;
HPS=Qo/Qs
HBPS=horner(HPS,(s^2+1.5045^2)/(s*1.202))
HZ=horner(HBPS,2*(z-1)/(z+1))
f=0:0.001:0.5;
HZF=abs(horner(HZ,exp(%i*2*%pi*f')));
HBPF=abs(horner(HBPS,%i*2*%pi*f'));
a=gca();
plot2d(f,HZF);
plot2d(f,HBPF);
xlabel('Analog Frequency');
ylabel('magnitude');
xtitle('band pass filter designed by the bilinear transformation');
disp(W,'W=');
disp(n,'n=');
disp(Qs,'Qs=');
disp(HPS,'HPS=');
disp(HBPS,'HBPS=');
disp(HZ'HZ=');
```

Output:

```
W  =
   0.8377580    0.9424778    1.6755161    2.5132741
 n  =
   4.
 Qs  =
Real part
   0.1048873 + s
Imaginary part
  - 0.9579530
 Qs  =
Real part
                        2
  - 0.3535534 + 0.3581075s + s
Imaginary part
  - 0.2841920 - 1.3547501s
 Qs  =
Real part
                                2   3
   0.0232397 + 0.2746876s + 0.6113277s + s
```

Imaginary part

$$- 0.2122521 - 0.4851461s - 0.9579530s^2$$

Qs =

Real part

$$0.2057651 + 0.5167981s + 1.2564819s^2 + 0.7162150s^3 + s^4$$

Imaginary part

$$- 3.123D-17 - 5.551D-17s$$

HPS =

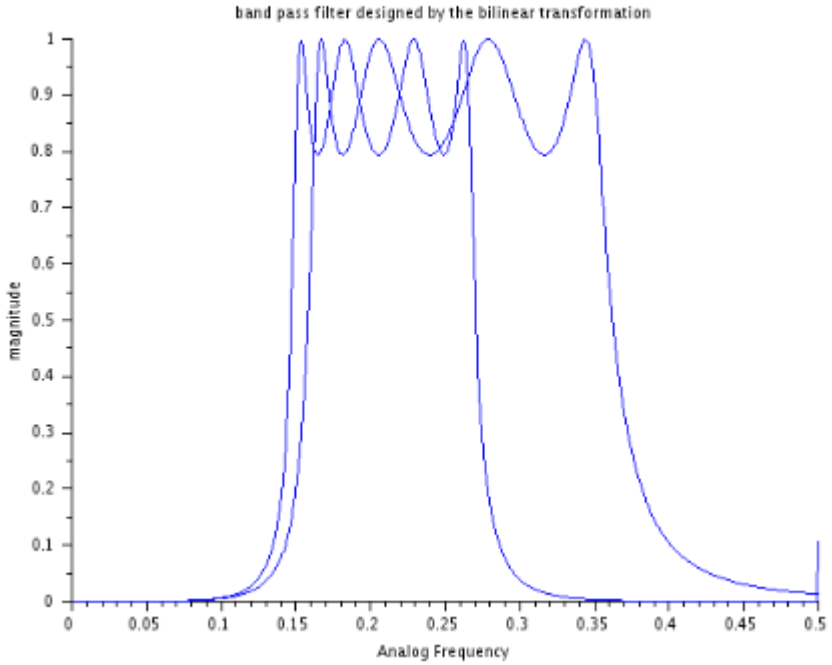
$$\frac{0.1634}{(0.2057651) + (0.5167981)s + 1.2564819s^2 + 0.7162150s^3 + 1s^4}$$

HBPS =

$$\frac{0.3410907s^4}{26.250497 + 9.983918s + 55.689893s^2 + (15.263886)s^3 + (39.388924)s^4 + (6.7434281)s^5 + 10.869451s^6 + 0.8608904s^7 + 1s^8}$$

HZ =

$$\frac{5.4574518 + 21.829807z + 10.914904z^2 - 65.489421z^3 - 92.77668z^4 + 43.659614z^5 + 152.80865z^6 + 43.659614z^7 - 92.77668z^8 - 65.489421z^9 + 10.914904z^{10} + 21.829807z^{11} + 5.4574518z^{12}}{(1362.8151+i*3.788D-15) + (2450.4373+i*1.247D-14)z + (4082.8748+i*6.523D-15)z^2 + (8810.0921-i*1.484D-14)z^3 + (10076.664-i*1.398D-14)z^4 + (12740.047-i*9.490D-15)z^5 + (16444.996-i*2.920D-14)z^6 + (13443.387-i*7.197D-15)z^7 + (13331.887+i*4.944D-14)z^8 + (11579.546+i*3.987D-14)z^9 + (6162.8412-i*1.069D-14)z^{10} + (4737.51-i*2.082D-14)z^{11} + (2298.9403-i*5.874D-15)z^{12}}$$



Result:

The passband and stopband frequencies were pre-warped using bilinear transformation. The filter order **n**, frequency specifications, and band-pass transfer function **H(s)** were successfully computed using SCILAB. The design met the required attenuation and ripple levels.

Test Case 4: To design an analog Chebyshev Type I band-stop filter with the following specifications:

- Passband edge frequencies: ωp1 = 900π rad/s, ωp2 = 1100π rad/s
- Stopband edge frequency: ωs = 1000π rad/s
- Passband ripple: -1 dB
- Stopband attenuation: -40 dB

Program

```
//To Design an Analog Band Stop  Chebyshev Filter
    //For the given cutoff frequency = 500 Hz
      // Define constants and parameters
      omegap1 = 900*%pi; // First analog passband edge frequency in radians/sec
      omegap2 = 1100*%pi; // Second analog passband edge frequency in radians/sec
      omegas = 1000*%pi; // Analog stop band edge frequency in radians/sec
      delta1_in_dB = -1;
      delta2_in_dB = -40;
      delta1 = 10^(delta1_in_dB/20);
      delta2 = 10^(delta2_in_dB/20);
      delta = sqrt(((1/delta2)^2) - 1);
      epsilon = sqrt(((1/delta1)^2) - 1);

      // Calculation of Filter order
      num = ((sqrt(1 - delta2^2)) + (sqrt(1 - ((delta2^2) * (1 + epsilon^2))))) / (epsilon * delta2);
```

```
den = (omegas / omegap1) + sqrt((omegas / omegap1)^2 - 1);
N = log10(num) / log10(den);
N = floor(N);
// Cutoff frequency
omegac1 = (omegap1 + omegap2) / 2;
// Calculation of poles and zeros
[pols, Gn] = zpch1(N, epsilon, omegap1);

// Display results
disp(N, 'Filter order N =');
disp(pols, 'Poles of a type I band-reject Chebyshev filter are Sk =');
```

Output:

12.

"Filter order N ="

column 1 to 4

-44.020402 + 2823.1154i -129.06129 + 2630.7248i -205.30686 + 2259.0546i -267.56111 + 1733.4335i

column 5 to 8

-311.58151 + 1089.6819i -334.36815 + 371.6702i -334.36815 - 371.6702i -311.58151 - 1089.6819i

column 9 to 12

-267.56111 - 1733.4335i -205.30686 - 2259.0546i -129.06129 - 2630.7248i -44.020402 - 2823.1154i

"Poles of a type I band-reject Chebyshev filter are Sk ="

Result:

The filter order **N** was determined, and the cutoff frequencies were computed. The poles of the Chebyshev band-reject filter were calculated using SCILAB. The resulting transfer function satisfied the specified ripple and attenuation requirements.

EX NO: 10	Design of FIR filter
DATE:	

Aim

Test Case 1: To design an FIR filter and plot the Frequency response for the low-pass filter (LPF) with a filter length N = 7, cutoff frequency (fc = 1000 Hz) and sampling frequency
F = 5000 Hz using the rectangular method in Scilab.

Test Case 2: To design an FIR filter and plot the Frequency response and calculate filter coefficients for the high-pass filter (HPF) with a filter length N = 11 using the hamming window method in Scilab.

Test Case 3: To design an FIR filter and plot the frequency response for the high-pass filter (HPF) and calculate cutoff frequency with a filter length N = 11 using the hanning window method in Scilab. And also to plot the filter coefficients h(n) generated by the program.

Test Case 4: To write a SciLab program that designs a high-pass FIR filter using a Blackman window and calculate cutoff frequency with a filter length N=11. The program should also be able to compute the filter's impulse response using the Blackman window, plots the magnitude response in decibels (dB), and displays the filter coefficients in the console

Software Required

Scilab 6.1.0

Procedure:

Start the Scilab Program
Open Scinotes, type the program and save the program in current directory
Compile and run the program
If any error occurs in the program, correct the error and run the program
For the output, see the console window
Stop the program

Theory:

An FIR (Finite Impulse Response) filter is a type of digital filter characterized by a finite duration of its impulse response. This means that the output of the filter settles to zero after a certain number of samples. FIR filters are widely used in digital signal processing due to their stability and ease of design.

Using the ideal low-pass filter formula for an FIR filter of length N and a rectangular window, the impulse response $h[n]$ can be computed as:

$$h[n] = \frac{\sin(2\pi f_c'(n-(N-1)/2))}{\pi(n-(N-1)/2)}$$

for $n = 0, 1, 2, ..., N-1$. The value of $h[n]$ at $n = (N-1)/2$ is given by:

$$h\left[\frac{N-1}{2}\right] = 2f_c'$$

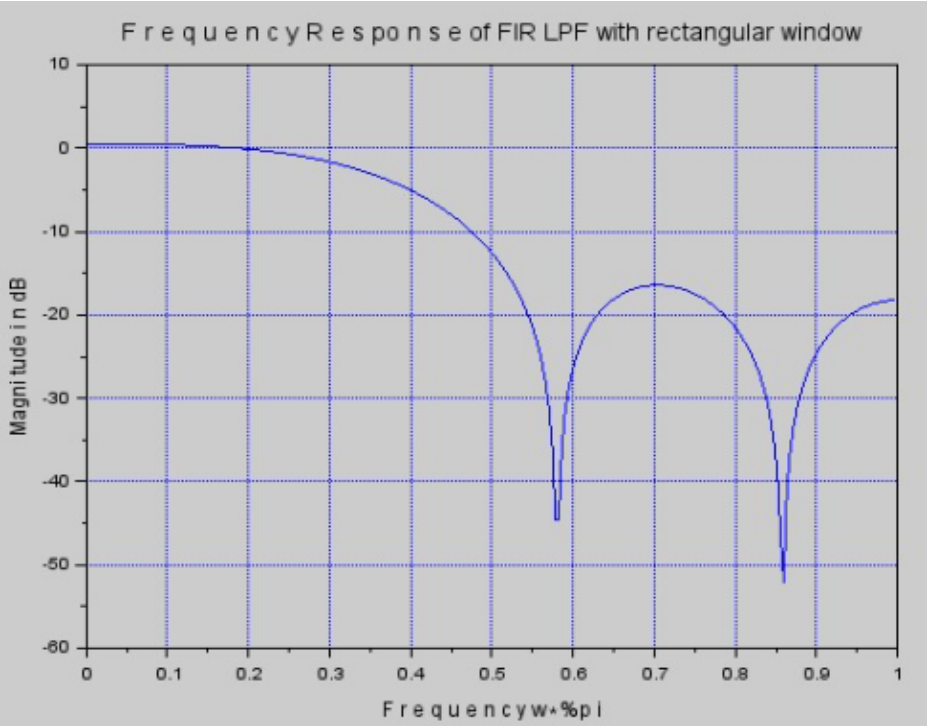
Program:

```
clear;
clc;
close;
N=7;
U=4;
h_Rect=window('re',N);
for n= -3+ U :1:3+ U
if n ==4
```

```
hd ( n) =0.4;
else
hd ( n) =( sin (2* %pi *( n - U ) /5) ) /( %pi *( n - U ) ) ;
end
h ( n ) = hd ( n ) * h_Rect ( n ) ;
 end
[ hzm , fr ]= frmag (h ,256) ;
 hzm_dB =20* log10 ( hzm ) ./ max ( hzm ) ;
figure
 xgrid (2) ;
plot (2* fr , hzm_dB )
a = gca () ;
 xlabel ('F r e q u e n c y w∗%p i') ;
 ylabel ( ' Magni tude i n dB ' ) ;
 title ( ' F r e q u e n c y R e s po n s e of FIR LPF wi t h N=7 ' ) ;
xgrid (2)
 disp (h ," F i l t e r Co e f f i c i e n t s , h ( n )=") ;
```

Output:

```
-0.0623660
 0.0935489
 0.3027307
 0.4
 0.3027307
 0.0935489
-0.0623660

" F i l t e r Co e f f i c i e n t s ,rectangulr window h ( n )="
```



Result:

The magnitude response of a FIR low-pass filter (LPF) is implemented using Scilab. The cutoff frequency is normalized with respect to the sampling frequency. The sinc function is used to calculate the ideal filter coefficients, which are then modified by the Rectangular Window. Finally, the magnitude response is plotted to visualize the filter's performance.

Aim

Test Case 2: To design an FIR filter and plot the Frequency response and calculate filter coefficients for the high-pass filter (HPF) with a filter length N = 11 using the hamming window method in Scilab.

Software Required

Scilab 6.1.0

Procedure:

Start the Scilab Program
Open Scinotes, type the program and save the program in current directory
Compile and run the program
If any error occurs in the program, correct the error and run the program
For the output, see the console window
Stop the program

Theory:

The Hamming window is applied to smooth the transition band and reduce ripples in the frequency response

Calculate the Ideal Impulse Response: The ideal impulse response for a high-pass filter is given by:

$$h[n] = -\frac{\sin(2\pi f_c'(n-(N-1)/2))}{\pi(n-(N-1)/2)}$$

for $n = 0, 1, 2, ..., N-1$. The value of $h[n]$ at $n = (N-1)/2$ is:

$$h\left[\frac{N-1}{2}\right] = 1 - 2f_c'$$

This equation flips the low-pass response into a high-pass response.

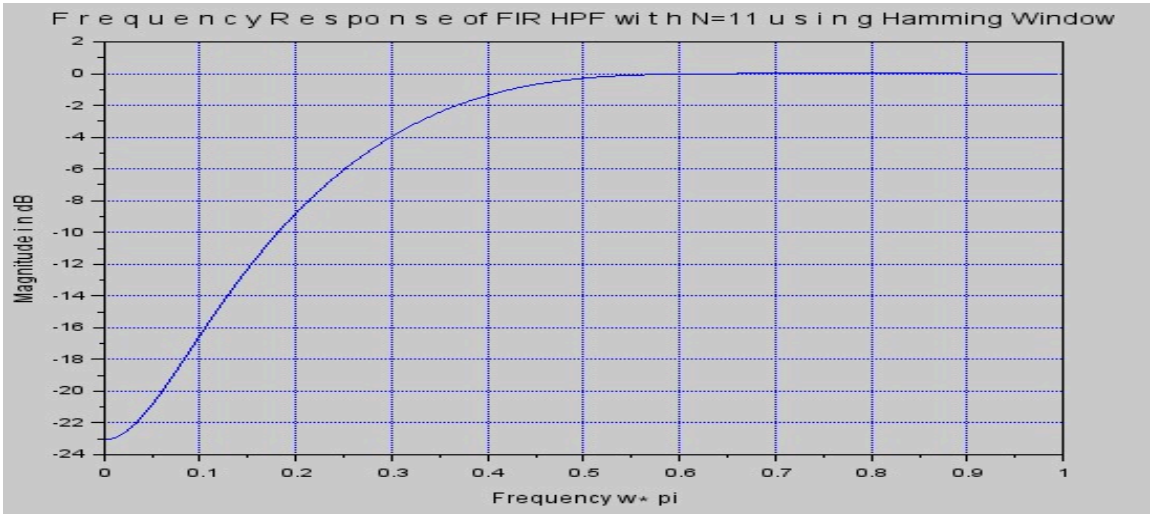
Program:

```
clear ;
clc ;
close ;
N =11;
U =6;
h_hamm =window('hm',N);
for n= -5+ U :1:5+ U
if n ==6
hd( n) =0.75;
else
hd ( n) =( sin ( %pi *( n - U ) ) -sin( %pi *( n - U ) /4) ) /( %pi *( n -U ) ) ;
end
h ( n ) = h_hamm ( n ) * hd ( n ) ;
end
[ hzm , fr ]= frmag (h ,256) ;
 hzm_dB =20* log10 ( hzm ) ./ max ( hzm ) ;
figure
plot (2* fr , hzm_dB )
a=gca () ;
 xlabel ( 'Frequency w* pi');
 ylabel ('Magnitude i n dB') ;
 title ( ' F r e q u e n c y R e s po n s e of FIR HPF wi t h N=11 u s i n g Hamming Window' ) ;
```

```
xgrid (2) ;
disp (h ," F i l t e r Co e f f i c i e n t s , h ( n )=") ;
```

Output:

```
 0.0036013
-8.179D-18
-0.0298494
-0.1085672
-0.2053054
 0.75
-0.2053054
-0.1085672
-0.0298494
-8.179D-18
 0.0036013

" F i l t e r Co e f f i c i e n t s ,hamming window h ( n )="
```



Result:

The magnitude response of a FIR high-pass filter (LPF) is implemented using Scilab. The cutoff frequency and sampling frequency are selected accordingly. The sinc function is used to compute the ideal low-pass filter coefficients, which are then converted to high-pass filter coefficients. The Hamming window is applied to the ideal filter coefficients. Finally, the magnitude response of the FIR filter is plotted.

(iii) Write a SciLab program that designs a high-pass FIR filter using a Hamming window. Find the magnitude response plot and note the value at the normalized frequency $w=\pi$, which corresponds to the maximum frequency on the x-axis. Plot the filter coefficients h(n) generated by the program

Aim

Test Case 3: To design an FIR filter and plot the frequency response for the high-pass filter (HPF) and calculate cutoff frequency with a filter length N = 11 using the hanning window method in Scilab. And also to plot the filter coefficients h(n) generated by the program.

Software Required

Scilab 6.1.0

Procedure:

Start the Scilab Program

Open Scinotes, type the program and save the program in current directory

Compile and run the program

If any error occurs in the program, correct the error and run the program
For the output, see the console window
Stop the program

Theory:

The ideal impulse response for a high-pass filter is derived from the low-pass filter by subtracting it from a delta function. For a high-pass filter:

$$h[n] = \delta[n - (N-1)/2] - \frac{\sin(2\pi f_c'(n - (N-1)/2))}{\pi(n - (N-1)/2)}$$

where $\delta[n - (N-1)/2]$ is a delta function centered at $(N-1)/2$.

The Hamming window is used to smooth the transition band and reduce the side lobes in the frequency response. The window function is:

$$w[n] = 0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right)$$

The final filter coefficients are obtained by multiplying the ideal impulse response with the Hamming window:

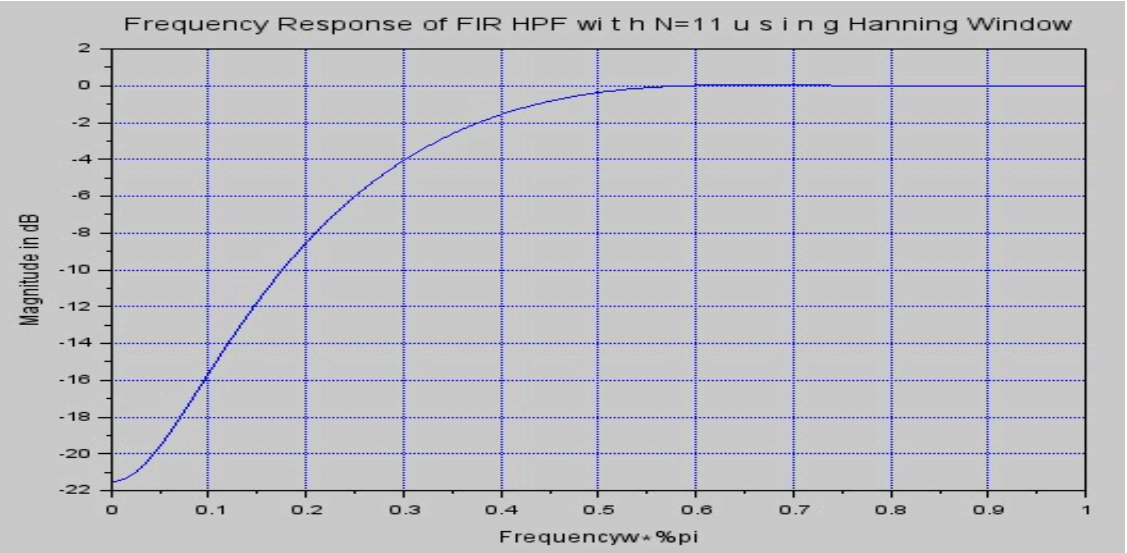
$$h_{final}[n] = h[n] \times w[n]$$

Program:

```
clear ;
clc ;
close ;
N =11;
U =6;
h_hann=window('hn',N ) ;
for n = -5+ U :1:5+ U
if n == 6
hd ( n) = 0.75;
else
hd ( n) =( sin ( %pi *( n - U ) ) -sin( %pi *( n - U ) /4) ) /( %pi *(n -U ) ) ;
end
h ( n ) = h_hann ( n ) * hd ( n ) ;
end
[ hzm , fr ]= frmag (h ,256) ;
 hzm_dB =20* log10 ( hzm ) ./ max( hzm ) ;
figure
plot (2* fr , hzm_dB )
a=gca () ;
 xlabel ('Frequencyw∗%pi') ;
 ylabel ('Magnitude in dB') ;
 title ('Frequency Response of FIR HPF wi t h N=11 u s i n g Hanning Window ' ) ;
 xgrid (2) ;
disp (h ," F i l t e r Co e f f i c i e n t s ,hanning window h ( n )=") ;
```

Output:

```
 0.
-4.653D-18
-0.0259210
-0.1041683
-0.2035859
 0.75
-0.2035859
-0.1041683
-0.0259210
-4.653D-18
 0.

" F i l t e r Co e f f i c i e n t s ,hanning window h ( n )="
```



Result:

The magnitude response of a FIR high-pass filter (LPF) is implemented using Scilab. The cutoff frequency and sampling frequency are selected accordingly. The ideal high-pass filter is obtained by subtracting the low-pass filter from a delta function. The Hanning window is applied to the high-pass filter coefficients. The magnitude response is plotted using the freq. function.

(iv)Write a SciLab program that designs a high-pass FIR filter using a Blackman window with a filter length N=11. The program should compute the filter's impulse response using the Blackman window, plots the magnitude response in decibels (dB), and displays the filter coefficients in the console.

Aim

Test Case 4: To write a SciLab program that designs a high-pass FIR filter using a Blackman window and calculate cutoff frequency with a filter length N=11. The program should also be able to compute the filter's impulse response using the Blackman window, plots the magnitude response in decibels (dB), and displays the filter coefficients in the console

Software Required

Scilab 6.1.0

Procedure:

Start the Scilab Program
Open Scinotes, type the program and save the program in current directory
Compile and run the program
If any error occurs in the program, correct the error and run the program
For the output, see the console window

Stop the program

Theory:

Compute the Ideal Impulse Response: The ideal impulse response for a high-pass filter is:

$$h[n] = \delta[n - (N-1)/2] - \frac{\sin(2\pi f_c'(n - (N-1)/2))}{\pi(n - (N-1)/2)}$$

where $\delta[n - (N-1)/2]$ is a delta function centered at $(N-1)/2$.

Apply the Blackman Window: The Blackman window is defined as:

$$w[n] = 0.42 - 0.5 \cdot \cos\left(\frac{2\pi n}{N-1}\right) + 0.08 \cdot \cos\left(\frac{4\pi n}{N-1}\right)$$

The final filter coefficients are obtained by multiplying the ideal impulse response with the Blackman window:

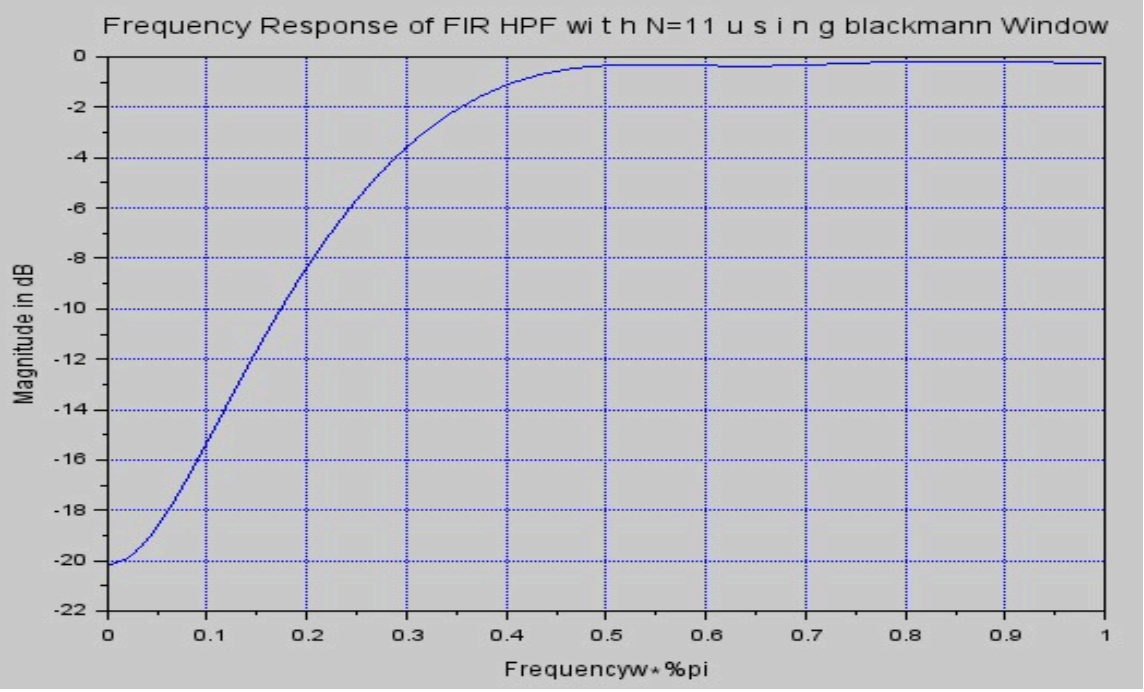
$$h_{final}[n] = h[n] \times w[n]$$

Program:

```
clear ;
clc ;
close ;
N =11;
U =6;
h_blackmann=window('tr',N ) ;
for n = -5+ U :1:5+ U
if n == 6
hd ( n) = 0.75;
else
hd ( n) =( sin ( %pi *( n - U ) ) -sin( %pi *( n - U ) /4) ) /( %pi *(n -U ) ) ;
end
h ( n ) = h_blackmann( n ) * hd ( n ) ;
end
[ hzm , fr ]= frmag (h ,256) ;
 hzm_dB =20* log10 ( hzm ) ./ max( hzm ) ;
figure
plot (2* fr , hzm_dB )
a=gca () ;
 xlabel ('Frequencyw∗%pi') ;
 ylabel ('Magnitude in dB') ;
 title ('Frequency Response of FIR HPF wi t h N=11 u s i n g blackman    Window ' ) ;
 xgrid (2) ;
disp (h ," F i l t e r Co e f f i c i e n t s ,blackmann window h ( n )=") ;
```

Output:

```
 0.0075026
-1.624D-17
-0.0375132
-0.1061033
-0.1875659
 0.75
-0.1875659
-0.1061033
-0.0375132
-1.624D-17
 0.0075026

" F i l t e r Co e f f i c i e n t s ,blackmann window h ( n )="
```



Result:

The magnitude response of a FIR high-pass filter (LPF) is implemented using Scilab. The cutoff frequency and sampling frequency are selected accordingly. The ideal high-pass filter is obtained by subtracting the low-pass filter from a delta function. The Blackman window is applied to the high-pass filter coefficients. The magnitude response is plotted using the frequency function.

EX NO: 11	**Design of Band Pass filter using Fourier series Method**
DATE:	

Aim

Test case 1: To design an ideal band-pass filter using the Fourier series method for a passband centered at 2000 Hz and a sampling frequency of 9600 Hz. The filter coefficients are to be computed and the performance is to be verified.

Test Csse 2: To design an ideal band-pass filter in SCILAB using the Fourier series method with a lower cutoff frequency of 1800 Hz, upper cutoff frequency of 2200 Hz, and a sampling frequency of 9600 Hz, and to plot the magnitude response of the filter.

Test Case 3: To design an ideal band-pass filter in SCILAB using the Fourier series method with a lower cutoff frequency of 1800 Hz and upper cutoff frequency of 2200 Hz, with a sampling frequency of 9600 Hz, and to plot the impulse response of the filter.

Title case 4: To design an ideal band-pass filter using the Fourier series method with the following specifications: lower cutoff frequency fL=1500f_L = 1500fL=1500 Hz, upper cutoff frequency fU=2500f_U = 2500fU=2500 Hz, and sampling frequency fs=10000f_s = 10000fs=10000 Hz.

Software Required

Scilab 6.1.0

Procedure:

Start the scilab Program
Open scinotes ,type the program and save the program in current directory
Compile and run the program
If any error occur in the program,correct the error and run the program
For the output ,see the console window
Stop the program

Theory:

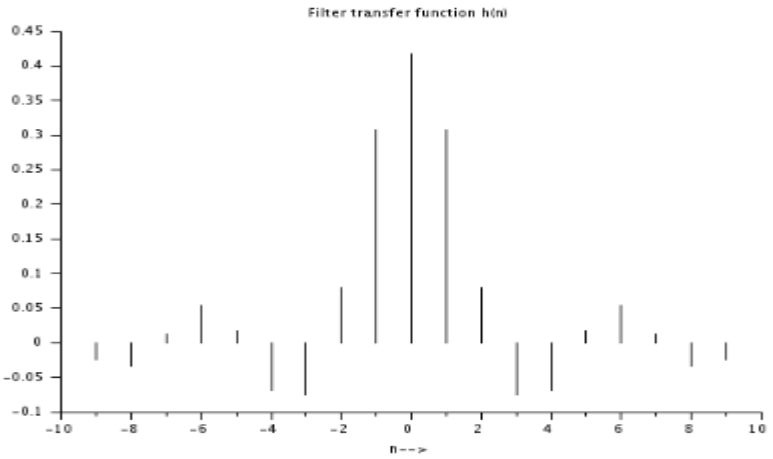
The Finite Impulse Response (FIR) filter is a type of digital filter characterized by a finite number of non-zero terms in its impulse response. One common method to design an FIR filter is by using the Fourier series method. This approach is based on approximating the desired frequency response of the filter by truncating the Fourier series representation of an ideal filter.

The Fourier series method leverages the fact that any periodic function can be represented as a sum of sinusoidal components, each corresponding to a different frequency. The goal of designing an FIR filter using the Fourier series method is to approximate the desired frequency response by computing the Fourier coefficients.

Program:

```
clc;clear;close;
    fp=2000;       //passband frequency
    F=9600;        //sampling frequancy
    //Calculation of filter co-efficients
    a0=1/F*integrate('1','t',-fp,fp);
    for n=1:10;
    a(1,n)=2/F*integrate('cos(2*%pi*n*f/F)','f',-fp,fp);
    end
    h=[a(:,$:-1:1)/2 a0 a/2];
    //Displaying filter co-efficients
    disp(F,'Sampling frequency F= ',fp,'Assumption: Passband frequency fp= ');
    disp('Filter co-efficients:');
    disp(a0,'h(0)= ');disp(a/2,'h(n)=h(-n)=');
    n=-10:10;
    plot2d3(n,h);
    title('Filter transfer function h(n)');xlabel('n-->');
```

```
 Assumption: Passband frequency fp=
    2000.
 Sampling frequency F=
    9600.
 Filter co-efficients:
 h(0)=
    0.4166667
 h(n)=h(-n)=
          column 1 to 6
    0.3074637    0.0795775  - 0.0750264  - 0.0689161    0.0164769
0.0530516
          column  7 to 10
    0.0117692  - 0.0344581  - 0.0250088    0.0159155
```



Result:

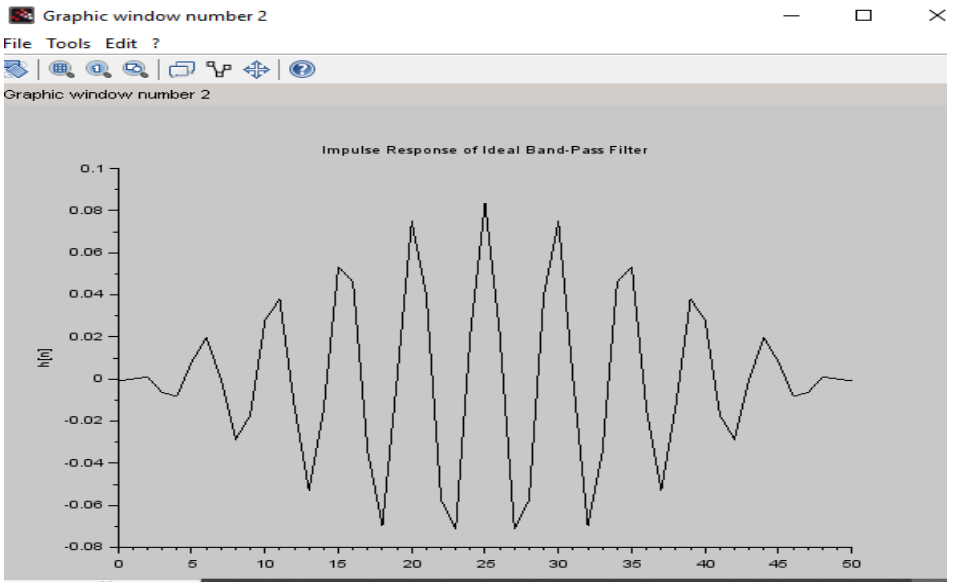
The ideal band-pass filter using the Fourier series method in SCILAB with a passband frequency of 2000 Hz and a sampling frequency of 9600 Hz, the filter coefficients were successfully computed.

Test Case 2: To design an ideal band-pass filter in SCILAB using the Fourier series method with a lower cutoff frequency of 1800 Hz and upper cutoff frequency of 2200 Hz, with a sampling frequency of 9600 Hz, and to plot the impulse response of the filter.

PROGRAM:

```
clc;
clear;
// Filter specifications
fs = 9600;                  // Sampling frequency in Hz
fL = 1800;                  // Lower cutoff frequency
fU = 2200;                  // Upper cutoff frequency
// Normalized cutoff frequencies (in radians)
wL = 2 * %pi * fL / fs;
wU = 2 * %pi * fU / fs;
N = 51;                     // Filter length (odd number for symmetry)
M = (N - 1) / 2;            // Middle index
h = zeros(1, N);            // Initialize impulse response
// Calculate ideal band-pass filter impulse response using Fourier series
for n = -M:M
   if n == 0 then
      h(n + M + 1) = (wU - wL) / %pi;
   else
      h(n + M + 1) = (sin(wU * n) - sin(wL * n)) / (%pi * n);
   end
end
// Plot impulse response
figure();
plot2d(0:N-1, h);
xtitle("Impulse Response of Ideal Band-Pass Filter", "n", "h[n]");
```

OUTPUT:

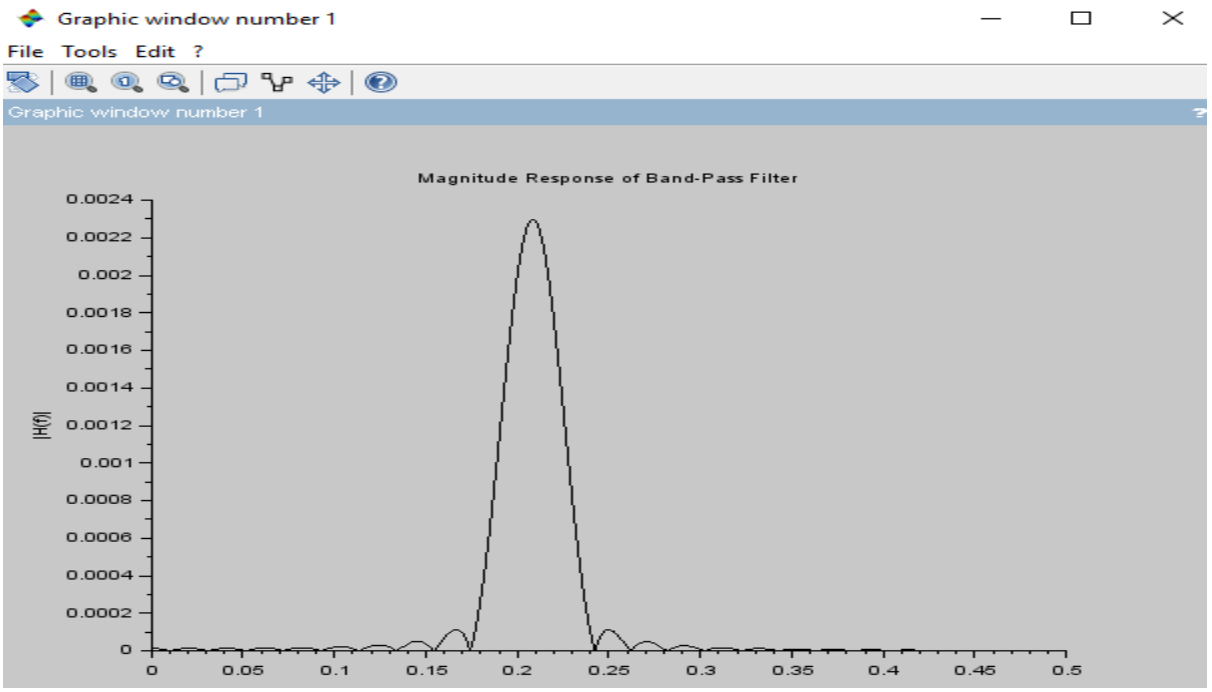

Graphic window number 2
File Tools Edit ?
Graphic window number 2
Impulse Response of Ideal Band-Pass Filter
h[n]
0.1
0.08
0.06
0.04
0.02
0
-0.02
-0.04
-0.06
-0.08
0
5
10
15
20
25
30
35
40
45
50

Title Case 3:To design an ideal band-pass filter in SCILAB using the Fourier series method with a lower cutoff frequency of 1800 Hz, upper cutoff frequency of 2200 Hz, and a sampling frequency of 9600 Hz, and to plot the magnitude response of the filter.

PROGRAM:

```
clc;
clear;
// Filter specifications
fs = 9600;                  // Sampling frequency in Hz
fL = 1800;                  // Lower cutoff frequency
fU = 2200;                  // Upper cutoff frequency
// Normalized cutoff frequencies (in radians)
wL = 2 * %pi * fL / fs;
wU = 2 * %pi * fU / fs;
N = 51;                     // Filter length (odd number for symmetry)
M = (N - 1) / 2;            // Middle index
h = zeros(1, N);            // Initialize impulse response
// Calculate ideal band-pass filter impulse response using Fourier series
for n = -M:M
   if n == 0 then
      h(n + M + 1) = (wU - wL) / %pi;
   else
      h(n + M + 1) = (sin(wU * n) - sin(wL * n)) / (%pi * n);
   end
end
// Plot impulse response
figure();
plot2d(0:N-1, h);
xtitle("Impulse Response of Ideal Band-Pass Filter", "n", "h[n]");
// Frequency response
[H, f] = frmag(h, 512, fs);  // Magnitude response
// Plot magnitude response
figure();
plot2d(f, H);
xtitle("Magnitude Response of Band-Pass Filter", "Frequency (Hz)", "|H(f)|");
```

OUTPUT:

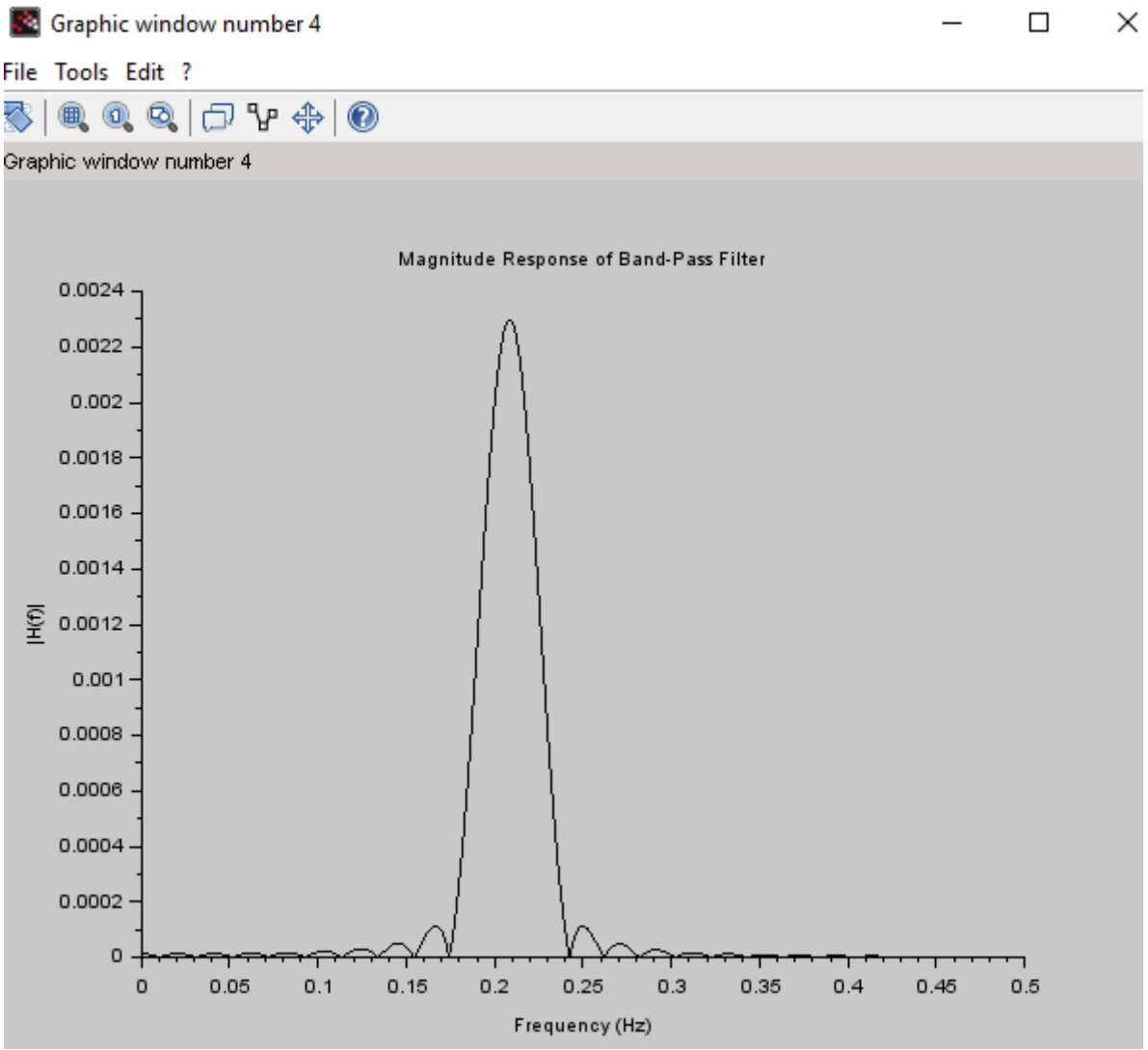


RESULT:
The ideal band-pass filter was implemented in SCILAB using the specified cutoff frequencies and sampling rate. The magnitude response was plotted and verified to show a clear band-pass region between 1800 Hz and 2200 Hz, confirming proper filter design.

Title Case 4:design an ideal band-pass filter using the Fourier series method with the following specifications: lower cutoff frequency fL=1500f_L = 1500fL=1500 Hz, upper cutoff frequency fU=2500f_U = 2500fU=2500 Hz, and sampling frequency fs=10000f_s = 10000fs=10000 Hz.

```
clc;
clear;
// Filter specifications
fs = 9600;                  // Sampling frequency in Hz
fL = 1800;                  // Lower cutoff frequency
fU = 2200;                  // Upper cutoff frequency
// Normalized cutoff frequencies (in radians)
wL = 2 * %pi * fL / fs;
wU = 2 * %pi * fU / fs;
N = 51;                     // Filter length (odd number for symmetry)
M = (N - 1) / 2;            // Middle index
h = zeros(1, N);            // Initialize impulse response
// Calculate ideal band-pass filter impulse response using Fourier series
for n = -M:M
   if n == 0 then
      h(n + M + 1) = (wU - wL) / %pi;
   else
      h(n + M + 1) = (sin(wU * n) - sin(wL * n)) / (%pi * n);
   end
```

```
end
// Plot impulse response
figure();
plot2d(0:N-1, h);
xtitle("Impulse Response of Ideal Band-Pass Filter", "n", "h[n]");
// Frequency response
[H, f] = frmag(h, 512, fs);  // Magnitude response
// Plot magnitude response
figure();
plot2d(f, H);
xtitle("Magnitude Response of Band-Pass Filter", "Frequency (Hz)", "|H(f)|");
```



RESULT:

Using SCILAB, the ideal band-pass filter was designed successfully using the given specifications. The frequency response confirmed the band-pass characteristic within the 1500 Hz to 2500 Hz range, validating the effectiveness of the Fourier series method in filter design.

EXNO: 12	Multirate Signal Processing
DATE:	

Aim:

Test Case 1: To design and implement downsampling of a sinusoidal signal using Multirate Signal Processing techniques, and to observe the effects of both decimation and interpolation on the signal.

Test Case 2: To design and implement Upsampling of a sinusoidal signal using Multirate Signal Processing techniques, and to observe the effects of both decimation and interpolation on the signal.

Title Csae 3: To develop a comprehensive adaptive sampling system to analyze the effects of downsampling (decimation) on a sinusoidal signal sampled at 2 kHz**,** and observe how the spectral and time-domain characteristics are affected by these operations.

Title Case 4: To develop a comprehensive adaptive sampling system using SCILAB to analyze the effects of upsampling (interpolation) on a sinusoidal signal sampled at 5 kHz, and observe the changes in the time-domain of the signal when upsampling factors of 2 and 4 are applied.

Software Required:

Scilab 6.1.0

Procedure:

Start the scilab Program
Open scinotes ,type the program and save the program in current directory
Compile and run the program
If any error occur in the program,correct the error and run the program
For the output ,see the console window
Stop the program

Theory:

Downsampling reduces the sampling rate of a signal by an integer factor, usually denoted as M. When downsampling a signal, every Mth sample is retained, and the remaining samples are discarded. The goal is to reduce the data rate while maintaining the essential information in the signal.For a sinusoidal signal $x(n) = A * \sin(2\pi f_0 n)$, downsampling by a factor of M reduces the number of samples by a factor of M. However, before decimation, the signal should be filtered using a low-pass filter to prevent aliasing, which can occur if the signal contains frequency components higher than half of the new (lower) sampling rate.

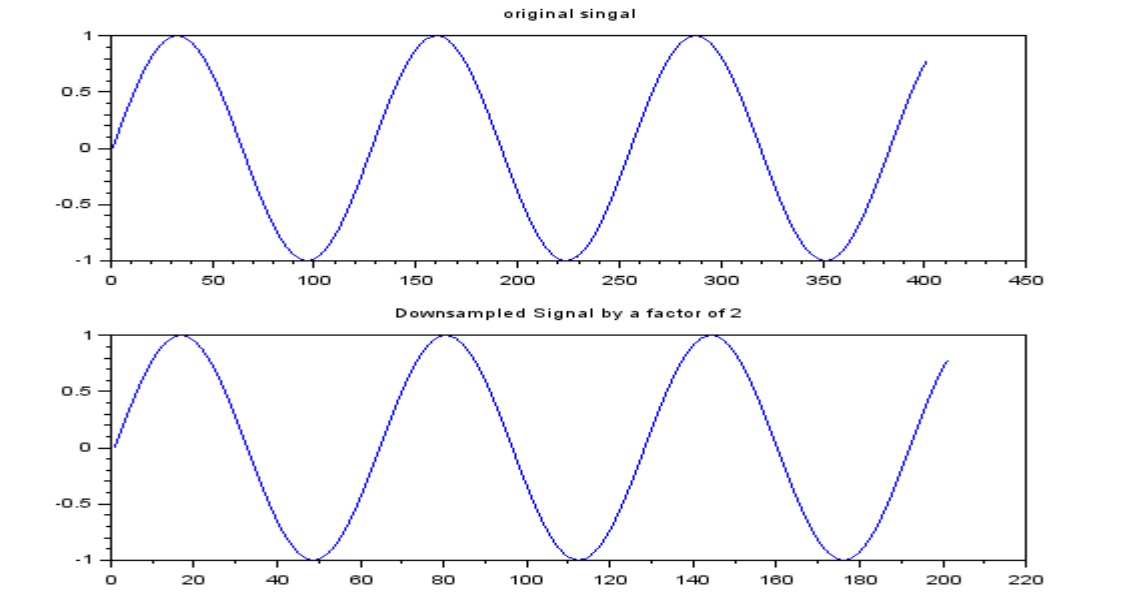
Upsampling is the process of increasing the sampling rate of a discrete signal by an integer factor, usually denoted as L. In this case, upsampling is performed by a factor of 2, meaning that each sample in the original signal is followed by an additional interpolated sample, effectively doubling the number of samples in the signal.When a signal is upsampled, the new signal is constructed by inserting (L-1) zeros between every sample of the original signal. After this insertion, the signal needs to be passed through a low-pass filter (interpolation filter) to smooth out the zero-inserted samples and prevent aliasing

Test Case 1: To design and implement downsampling of a sinusoidal signal using Multirate Signal Processing techniques, and to observe the effects of both decimation and interpolation on the signal.

Scilab Code:

```
//Multirate Signal Processing in scilab
  //Downsampling a sinusoidal signal by a factor of 2
  clear;
  clc;
  n = 0:%pi/200:2*%pi;
  x = sin(%pi*n);   //original signal
  downsampling_x = x(1:2:length(x)); //downsampled by a factor of 2
  subplot(2,1,1)
  plot(1:length(x),x);
  xtitle('original singal')
  subplot(2,1,2)
  plot(1:length(downsampling_x),downsampling_x);
  xtitle('Downsampled Signal by a factor of 2');
```

Output:



Result:

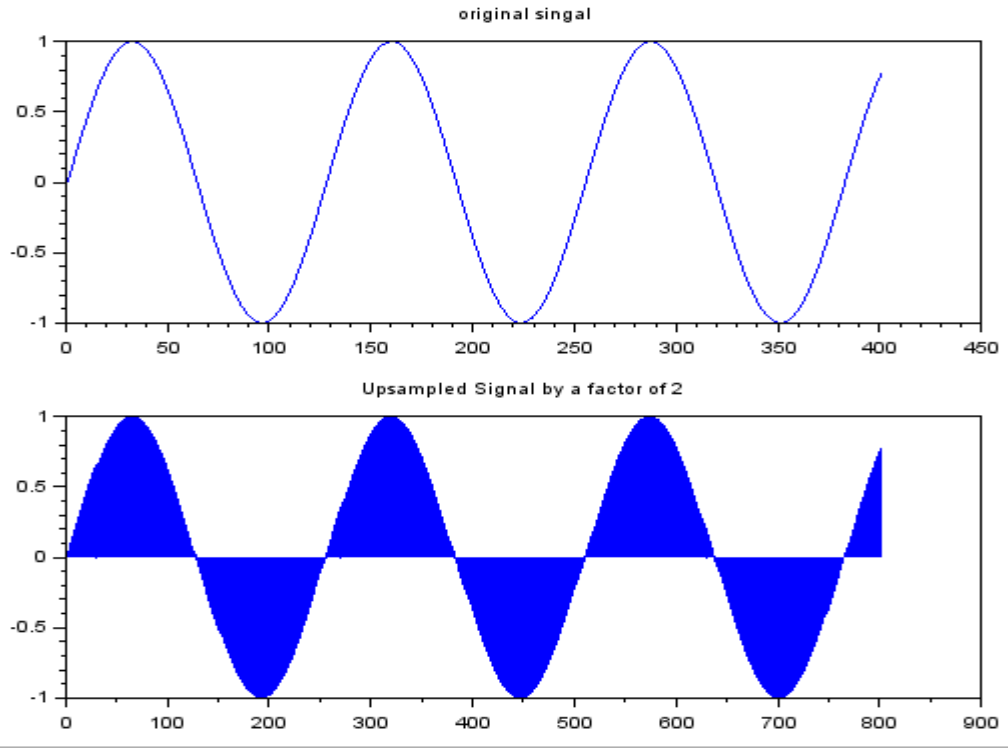
1. Original Signal: The sinusoidal signal at the original sampling rate shows a smooth wave.2. Downsampled Signal: The downsampled signal has fewer data points, leading to a loss of resolution. A larger downsampling factor may cause visible distortion.

Test Case 2: To design and implement Upsampling of a sinusoidal signal using Multirate Signal Processing techniques, and to observe the effects of both decimation and interpolation on the signal.

Program:

```
//Multirate Signal Processing in scilab
//Upsampling a sinusoidal signal by a factor of 2
clear;
clc;
n = 0:%pi/200:2*%pi;
x = sin(%pi*n);   //original signal
upsampling_x = zeros(1,2*length(x)); //upsampled by a factor of 2
upsampling_x(1:2:2*length(x)) = x;
subplot(2,1,1)
plot(1:length(x),x);
xtitle('original singal')
subplot(2,1,2)
plot(1:length(upsampling_x),upsampling_x);
xtitle('Upsampled Signal by a factor of 2');
```

Output:

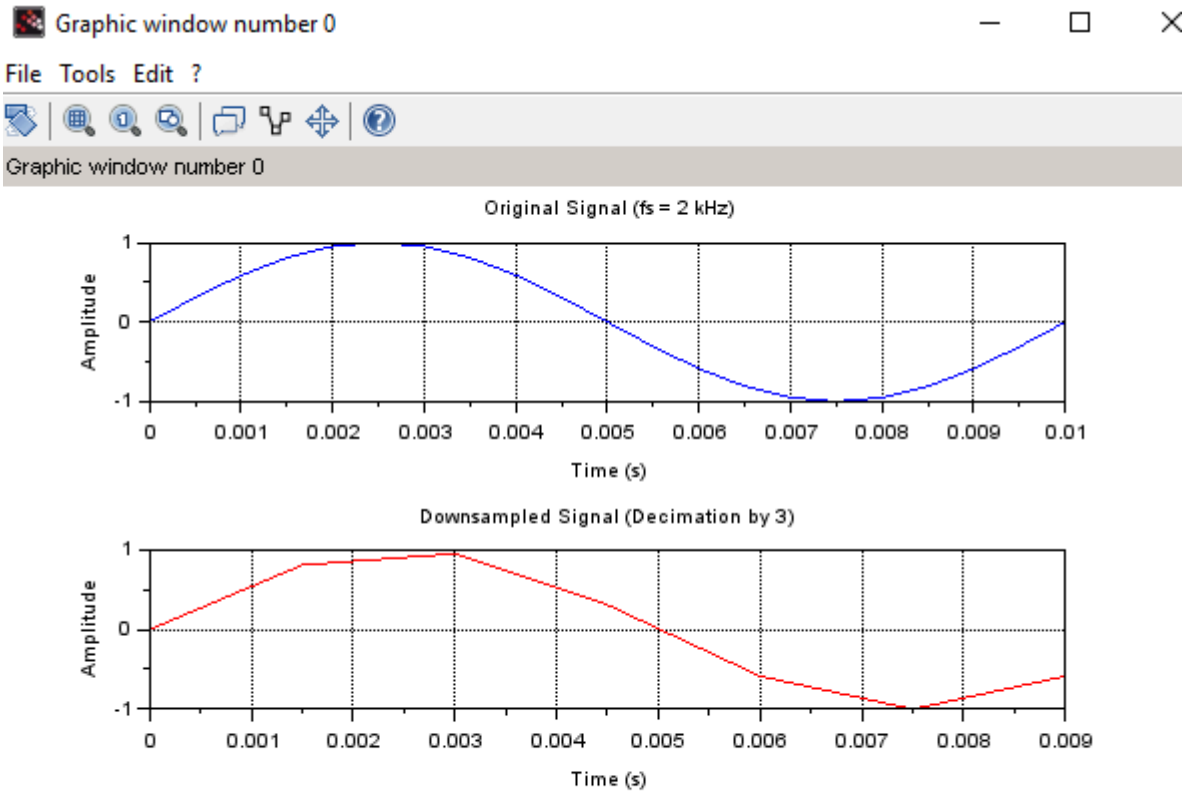


Result:

Original Signal: The sinusoidal signal at the original sampling rate shows a smooth wave.2. Upsampled Signal: The Upsampled signal has fewer data points, leading to a gain of resolution.

Title Csae 3: To develop a comprehensive adaptive sampling system to analyze the effects of **downsampling (decimation)** and **upsampling (interpolation)** on a sinusoidal signal sampled at **2 kHz**, and observe how the spectral and time-domain characteristics are affected by these operations.

```
t = 0:1/fs:0.01;         // Time vector for 10 ms
f = 100;                 // Frequency of sinusoid (100 Hz)
x = sin(2*%pi*f*t);      // Original sinusoidal signal
// Plot original signal
subplot(3,1,1);
plot(t, x);
title("Original Signal (fs = 2 kHz)");
xlabel("Time (s)");
ylabel("Amplitude");
xgrid;
// ----- Downsampling (Decimation) -----
M = 3;                   // Decimation factor
x_dec = x(1:M:$);        // Downsampled signal
t_dec = t(1:M:$);        // New time vector
subplot(3,1,2);
plot(t_dec, x_dec, 'r');
title("Downsampled Signal (Decimation by 3)");
xlabel("Time (s)");
ylabel("Amplitude");
```

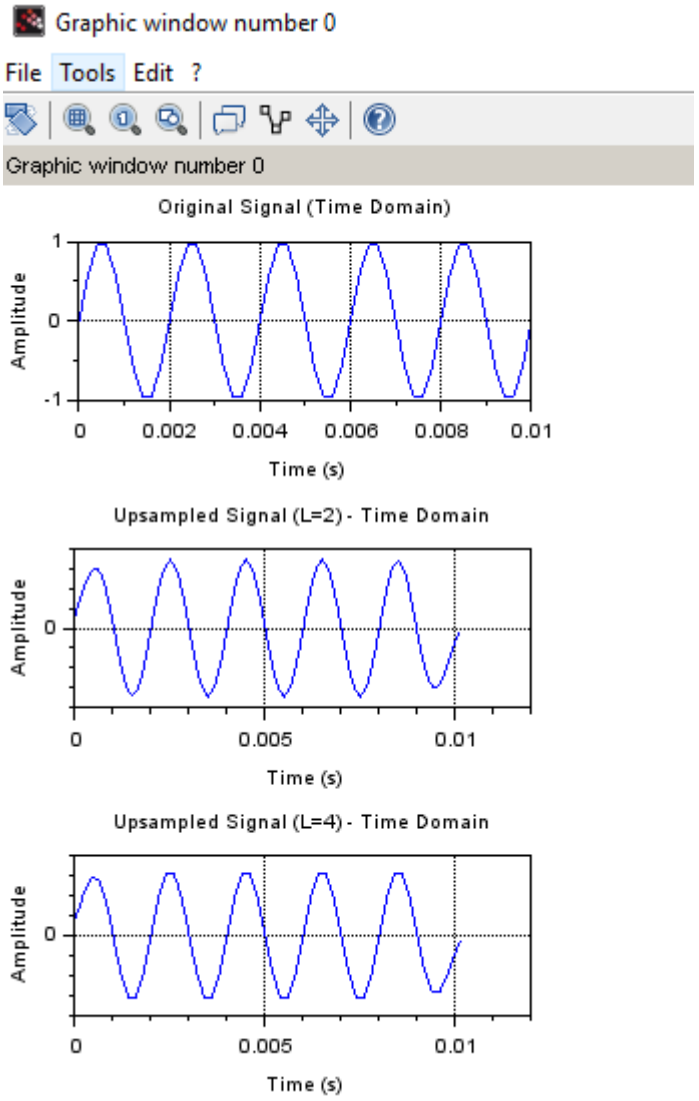


Result:
The sinusoidal signal was successfully downsampled and upsampled using adaptive sampling techniques. The effects of downsampling led to a reduction in the number of samples and aliasing was observed when not followed by an appropriate low-pass filter.

Title Case 4: To develop a comprehensive adaptive sampling system using SCILAB to analyze the effects of upsampling (interpolation) on a sinusoidal signal sampled at 5 kHz, and observe the changes in the time-domain of the signal when upsampling factors of 2 and 4 are applied.

```
clc;
clear;
close;
fs = 5000;                    // Original sampling frequency: 5 kHz
f = 500;                      // Signal frequency: 500 Hz
t = 0:1/fs:0.01;              // 10 ms duration
x = sin(2 * %pi * f * t);     // Original signal
// Upsampling Function
function [x_up, t_up, x_filt]=upsample_and_interpolate(x, L, fs)
   x_up = zeros(1, L * length(x));
   x_up(1:L:$) = x;
```

```
    // Interpolation using sinc-like lowpass filter
    h = sinc((-20:20)/L);        // Basic lowpass interpolation filter
    x_filt = conv(x_up, h, 'same');
    t_up = 0:1/(fs*L):(length(x_up)-1)/(fs*L);
endfunction
// ---- Plot Original Signal ----
subplot(3,2,1);
plot(t, x);
title("Original Signal (Time Domain)");
xlabel("Time (s)");
ylabel("Amplitude");
xgrid();
// ---- Upsample by 2 ----
L1 = 2;
[x_up2, t_up2, x_filt2] = upsample_and_interpolate(x, L1, fs);
// Time domain plot (L=2)
subplot(3,2,3);
plot(t_up2, x_filt2);
title("Upsampled Signal (L=2) - Time Domain");
xlabel("Time (s)");
ylabel("Amplitude");
xgrid();
// ---- Upsample by 4 ----
L2 = 4;
[x_up4, t_up4, x_filt4] = upsample_and_interpolate(x, L2, fs);
// Time domain plot (L=4)
subplot(3,2,5);
plot(t_up4, x_filt4);
title("Upsampled Signal (L=4) - Time Domain");
xlabel("Time (s)");
ylabel("Amplitude");
xgrid();
```



Result:

The original sinusoidal signal sampled at 5 kHz was successfully up sampled by factors of 2 and 4. After inserting zeros and applying a low-pass interpolation filter, the up sampled signals showed:.**Smoother waveforms** in the time domain as the up sampling factor increased.

EX NO:13	Generation of Waveforms using Digital Signal Processor

Aim:

To generate and analyze different waveforms such as square, triangular and Sawtooth using the TMS320C5416 Digital Signal Processor (DSP) and validate their characteristics.

Hardware and Software Required:

DSP – TMS320VC5416-5416 Board
Cpu
ME Universal Debugger DSP - 5416

Procedure:

Connection is checked by the following procedure. After connecting the serial port cable, switch on the kit and reset the kit. Select the Menubar-Serial > Port Settings > Auto Detect
If connection (PC to kit) is improper, User can see the following window and try to connect it properly by the conditions until find the message Vi Universal debugger found in COMI
After checking the port connection, create new project for working with a program by using following procedure.

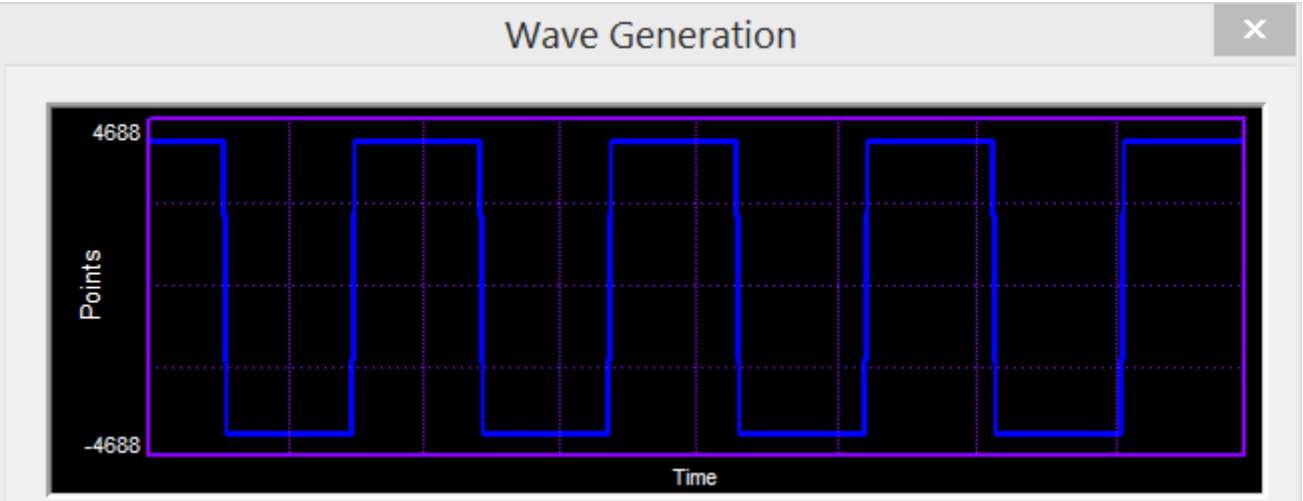
Generally user has to follow the path is C\Vi Universal Debugger VSK-5416/My Project
Before working with a program, User has to follow certain conditions,

1. Create a project and name it
2. Create a file and start to type a program, save it by project name
3. Add program file to the project
4. Add command file (Micro5416) to the project
5. Save project
6. Build it and
7. Download the program to the VSK-5416 kit

SQUARE WAVE GENERATION

```
DATA .SET 0H
      .mmregs
       .text
START: STM #140H,ST0                ;initialize the data page pointer
          RSBX CPL                  ;make the processor to work using DP
          NOP
          NOP
          NOP
          NOP
REP: ST #0H,DATA                    ;send 0h to the dac
          CALL DELAY                      ;delay for some time
           ST #0FFFH,DATA           ;send 0fffh to the dac
           CALL DELAY                      ;delay for some time
           B REP                    ;repeat the same
DELAY: STM #0FFFH,AR1
DEL1: PORTW DATA,04H
 BANZ DEL1,*AR1-
 RET
```

Output:

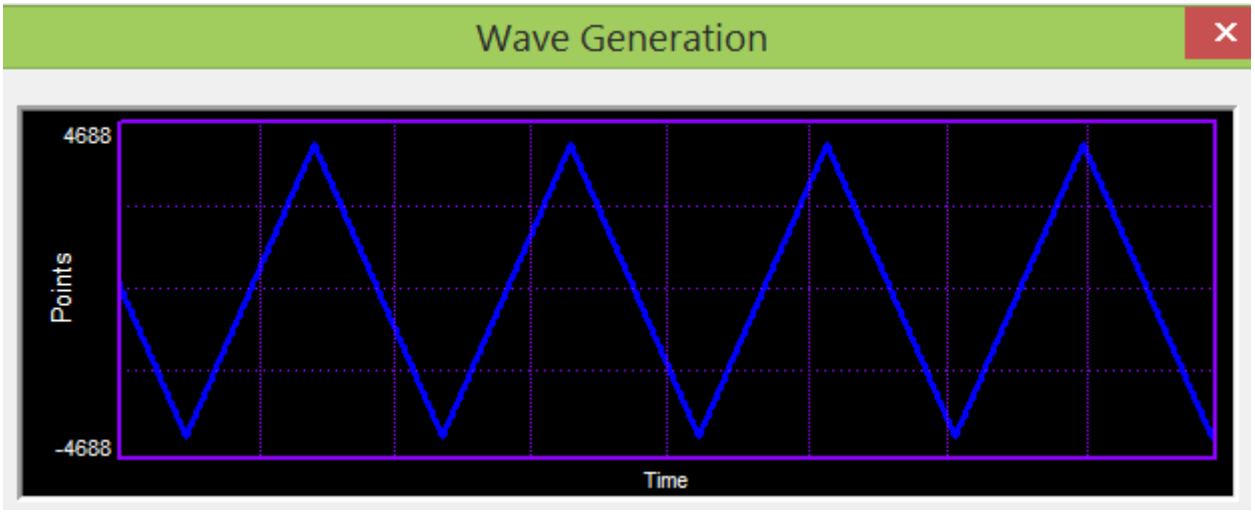


TRIANGULAR WAVE GENERATION:

```
DATA .SET 0H
     .mmregs
     .text
START: STM #140H,ST0            ;initialize the data page pointer
          RSBX CPL               ;make the processor to work using DP
```

```
        NOP
        NOP
        NOP
        NOP
REP: ST #0H,DATA            ;initialize the value as 0h
 INC: LD DATA,A             ;increment the value
      ADD #1H,A
       STL A,DATA
       PORTW DATA,04H       ;send the value to the dac
       CMPM DATA,#0FFFH          ;repeat the loop until the value becomes 0fffh
       BC INC,NTC
DEC: LD DATA,A              ;decrement the value
      SUB #1H,A
      STL A,DATA
      PORTW DATA,04H        ;send the value to the dac
      CMPM DATA,#0H
      BC DEC,NTC            ;repeat the loop until the value becomes 0h
      B REP                 ;repeat the above
```

Output:

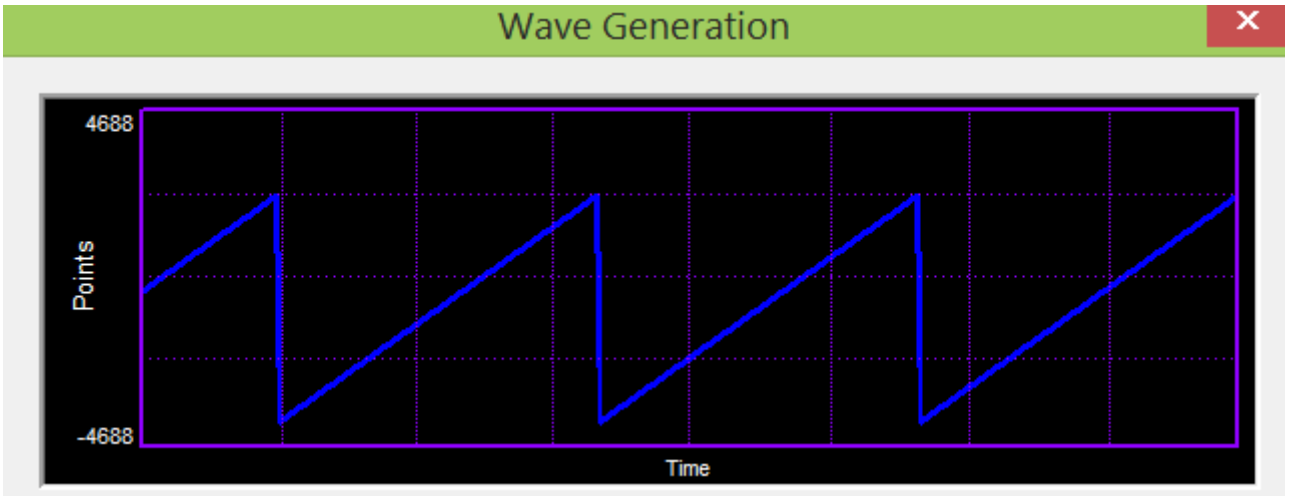


SAWTOOTH WAVE GENERATION:

```
 DATA .SET 0H
     .mmregs
     .text
START: STM #140H,ST0            ;initialize the data page pointer
       RSBX CPL                  ;make the processor to work using DP
       NOP
       NOP
       NOP
```

```
        NOP
REP: ST #0H,DATA                ;initialize the value as 0h
INC: LD DATA,A
ADD #1H,A                       ;increment the value
STL A,DATA
PORTW DATA,04H                  ;send the value to the dac
CMPM DATA,#0FFFH                ;repeat the loop until the value becomes 0fffh
BC INC,NTC
 B REP                          ;repeat the above
```

Output:



Result: square, triangular and sawtooth waveforms were successfully generated using the TMS320C5416 Digital Signal Processor. The characteristics of the generated waveforms were observed and validated using an oscilloscope.

EX NO:14	Design and Implementation of Butterworth IIR Filter

Aim:

To design and implement a 2nd-order Low Pass Butterworth IIR Filter with a cutoff frequency of 4 kHz and a sampling frequency of 62 kHz, and to analyze its frequency response to validate the filter's performance.

Hardware and Software Required:

DSP – TMS320VC5416-5416 Board
Cpu
ME Universal Debugger DSP - 5416

Procedure:

Connection is checked by the following procedure.
After connecting the serial port cable, switch on the kit and reset the kit.
Select the Menubar-Serial > Port Settings > Auto Detect
If connection (PC to kit) is improper, User can see the following window and try to connect it properly by the conditions until find the message Vi Universal debugger found in COMI
After checking the port connection, create new project for working with a program by using following procedure.

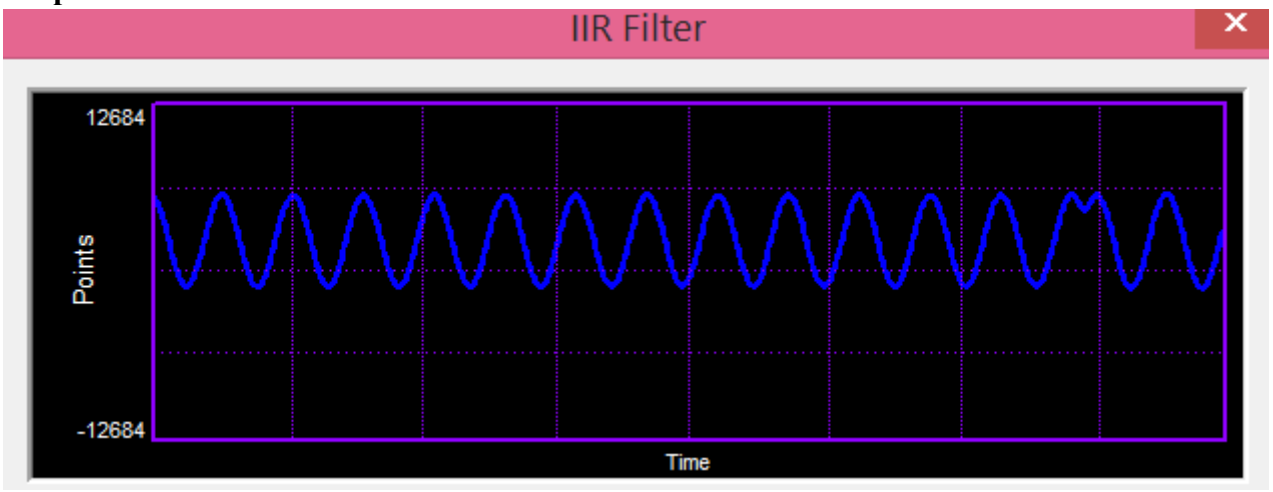
Generally user has to follow the path is C\Vi Universal Debugger VSK-5416/My Project
Before working with a program, User has to follow certain conditions,
1. Create a project and name it
2. Create a file and start to type a program, save it by project name
3. Add program file to the project
4. Add command file (Micro5416) to the project
5. Save project
6. Build it and
7.Download the program to the VSK-5416 kit

```
*****************************************************************************
                        LOW PASS FILTER (IIR)
;*****************************************************************************
*    butterworth filter
* IIR Filter, Created by Filter Design Package,
* (c) 2005, Vi Microsystems Pvt. Ltd.,
* Approximation Type: ButterWorth
* Filter Type: Low Pass Filter
* Filter Order: 2
* Sampling Frequency: 62
* Cutoff frequency in KHz = 4 .000000
```

```
;Variable declaration for the filter
XN          .SET    10H
XNM1        .SET    11H
XNM2        .SET    12H
YN          .SET    13H
YNM1        .SET    14H
YNM2        .SET    15H
SUM         .SET    16H
; Filter              assignmen
coefficient         t
A1          .SET    0E903H
A2          .SET    0905H
B0          .SET    082H
B1          .SET    0104H
B2          .SET    082H
      .mmregs
      .tex
t
START:
        STM     #0140H,ST0      ; data page initialization to page
                                140
        RSBX    CPL             ; reset the compiler mode bit
        RSBX    FRCT            ; reset the fractional mode bit
        NOP                     ; wait for the reset action
        NOP
        NOP
;initialize x(n),x(n-1),x(n-2),y(n),y(n-1),y(n-2)
; make all the variables to initially
        zero ST #0H,XN
        ST      #0H,XNM1
        ST      #0H,XNM2
        ST      #0H,YN
        ST      #0H,YNM1
        ST      #0H,YNM2
REPEAT:

        PORTR   6,0             ; give the start of conversion for adc
        nop                         ; wait the some delay to conversion
        nop
        nop
        nop
        PORTR   4,0             ; read the sampled data from the port
        LD      0,A             ; load it the accumulator
        AND     #0FFFH,A        ; extract the 12 bit alone
        XOR     #0800H,A        ; to correct 2's complement
        SUB     #0800H,A        ; convert the data to processor format
        STL     A,XN            ; store the sampled data to XN
```

```
LD      #0H,B          ; initialize the B reg to 0 for result
LD      XN,A           ; get the XN to accumulator
STLM    A,T            ; load it to the t reg
MPY     #B0,A          ; multiply the b0 coeff to the accumulator
LD      A,B            ; load the content of B reg to accumulator
LD      XNM1,A      ; get the XNM1 to accumulator
STLM    A,T            ; load it to T reg
MPY     #B1,A          ; multiply the b0 coeff to the accumulator
ADD     A,B            ; sum the current output with the B reg
LD      XNM2,A      ; get the XNM2 to accumulator
STLM    A,T            ; store the content of A reg to T reg
STLM    A,T            ; multiply the b2 coeff to the accumulator
MPY     #B2,A          ; sumup the current output with the B reg
ADD     A,B
LD      YNM1,A      ; get the YNM1 to accumulator
STLM    A,T            ; store the content of A reg to T reg
MPY     #A1,A          ; multiply the A1 coeff to the accumulator
SUB     A,B           ; subtract the current output from the B reg
LD      YNM2,A      ; get the YNM2 to accumulator
STLM    A,T            ; store the content of A reg to T reg
MPY     #A2,A          ; multiply the A1 coeff to the accumulator
SUB     A,B           ; subtract the current output from the B reg
SFTL    B,-12          ; the resultant value in the B reg is shifted
STL     B,YN           ; right by 12 bits store the final output to
LD      YNM1,A       ; the variable YN exchange YNM1 to YNM2
STL     A,YNM2
LD      YN,A           ; exchange YN to YNM1
STL     A,YNM1
LD      XNM1,A         ; exchange XNM1 to XNM2
STL     A,XNM2
LD      XN,A           ; exchange XN to XNM1
STL     A,XNM1
LD      YN,A           ; load the final output data to accumulator
ADD     #800H,A        ; add the offset value
STL     A,YN           ; store the result in YN
PORTW   YN,04H         ; out the output to the DAC
B       REPEAT         ; branch to continue
```

Output:



Result: A 2nd-order Low Pass Butterworth IIR Filter with a cutoff frequency of 4 kHz and a sampling frequency of 62 kHz was successfully designed and implemented.

EX NO:15	Design and Implementation of FIR Filter using Rectangular Window

Aim: To design and implement a High Pass Butterworth IIR Filter with a sampling frequency of 43 kHz, a cutoff frequency of 2 kHz, and filter order N=2N = 2N=2, and to analyze its frequency response to validate its performance.

Hardware and Software Required:

DSP – TMS320VC5416-5416 Board
Cpu
ME Universal Debugger DSP - 5416

Procedure:

Connection is checked by the following procedure. After connecting the serial port cable, switch on the kit and reset the kit. Select the Menubar-Serial > Port Settings > Auto Detect
If connection (PC to kit) is improper, User can see the following window and try to connect it properly by the conditions until find the message Vi Universal debugger found in COMI
After checking the port connection, create new project for working with a program by using following procedure.

Generally user has to follow the path is C\Vi Universal Debugger VSK-5416/My Project
Before working with a program, User has to follow certain conditions,

1. Create a project and name it
2. Create a file and start to type a program, save it by project name
3. Add program file to the project
4. Add command file (Micro5416) to the project
5. Save project
6. Build it and
7.Download the program to the VSK-5416 kit

```
;*************************************************************************
                        HIGH PASS FIR FILTER
;*************************************************************************
;Sampling freq      : 43khz
;Cut-off freq       : 2khz
;N                   52
;Filter type        : High pass filter
;Window type        : Rectangular
;Program Description:
;1. Make all the x(n) zero initially
;2. Read the data from the adc.
;3. Store the adc data in x(0)
;4. Make the pointer to point the x(n_end)
;5. Perform the convolution of x(n) and the coefficients h(n) using
```

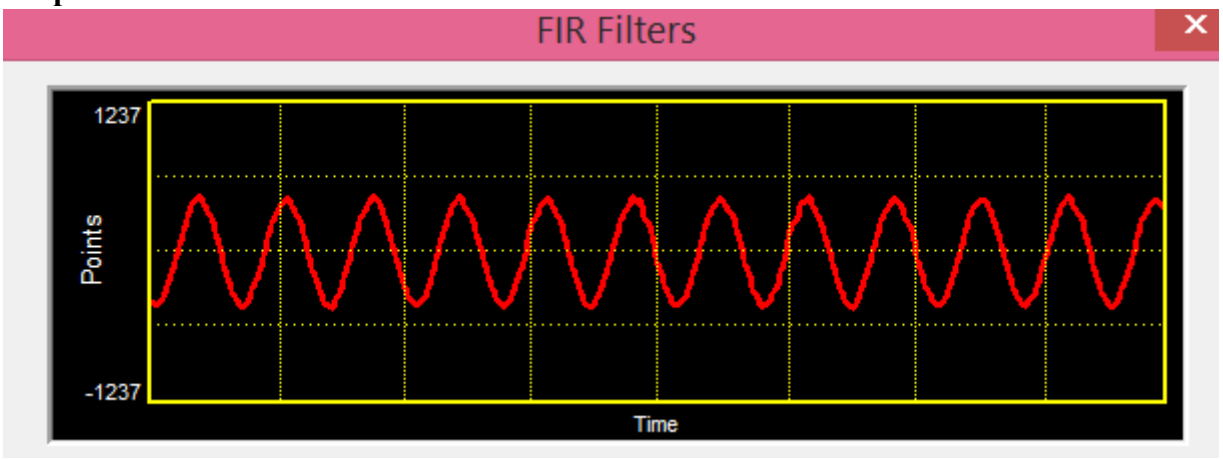
```
; MACD instruction.
;6. Send the convolution output to the dac
;7. Repeat from step 2.
        .mmregs
        .tex
t START:
        STM     #01h,ST0        ;initialize the data page pointer
        RSBX    CPL             ;Make the processor to work using
                                DP
        RSBX    FRCT            ;reset the fractional mode bit
        NOP
        NOP
;*****loop to make all x(n) zero initially*****
        STM     #150H,AR1       ;initialize ar1 to point to
        x(n) LD #0H,A           ;make acc zero
        RPT     #34H
        STL     A,*AR1+         ;make all x(n) zero
;*****to read the adc data and store it in x(0)*****
LOOP:
        PORTR   06,0            ;start of
conversion CHK_BUSY:
        ; PORTR  07,0           ;check for busy
        ; BITF   0,#20H
        ; BC     CHK_BUSY,TC
        PORTR   04,0            ;read the adc data
        LD      0,A
        AND     #0FFFH,A        ;AND adc data with 0fffh for 12 bit
                                adc
        XOR     #0800H,A        ;recorrect the 2's complement adc data
        SUB     #800H,A         ;remove the dc shift
        STM     #150H,AR1       ;initialize ar1 with x(0)
        STL     A,*AR1          ;store adc data in x(0)
```

```
        STM     #183H,AR2       ;initialize ar2 with x(n_end)
;*****start of convolution*****

        LD      #0H,A           ;sum is 0 initially
        RPT     #33H
        MACD    *AR2-,TABLE,A   ;convolution process
        STH     A,1,0H
        LD      0H,A
        ADD     #800H,A         ;add the dc shift to the convolution
                                output
        STL     A,1H
        PORTW   1H,04H          ;send the output to the dac
        B       LOOP

TABLE:

      .word   0FCEFH
      .word   62H
      .word   0FD50H
      .word   14AH
      .word   0FE1BH
      .word   28FH
      .word   0FF11H
      .word   3E5H
      .word   0FFD1H
      .word   4ECH
      .word   0FFF5H
      .word   54FH
      .word   0FF28H
      .word   4DAH
      .word   0FD38H
      .word   398H
      .word   0FA2EH
      .word   1DDH
      .word   0F627H
      .word   55H
      .word   0F131H
      .word   4BH
      .word   0EA6DH
      .word   568H
      .word   0D950H
      .word   459EH
      .word   459EH
      .word   0D950H
```

.word 568H

```
.word   0EA6DH
.word   4BH
.word   0F131H
.word   55H
.word   0F627H
.word   1DDH
.word   0FA2EH
.word   398H
.word   0FD38H
.word   4DAH
.word   0FF28H
.word   54FH
.word   0FFF5H
.word   4ECH
.word   0FFD1H
.word   3E5H
.word   0FF11H
.word   28FH
.word   0FE1BH
.word   14AH
.word   0FD50H
.word   62H
.word   0FCEFH
```

Output:



Result: A 2nd-order High Pass Butterworth IIR Filter with a cutoff frequency of 2 kHz and a sampling frequency of 43 kHz was successfully designed and implemented.